

Matr. N INLM03/23286



UNIVERSITÀ CAMPUS BIO-MEDICO DI ROMA

**FACOLTÀ DIPARTIMENTALE DI INGEGNERIA
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA DEI SISTEMI
INTELLIGENTI**

**IMPLEMENTAZIONE E
OTTIMIZZAZIONE DELL'ALGORITMO
DI SHOR IN AMBIENTI QUANTISTICI
RUMOROSI MEDIANTE MODELLI DI
INTELLIGENZA ARTIFICIALE**

Relatore

Prof. Paolo Soda

Laureando

Claudio Dragotta

Correlatore

Ing. Floriano Caprio

ANNO ACCADEMICO 2025/2026

*Alla mia famiglia,
per il sostegno incondizionato e l'amore che non è mai venuto meno.
A me stesso, per la determinazione e la perseveranza con cui ho affrontato questo
percorso.*

Abstract

Introduzione

L'algoritmo di Shor [1] fattorizza un intero composto in tempo polinomiale, $O((\log N)^3)$, contro il tempo sub-esponenziale del *general number field sieve* [2], il miglior algoritmo classico noto. È questo divario a minacciare RSA [3], ma è asintotico e i dispositivi *Noisy Intermediate-Scale Quantum* [4] non lo realizzano: il rumore distrugge la struttura periodica su cui l'algoritmo si fonda, le dimostrazioni restano confinate a moduli didattici [5] e RSA-2048 richiederebbe venti milioni di qubit fisici [6]. Restano due rimedi, la mitigazione a valle e la correzione d'errore durante il calcolo, ai quali un numero crescente di lavori applica modelli appresi, spesso senza un termine di confronto che isoli il contributo del modello da quello della regola decisionale.

Il lavoro riprende un progetto preesistente — Shor in Qiskit e una prima campagna rumorosa su $N = 15$ — e lo rigenera dopo che una verifica per truth table ne ha invalidato il moltiplicatore modulare controllato, il blocco che realizza $|y\rangle \mapsto |ay \bmod N\rangle$. Chiede quindi *in quale punto della pipeline un modello appreso migliori davvero la resa di Shor in ambiente rumoroso*, e con la stessa tecnologia e lo stesso impianto statistico ne confronta due: un classificatore dell'istogramma di output e un decoder addestrato sulle sindromi di un codice correttore. La risposta ha due facce, negativa a valle e positiva ma solo residuale sulla sindrome. Il contributo è metodologico: disegno appaiato con *ablazione* — un braccio di controllo che esegue la sola regola decisionale, senza il modello —, separazione fra metriche di correzione d'errore e successo di Shor, artefatti riproducibili (seed, manifest e hash del circuito).

Materiali e Metodi

Formalizzazione matematica del problema

Per N composto, dispari e non potenza di un primo, e a coprimo con N , la fattorizzazione si riduce alla ricerca dell'*ordine* $r = \text{ord}_N(a) = \min\{k \in \mathbb{N}^+ : a^k \equiv 1 \pmod{N}\}$: se r è pari e $a^{r/2} \not\equiv -1 \pmod{N}$, allora $\gcd(a^{r/2} \pm 1, N)$ è un fattore non banale, con probabilità almeno $1/2$ su a casuale [7]. La parte quantistica stima r per *phase estimation* dell'unitario $U_a|y\rangle = |ay \bmod N\rangle$: con n qubit di conteggio e $M = 2^n$, dopo la trasformata di Fourier quantistica inversa la misura si concentra sui multipli di M/r , con dispersione descritta dal nucleo di Fejér. Il post-processing Φ è deterministico: frazioni continue di y/M , esatte quando $M \geq N^2$, poi parità e massimo comun divisore. Detto $\mathcal{U}$ l'insieme degli y da cui Φ ricava un fattore non

banale, ogni esecuzione è delimitata da due costanti in forma chiusa,

$$P_{\text{id}} = \sum_{y \in \mathcal{U}} P(y) = \frac{3}{4}, \quad P_{\text{unif}} = \frac{|\mathcal{U}|}{M} = \frac{63}{256} \simeq 0,2461, \quad (1)$$

per l'istanza canonica $N = 15$, $a = 7$, $n = 8$, dove $r = 4$ divide M e P è supportata sui quattro picchi $\{0, 64, 128, 192\}$: di questi solo $y = 0$ non porta informazione sull'ordine, da cui i tre quarti. Le due costanti fissano in anticipo la scala di lettura: P_{id} è la resa per shot della distribuzione ideale; P_{unif} è il *pavimento uniforme*, la resa attesa quando il rumore rende equiprobabili tutte le M celle, non nulla perché il post-processing accetta 63 esiti su 256. Non sono limiti universali: sono i riferimenti ideale e uniforme, e toccare il pavimento segnala la perdita del segnale utile, non l'impossibilità di fattorizzare.

Modello di rumore, scenari e mitigazione a valle

Il circuito è compilato deterministicamente (ottimizzazione 2 e seed registrato) nella base hardware $\{r_z, \sqrt{X}, X, CX\}$, in cui r_z è una rotazione virtuale attorno a z , e il rumore è modellato su quelle porte. Dopo ogni gate g si compone un canale depolarizzante di parametro λ con il rilassamento termico, descritto dai tempi T_1 (rilassamento energetico) e T_2 (perdita di coerenza di fase) e applicato in proporzione alla durata di g : r_z ha durata nulla nel modello, $\sqrt{X}/X$ dura 50 ns e CX 300 ns. La lettura è una matrice di confusione simmetrica di parametro p_{ro} ; su S shot l'istogramma normalizzato $\hat{h}$ è la frequenza campionaria multinomiale della distribuzione così letta. Due preset definiscono gli scenari confrontati: **UC1** (riferimento) con $\lambda_{1q} = 10^{-3}$, $\lambda_{2q} = 10^{-2}$, $T_1 = 100 \mu\text{s}$, $T_2 = 80 \mu\text{s}$, $p_{\text{ro}} = 0,02$; **UC2** (stress) con $\lambda_{1q} = 5 \times 10^{-3}$, $\lambda_{2q} = 5 \times 10^{-2}$, $T_1 = 50 \mu\text{s}$, $T_2 = 30 \mu\text{s}$, $p_{\text{ro}} = 0,05$: errori di gate cinque volte più intensi e tempi di coerenza dimezzati. Sono scenari uniformi illustrativi, non calibrazioni di una QPU. Come termine di paragone a valle si valuta la *zero-noise extrapolation* [8]: il circuito gira con il rumore scalato di $\lambda = 1, 2, 3$ e l'istogramma è estrapolato a $\lambda \rightarrow 0$ (lineare su due livelli, Richardson su tre), al prezzo di altrettante repliche.

Politiche decisionali, modello appreso e stimando statistico

Su $\hat{h}$ si definisce l'ordinamento totale $\prec$ per conteggio decrescente, con pareggi risolti da una priorità deterministica seeded (SHA-256); la politica π_K prova i primi K

candidati secondo $\prec$. Il Metodo 1 è π_1 , l'ablazione è π_4 , il Metodo 2 applica un classificatore $g_\theta(\hat{h}) \in \{0, 1\}$ ed esegue π_4 solo se il filtro è positivo. Il classificatore riceve $\hat{h}$ e l'etichetta binaria $Y_1 = \mathbf{1}[\pi_1 \text{ ha successo}]$; è addestrato su 2000 istogrammi per scenario, campionati entro il $\pm 50\%$ attorno al preset, con partizione stratificata 60/20/20. Fra random forest, *support vector machine* a kernel RBF calibrata e perceptrone multistrato, la selezione per F1 sulla validation sceglie in entrambi gli scenari la *support vector machine*, valutata una sola volta sul test. Per ogni replica si osserva il numero di iterazioni $\mathcal{M}^{(\pi)} = \min\{m \geq 1 : \pi \text{ riesce all'iterazione } m\}$, posto uguale a $\mathcal{M}_{\max} + 1$ in caso di fallimento, con $\mathcal{M}_{\max} = 50$: una sentinella che codifica esplicitamente la censura. Il carattere calligrafico tiene il conteggio delle iterazioni distinto dalla dimensione $M = 2^n$ del registro. Lo stimando dichiarato a priori, $\rho = \bar{\mathcal{M}}_1/\bar{\mathcal{M}}_2$, resta la misura di effetto; l'inferenza è però affidata alle differenze appaiate $D_i = \mathcal{M}_i^{(A)} - \mathcal{M}_i^{(B)}$, perché un rapporto di medie su una variabile discreta e censurata non ammette test esatto: test dei ranghi con segno di Wilcoxon (zeri trattati con il metodo di Pratt), H_0 di simmetria attorno a zero, $\alpha = 0,05$ e correzione di Holm entro famiglia. Poiché ogni esecuzione Monte Carlo dà un istogramma diverso anche a parametri identici, le tre strategie ne ricevono uno identico a ogni coppia (replica, iterazione): D_i confronta le regole sulla stessa realizzazione, senza variabilità spuria. Un audit separato scarta la variante Y_{16} dell'etichetta — successo entro i primi sedici candidati — perché degenerata a classe unica.

Correzione d'errore: formalismo, metriche e decodifica appresa

Un codice stabilizzatore protegge l'informazione misurando ripetutamente osservabili che lasciano invariati gli stati codificati (il sottogruppo abeliano $\mathcal{S} \subset \mathcal{P}_n$ del gruppo di Pauli): l'esito di quelle misure è la *sindrome* $\sigma(E)$ dell'errore E , e un decoder $\mathcal{D}$ è la regola che dalla sindrome inferisce la correzione. Il decoder fallisce quando la correzione riporta lo stato nel codice applicandovi però un operatore logico non banale, cioè un errore che nessuna sindrome poteva rivelare; la probabilità di quell'evento è il *logical error rate* $p_L(p, d)$, dove d è la distanza del codice, il peso minimo di un errore non rivelabile. Si simulano il codice a ripetizione, con la forma chiusa $p_L = 3p^2 - 2p^3$ come test di correttezza; il codice di Steane [9], con verifica della corrispondenza sindrome $\rightarrow$ errore sui 21 casi e stima di γ in $p_L \propto p^\gamma$; il surface code ruotato [10] sotto rumore *circuit-level*, decodificato per matching di peso minimo (MWPM) e

descritto dalla legge di scala

$$p_L(p, d) = A (p/p_{\text{th}})^{\lfloor (d+1)/2 \rfloor}, \quad (2)$$

con A e p_{th} adattati ai punti sotto soglia per $d = 3, 5, 7, 9$, così da stimare la soglia indipendentemente dall'incrocio delle curve. Il decoder appreso è un percettore multistrato f_θ sugli stessi *detection events*, le variazioni di sindrome fra cicli consecutivi, confrontato con il matching sia come sostituto sia nella forma ibrida $\mathcal{D}_{\text{hyb}}(\sigma) = \mathcal{D}_{\text{MWPM}}(\sigma) \oplus \mathbf{1}[f_\theta(\sigma) > t]$, in cui la rete, quando segnala che il matching sta sbagliando, ne ribalta la decisione applicando l'operatore logico; la soglia t è scelta in validazione e la significatività dal test di McNemar.

Proxy fenomenologico per la sensibilità di Shor

Dopo ogni gate compilato si applica un canale di Pauli isotropo, che con probabilità p_g sostituisce l'identità con un errore di Pauli non banale scelto uniformemente, ossia $\lambda_1 = \frac{4}{3}p_g$ e $\lambda_2 = \frac{16}{15}p_g$ in forma depolarizzante. È indicato p_g , non p_L : è la probabilità di un Pauli non identità *per gate compilato*, non il tasso logico per ciclo di sindrome della sezione *Correzione d'errore*, e le due unità non sono convertibili senza un modello architetturale esplicito. Su una griglia preregistrata di p_g si misura il successo per singola misura P_s , da cui la cumulata $P(k) = 1 - (1 - P_s)^k$ su k tentativi. Non è una compilazione fault-tolerant: ne risulta un'analisi di sensibilità, non una previsione di prestazione.

Strumenti, verifica e validazione

Qiskit/Aer [11] gestisce i circuiti generali, Stim [12] e PyMatching [13] quelli stabilizzatori e scikit-learn [14] i modelli appresi; ambiente e versioni sono registrati in un formato JSON versionato (input, seed, manifest e hash della netlist). Una demo web separa esecuzione ideale e rumorosa; codice e artefatti sono resi disponibili con la tesi (demo: <https://shor-demo-6knp.onrender.com>). L'aritmetica di Beauregard [15], realizzata anche per $N = 21$ e $N = 35$, è validata nel solo caso ideale: $N = 15$ resta l'unica istanza rumorosa. La verifica è *funzionale* (truth table, azzeramento delle ancille e confronto con la phase estimation teorica: distanza di variazione totale $1,38 \times 10^{-9}$ per $N = 21$), *di identità* (profondità, conteggi e hash) e *statistica* (ipotesi e griglie preregistrate, seed separati, intervalli di Wilson, Bonferroni sulla monotonia,

Holm entro famiglia e censura esplicita). L’ablazione sugli stessi dati resta il controllo causale.

Risultati

Fase 1. Sul circuito verificato ($N = 15$, $a = 7$, profondità 412, 224 CX, hash registrato) i classificatori raggiungono le prestazioni ipotizzate a priori ($F1 = 0,919$ e $0,923$; $AUC = 0,965$ e $0,958$), ma l’ablazione appaiata mostra che il guadagno non è loro. Su 30 repliche per scenario, con successo 30/30 in ogni braccio, la sola ricerca π_4 porta le iterazioni al minimo ($\bar{\mathcal{M}} = 1,00$ contro 1,37 in UC1 e 1,50 in UC2; $p_{\text{Holm}} = 0,0024$ e $0,0015$), mentre il Metodo 2 non migliora la baseline (1,50 e 1,37; $p_{\text{Holm}} = 0,7314$ e $0,0874$) e peggiora la propria ablazione ($p_{\text{Holm}} = 0,0033$ e $0,0096$). Nemmeno la zero-noise extrapolation migliora la baseline, e triplica il costo di campionamento.

Fase 2. I codici riproducono le previsioni a priori, con la stima di p_{th} dall’Eq. (2) concorde con l’incrocio delle curve: la ripetizione a $d = 3$ accorda con $p_L = 3p^2 - 2p^3$ entro l’errore statistico; Steane $[[7, 1, 3]]$ dà $\gamma = 1,94$ e pseudo-soglia (il rumore oltre il quale, a distanza fissata, codificare non conviene) $p \simeq 0,08$; il surface code per $d = 3-9$ dà $p_{\text{th}} = 0,86\%$ (Z) e $0,81\%$ (X), prefattore $A = 3,5 \times 10^{-2}$ e residui di RMS 0,08 in $\ln p_L$; la decodifica appresa non vince mai come sostituto del matching e, come correttore ibrido, riduce p_L del 18–21% a $d = 3$ *soltanto in presenza di errori correlati* — crosstalk fra qubit dati adiacenti, non rappresentabile nel grafo di matching —, dell’1–3% a $d = 5$ e per nulla a $d = 7$.

Chiusura quantitativa. Nel proxy fenomenologico descritto sopra — in unità di p_g , non del p_L appena riportato — gli estremi della curva di sensibilità coincidono con le costanti dell’Eq. (1): il punto senza rumore ($p_g = 0$) vale 0,7489 [0,7459; 0,7518] e stima $P_{\text{id}} = 3/4$; il plateau è già raggiunto a $p_g \simeq 0,1$ e vale 0,2459 a $p_g = 0,5$, cioè il pavimento uniforme $P_{\text{unif}} = 63/256 = 0,2461$. La curva non descrive un algoritmo che fallisce, ma uno che degrada verso l’estrazione casuale, con entrambi gli asintoti fissati in anticipo dalla teoria.

Discussione e Conclusioni

La domanda di localizzazione ha una risposta a due facce. A valle dell’esecuzione l’ipotesi principale, $\rho \gg 1$, è *smentita* dai suoi stessi dati: il classificatore raggiunge

le soglie di F1 e AUC fissate a priori e resta inutile, perché scarta istogrammi che π_4 avrebbe risolto. Sulla sindrome, dove il dato conserva una struttura locale più ricca del solo istogramma finale, la stessa tecnologia guadagna, ma solo nella forma ibrida — la rete affianca il matching invece di sostituirlo — e solo finché il rumore contiene correlazioni che il decoder analitico non rappresenta. Il criterio unificante è quindi verificabile: un modello appreso aggiunge valore quando sfrutta informazione che il metodo di riferimento lascia inutilizzata, non quando ne replica la regola decisionale.

L’autovalutazione riguarda il metodo: senza il braccio π_4 la riduzione da 1,37 a 1,00 sarebbe stata attribuita al classificatore, un falso positivo del tutto presentabile; l’audit dell’etichetta e la verifica per truth table del moltiplicatore, che ha imposto la rigenerazione dell’intera campagna, sono stati più informativi delle metriche di accuratezza.

I limiti delimitano ogni conclusione: il rumore della campagna su Shor è uniforme, indipendente e stazionario, senza topologia, crosstalk, leakage o deriva; $N = 15$ è un’istanza didattica con post-processing permissivo; le stime QEC valgono entro il modello Clifford–Pauli, indicato come potenzialmente ottimistico [16]; l’analisi di sensibilità è un proxy per gate, non una simulazione fault-tolerant. Rispetto al piano, $N = 21$ e $N = 35$ restano validati solo idealmente ed esclusi per costo dalle campagne rumorose, e l’accelerazione GPU prevista non è stata impiegata.

Gli sviluppi futuri discendono dai limiti — mappatura per operazione logica, rumore *channel-first*, decoder ricorrenti o a grafo [17] — ma la ricaduta più immediata è metodologica: ablazione appaiata, contratto interpretativo fra unità diverse e artefatti versionati formano un protocollo riutilizzabile per valutare qualunque mitigazione appresa del rumore quantistico.

Indice

Abstract	i
Elenco delle figure	xiii
Elenco delle tabelle	xv
Elenco degli Acronimi	xviii
1 Introduzione	2
1.1 Il Limite del Calcolo Classico	2
1.2 Motivazione e Contributo del Presente Lavoro	3
1.3 Struttura della Tesi	4
2 Obiettivi e Piano Sperimentale	5
2.1 Obiettivo e Perimetro del Lavoro	5
2.2 Domande di Ricerca e Ipotesi	6
2.3 Contributo e Disegno Sperimentale	7
2.4 Campagne Rumorose e Validazioni Ideali	7
2.4.1 Razionale per la Scelta delle Istanze	8
2.5 Metriche di Valutazione	8
2.5.1 Post-processing dell'Output	8
2.5.2 Correzione d'Errore	8
2.6 Approccio Metodologico	9
3 L'Algoritmo di Shor	10
3.1 Richiami di Calcolo Quantistico	10
3.1.1 Qubit, misura ed entanglement	10
3.1.2 Operazioni essenziali e matrici di Pauli	11
3.2 Fattorizzazione e ricerca del periodo	12

3.3	QFT e stima quantistica del periodo	13
3.3.1	Quantum Fourier Transform	13
3.3.2	Phase estimation e periodicità	14
3.3.3	Ruolo dell'entanglement	14
3.4	Algoritmo completo	15
3.5	Complessità e limiti implementativi	15
3.6	Perimetro sperimentale	16
4	Rumore Quantistico nei Sistemi NISQ	17
4.1	Dal modello ideale alla macchina fisica	17
4.2	I vincoli hardware rilevanti	17
4.3	Fonti fisiche e canali adottati	18
4.3.1	Rilassamento e perdita di fase	18
4.3.2	Canale depolarizzante	19
4.3.3	Lettura e rumore correlato	19
4.4	Perché Shor è sensibile al rumore	20
5	Strategie per la Riduzione del Rumore Quantistico	21
5.1	Panoramica e criteri di scelta	21
5.2	Machine learning classico su dati quantistici	22
5.3	Correzione quantistica e decodifica	22
5.3.1	Il modello appreso come componente del decoder	23
5.4	Strategia sperimentale della tesi	23
6	Metodo e Implementazione	24
6.1	Strumenti e Ambiente di Simulazione	24
6.2	Disegno e perimetro sperimentale	25
6.3	Pipeline appaiata	25
6.4	Modello di rumore e controlli	26
6.5	Regole decisionali e analisi statistica	26
6.6	Implementazione e tracciabilità	27
6.7	Rumore e strategie della prima fase	27
6.8	Raccolta dati e implementazione QEC	28
7	Verifica Sperimentale dell'Ipotesi Iniziale: dal Classificatore ML alla Strategia TOP-K	29
7.1	Perimetro e controlli di validità	29

7.2	Ablazione M1–TOP-4–M2	30
7.3	Robustezza e confronti secondari	31
7.4	Bilancio e passaggio alla correzione d'errore	31
8	La Correzione d'Errore Quantistico: dal Codice a Ripetizione al Codice di Steane	32
8.1	Dalla Ridondanza Ingenua alla Codifica	32
8.2	Il Ciclo della Correzione: Stabilizzatori, Sindrome, Decoder	33
8.3	Il Panorama dei Codici e la Scelta Metodologica	34
8.4	Laboratorio I: il Codice a Ripetizione	34
8.4.1	Il codice bit-flip a 3 qubit	34
8.4.2	Risultati del laboratorio repetition	35
8.5	Laboratorio II: il Codice di Steane $[[7, 1, 3]]$	36
8.5.1	Struttura del codice	36
8.5.2	Protocollo di laboratorio	36
8.5.3	Risultati del laboratorio Steane	36
9	Il Surface Code e la Stima dell'Errore Logico	38
9.1	Il Codice su Griglia: Concetto Operativo	38
9.2	La Decodifica: Minimum Weight Perfect Matching	39
9.3	Disegno Sperimentale	40
9.4	Risultati	40
9.5	Oltre il Modello di Pauli: Errori Coerenti e Leakage	42
9.6	La Decodifica Appresa: Sintesi della Campagna	42
10	Analisi di Sensibilità: Shor sotto un Proxy di Errore per Gate	44
10.1	L'architettura a quattro livelli	44
10.2	Definizione operativa del proxy	45
10.3	Il contesto delle stime fault-tolerant	45
10.4	Risultati M8 e M13	45
10.4.1	Discussione	47
11	Sviluppi Futuri	48
11.1	Scalabilità e Validazione su Hardware Reale	48
11.2	Dalla QEC a uno Shor Fault-Tolerant	49
11.3	Trasferibilità del Post-processing e dell'Apprendimento	49

12 Conclusioni	50
12.1 Le Domande di Ricerca	50
12.2 Verifica delle Ipotesi	51
12.3 Contributi del Lavoro	52
12.4 Limiti	52
12.5 Il Criterio che Unifica il Lavoro	52
A Trattazione Teorica Estesa	54
A.1 Fondamenti del Calcolo Quantistico	54
A.1.1 Dalla Meccanica Classica alla Meccanica Quantistica	54
A.1.2 I Postulati della Meccanica Quantistica	55
A.1.3 Il Qubit	57
A.1.4 Sistemi Multi-Qubit e Prodotto Tensoriale	58
A.1.5 Porte Quantistiche	60
A.1.6 Circuiti Quantistici	63
A.1.7 Misura Quantistica	63
A.1.8 Interferenza Quantistica	63
A.1.9 Modello di Complessità Quantistica	64
A.2 L'Anatomia Fisica di una QPU	65
A.2.1 L'Anatomia Fisica di una QPU	65
A.3 L'Algoritmo di Shor — Approfondimenti e Stato dell'Arte	70
A.3.1 Sviluppi Recenti e Stato dell'Arte	71
A.3.2 Contesto: Confronto con l'Algoritmo di Grover	73
A.4 I Canali di Rumore: Derivazioni Complete	74
B Strumenti, Ambiente e Codice	76
B.1 Strumenti e Ambiente di Simulazione — Trattazione Estesa	76
B.1.1 Qiskit: il Framework Quantistico di Riferimento	76
B.1.2 Qiskit Aer: il Simulatore	78
B.1.3 Accelerazione GPU con Qiskit Aer	80
B.1.4 scikit-learn: la Libreria per il Classificatore	81
B.1.5 Ambiente di Esecuzione: WSL con Ubuntu	81
B.2 Codice Sorgente dei Componenti Implementati	81
C Contesto e Motivazione	90
C.1 Contesto Esteso dell'Introduzione	90
C.1.1 Dal Bit al Qubit: i Principi Fisici del Calcolo Quantistico	90

C.1.2	La Rivoluzione Algoritmica degli Anni Novanta	94
C.1.3	RSA e le Implicazioni Crittografiche dell'Algoritmo di Shor . .	95
C.1.4	Crittografia Post-Quantistica: la Risposta dell'Industria	98
C.1.5	Lo Stato dell'Arte: dall'Era NISQ ai Computer Fault-Tolerant	100
C.2	Dettaglio degli Obiettivi e del Piano Sperimentale	101
C.2.1	Motivazione: la Vulnerabilità della Crittografia Attuale	102
C.2.2	Requisiti Funzionali	103
C.2.3	Parametri di Input e Output	104
D	Fase 1: la Campagna Classica nel Dettaglio	106
D.1	Campagna Parametrica e Confronto con la Zero-Noise Extrapolation	106
D.1.1	Variazione di K	107
D.1.2	Variazione di λ_{2q}	107
D.1.3	Variazione degli Shot	108
D.1.4	Analisi Congiunta $K \times \lambda_{2q}$	108
D.1.5	Parametri Secondari	108
D.1.6	Livello di Ottimizzazione	109
D.1.7	Diagnostiche della struttura QPE	109
D.1.8	Confronto con Zero-Noise Extrapolation	110
D.2	Il Classificatore ML: Architettura, Addestramento e Metriche	112
D.2.1	L'Ipotesi alla Prova: il Classificatore ML	112
D.2.2	Audit delle Etichette: TOP-1 e TOP-16 Sono Problemi Diversi	113
E	Fase 2: la Campagna QEC nel Dettaglio	117
E.1	La Decodifica Appresa: la Campagna Completa	117
E.1.1	Disegno sperimentale	118
E.1.2	E1 — la sostituzione a parità di modello	119
E.1.3	E2 — quando il rumore esce dal modello del decoder	120
E.1.4	E3 — la rete come validatore della correzione	121
E.1.5	E4 — il decoder ibrido	122
E.1.6	E6 — e se il modello del decoder fosse semplicemente sbagliato?	128
E.1.7	E7 — e se si scegliesse semplicemente un decoder analitico migliore?	130
E.1.8	E8 — le due sorgenti si sommano davvero? E10 — il pavimento comune	133
E.1.9	E5 — il decoder è trasferibile fra dispositivi?	137

E.1.10 Bilancio	138
E.2 Compilazione Consapevole della Struttura: un'Esplorazione	139
E.3 Previsione e selezione non sono la stessa domanda	140
E.4 M11: pilota sui layout	140
E.5 M11b: readout effettivo dopo il routing	142
E.6 M13: TOP-4 sotto il proxy per gate	143
E.7 Conclusione e limiti	144

Bibliografia	145
---------------------	------------

Elenco delle figure

3.1	La sfera di Bloch	11
3.2	Periodicità di $7^x \bmod 15$	13
3.3	Circuito QFT su tre qubit	13
3.4	Circuito QPE per $N = 15$, $a = 2$	14
4.1	Localizzazione del rumore nella pipeline di Shor	20
6.1	Ogni istogramma è riutilizzato dalle tre strategie, eliminando differenze dovute a realizzazioni Monte Carlo diverse.	26
7.1	Iterazioni richieste da M1, TOP-4 e M2 nelle repliche appaiate dei due scenari.	30
8.1	Errore logico del codice a ripetizione	35
8.2	Errore logico del codice di Steane $[[7, 1, 3]]$	37
9.1	Comportamento a soglia del surface code	41
10.1	Shor sotto un proxy fenomenologico per gate	46
A.1	Circuito per lo stato di Bell $ \Phi^+\rangle$	59
A.2	Porte quantistiche a singolo e due qubit	62
A.3	Gerarchia delle classi di complessità quantistica	65
A.4	Anatomia schematica di una QPU superconduttiva a 5 qubit	66
A.5	La QPU reale: sistema integrato e criostati con cablaggio	67
A.6	Chip quantistico superconduttivo a 6 transmon con giunzioni di test	68
A.7	Il bordo di coerenza: budget di operazioni e rischio di misura	70
D.1	Repliche UC1 con massa massima e minima sui quattro picchi QPE teorici.	110

D.2	Il proxy di nessun evento decade più rapidamente della massa strutturata osservata sui picchi.	111
D.3	Curve ROC dei classificatori TOP-1 selezionati	113
D.4	Istogramma TOP-1 negativo ma recuperabile con TOP-4	114
D.5	Tetto combinatorio alla classe negativa	116
E.1	Decodifica appresa a parità di modello di rumore	119
E.2	Decodifica appresa in presenza di rumore correlato	121
E.3	Precisione e copertura del flag di validazione	122
E.4	Il decoder ibrido a confronto con matching e rete	124
E.5	Ortogonalità delle due sorgenti di guadagno	132

Elenco delle tabelle

2.1	Preset uniformi delle campagne rumorose N15; non rappresentano la calibrazione di una QPU.	7
4.1	Corrispondenza fra principali fonti di rumore, modelli e vulnerabilità dell'algoritmo.	18
5.1	Livelli di intervento contro il rumore e relativi vincoli.	21
6.1	Contratto essenziale delle campagne rumorose su $N = 15$	25
7.1	Confronto appaiato delle tre strategie, 30 repliche per scenario.	30
9.1	Disegno sperimentale surface code	40
12.1	Esito delle ipotesi nel perimetro sperimentale della tesi.	51
A.1	Confronto tra tecnologie di QPU: superconduttori vs ioni intrappolati	69
A.2	Confronto tra l'algoritmo di Shor e l'algoritmo di Grover	73
B.1	Confronto tra i principali framework per la simulazione quantistica con rumore.	78
C.1	Parametri principali del sistema sperimentale. λ segue la convenzione Aer e non coincide con la probabilità aggregata di Pauli non identità.	105
D.1	Sweep del numero di candidati	107
D.2	Sweep del parametro Aer λ_{2q}	107
D.3	Sweep degli shot	108
D.4	Griglia $K \times \lambda_{2q}$	108
D.5	Sweep di T_1/T_2	108
D.6	Sweep di λ_{1q}	109
D.7	Sweep del readout simmetrico	109

D.8	Sweep di <code>optimization_level</code>	109
D.9	Massa efficace sui picchi e proxy di nessun evento 2Q.	110
D.10	Confronto ZNE appaiato	111
D.11	Metriche TOP-1 sul test set indipendente (400 istogrammi per scenario)	112
D.12	Audit delle etichette TOP-1 e TOP-16	113
D.13	Distribuzione della moda su UC1	114
D.14	Tetto combinatorio alla classe negativa in funzione di K	115
E.1	Disegno sperimentale della decodifica appresa	118
E.2	Decodifica in presenza di rumore correlato	120
E.3	La rete come validatore della correzione del matching	122
E.4	Prestazioni del decoder ibrido	123
E.5	Larghezza dell'ingresso contro distanza del codice	126
E.6	Discriminazione e guadagno al crescere della larghezza dell'ingresso	127
E.7	Effetto di un modello di rumore mis-calibrato sul MWPM	129
E.8	BP+OSD contro MWPM e contro il decoder ibrido	131
E.9	Il residuo appreso costruito sopra BP+OSD	134
E.10	Sei decoder, un solo pavimento	135
E.11	Il pavimento alla prova di modelli più potenti	136
E.12	Trasferibilità del decoder appreso fra profili di rumore	137
E.13	Disegno del pilota sui layout	141
E.14	Stimandi del pilota M11	141
E.15	Disegno osservazionale M11b	142
E.16	Strategie descrittive di M11b	142
E.17	Endpoint preregistrati di M13	143

Elenco degli Acronimi

AUC Area Sotto la Curva ROC (*Area Under the Curve*)

BQP *Bounded-error Quantum Polynomial time*

CCNOT Controlled-Controlled NOT (porta di Toffoli)

CNOT Controlled-NOT

CPTP *Completely Positive Trace-Preserving*

CPU Unità di Elaborazione Centrale (*Central Processing Unit*)

CSS Calderbank-Shor-Steane

DEM *Detector Error Model*

DFT Trasformata Discreta di Fourier (*Discrete Fourier Transform*)

ECDSA *Elliptic Curve Digital Signature Algorithm*

FFT Trasformata Rapida di Fourier (*Fast Fourier Transform*)

GNFS *General Number Field Sieve*

GPU Unità di Elaborazione Grafica (*Graphics Processing Unit*)

HTTPS *HyperText Transfer Protocol Secure*

IBM International Business Machines

MLP Percettrone Multistrato (*Multi-Layer Perceptron*)

MWPM *Minimum Weight Perfect Matching*

NISQ *Noisy Intermediate-Scale Quantum*

NIST *National Institute of Standards and Technology*

NMR *Risonanza Magnetica Nucleare (Nuclear Magnetic Resonance)*

PQC *Post-Quantum Cryptography*

QEC *Correzione degli Errori Quantistici (Quantum Error Correction)*

QFT *Trasformata di Fourier Quantistica (Quantum Fourier Transform)*

QMA *Quantum Merlin-Arthur*

QML *Quantum Machine Learning*

QPE *Quantum Phase Estimation*

QPU *Unità di Elaborazione Quantistica (Quantum Processing Unit)*

RSA *Rivest-Shamir-Adleman*

SDK *Software Development Kit*

SSH *Secure Shell*

SVM *Support Vector Machine*

WSL *Windows Subsystem for Linux*

ZNE *Zero-Noise Extrapolation*

Capitolo 1

Introduzione

1.1 Il Limite del Calcolo Classico

Il progresso dell'informatica classica ha reso disponibili dispositivi sempre più potenti, ma non elimina due limiti distinti: la miniaturizzazione dell'hardware incontra vincoli fisici e, per alcuni problemi, il costo dei migliori algoritmi noti cresce troppo rapidamente con la dimensione dell'input. La fattorizzazione di grandi interi appartiene a questa seconda categoria ed è rilevante perché sostiene la sicurezza di sistemi crittografici come Rivest-Shamir-Adleman (RSA).

Il calcolo quantistico affronta tali problemi usando sovrapposizione, entanglement e interferenza come risorse computazionali. L'idea, anticipata da Feynman per la simulazione efficiente dei sistemi quantistici [18] e formalizzata da Deutsch nel modello di calcolatore quantistico universale [19], ha condotto ad algoritmi privi di un analogo classico efficiente noto. Tra questi, l'algoritmo di Shor mostra che la fattorizzazione può essere ricondotta alla stima di un periodo ed eseguita in tempo polinomiale su un calcolatore quantistico ideale.

Il vantaggio non deriva dalla semplice valutazione simultanea di tutte le soluzioni. Un circuito quantistico deve trasformare la struttura del problema in interferenza osservabile e produrre, con probabilità sufficiente, misure dalle quali il calcolo classico possa ricostruire la risposta. In Shor questo passaggio è affidato alla stima di fase e alla Trasformata di Fourier Quantistica (*Quantum Fourier Transform*) (QFT): l'uscita quantistica non contiene direttamente i fattori, ma informazione sul periodo della funzione modulare. La verifica classica resta quindi parte dell'algoritmo e rende naturale studiare non soltanto il circuito, ma anche la regola con cui si selezionano i candidati misurati.

La distanza tra questo risultato teorico e una realizzazione utile è tuttavia ampia. I circuiti di Shor richiedono aritmetica reversibile, molte operazioni coerenti e una stima di fase sufficientemente accurata; errori di gate, decoerenza e misura possono quindi cancellare la struttura necessaria al post-processing classico. I fondamenti fisici e matematici sono richiamati nella Sezione 3.1; il contesto storico, la vulnerabilità di RSA, la transizione post-quantum e lo stato dell'hardware sono sviluppati nella Sezione C.1.

1.2 Motivazione e Contributo del Presente Lavoro

La domanda centrale della tesi è la seguente: *quanta informazione utile può essere recuperata dall'output rumoroso di Shor e quando diventa invece necessario correggere l'errore durante il calcolo?* Per rispondere, il lavoro separa esplicitamente il post-processing dell'output dalla protezione dello stato quantistico e adotta metriche coerenti con ciascun livello.

La distinzione è sostanziale. Il post-processing può sfruttare meglio campioni già acquisiti, ma non può ricostruire una correlazione che il rumore ha cancellato prima della misura. La mitigazione stima o riduce un bias senza codificare stabilmente l'informazione; la QEC introduce invece ridondanza, misura sindromi e corregge gli errori preservando lo stato logico. Valutare insieme questi livelli senza confonderne le metriche permette di stabilire quale guadagno appartenga alla decisione classica e quale richieda una protezione fisica del calcolo.

La **prima fase** studia un'istanza controllata di Shor su simulatore ideale e rumoroso. Sul medesimo insieme di istogrammi vengono confrontati una baseline TOP-1, una ricerca multi-candidato TOP-4 e un metodo che antepone un classificatore appreso alla stessa ricerca. L'ablazione consente di distinguere il contributo della regola TOP-K da quello del modello di apprendimento. Le istanze più grandi sono usate per validare idealmente l'aritmetica e valutarne il costo, senza attribuire loro risultati rumorosi non misurati.

Il disegno appaiato è centrale: metodi diversi ricevono gli stessi conteggi, nello stesso ordine, e differiscono soltanto nella regola decisionale. In questo modo l'eventuale variazione del costo non viene confusa con una diversa realizzazione Monte Carlo. I preset di rumore sono scenari uniformi e riproducibili, utili per confronti controllati ma non presentati come calibrazioni di uno specifico processore.

La **seconda fase** sposta l'attenzione sulla Correzione degli Errori Quantistici (*Quantum Error Correction*) (QEC). Il percorso procede dal codice a ripetizione

al codice di Steane $[[7, 1, 3]]$ e al surface code, del quale si misura il *logical error rate* mediante decodifica *Minimum Weight Perfect Matching* (MWPM). Un decoder appreso viene poi valutato sulla sindrome, dove l'informazione diagnostica è ancora disponibile. Infine, un proxy di errore per gate permette un'analisi di sensibilità di Shor, mantenendo distinta questa quantità dai tassi per ciclo o per memoria dei benchmark QEC.

La progressione dei codici ha anche una funzione metodologica: il codice a ripetizione rende trasparente l'estrazione della sindrome, Steane estende la correzione agli errori Pauli arbitrari e il surface code introduce distanza variabile, soglia e decodifica su reticolo. Essa consente di passare gradualmente dal meccanismo elementare alla domanda di scalabilità, dichiarando a ogni livello assunzioni e limiti.

Il contributo non consiste quindi nel proporre l'apprendimento automatico come soluzione generale, ma nel costruire confronti controllati che ne identifichino il punto di applicazione e nel collegare, senza confonderli, post-processing, decodifica e protezione fault-tolerant. Obiettivi, ipotesi e perimetro sperimentale sono fissati nel Capitolo 2.

Riproducibilità e materiale supplementare. Sorgenti L^AT_EX, codice, seed, parametri e risultati delle campagne sono disponibili nel repository pubblico <https://github.com/claudio-dragotta/shor-nisq-qec-thesis>. Una demo interattiva dell'algoritmo di Shor è consultabile all'indirizzo <https://shor-demo-6knp.onrender.com> (sorgenti: <https://github.com/claudio-dragotta/shor-demo>).

1.3 Struttura della Tesi

Il Capitolo 2 definisce domande, ipotesi, perimetro e metriche. Il Capitolo 3, con i richiami della Sezione 3.1, presenta Shor; i Capitoli 4 e 5 introducono rumore e strategie di contrasto. Il Capitolo 6 descrive il protocollo, mentre i Capitoli 7, 8, 9 e 10 riportano, nell'ordine, la verifica TOP-K/ML, la progressione dei codici, lo studio del surface code e l'analisi di sensibilità di Shor. I Capitoli 11 e 12 discutono prospettive, risultati e limiti. Le Appendici A–E raccolgono teoria estesa, strumenti e codice, contesto, protocolli completi e analisi supplementari.

Capitolo 2

Obiettivi e Piano Sperimentale

Il capitolo traduce la motivazione introduttiva in un contratto sperimentale: definisce ciò che viene verificato, il perimetro entro cui valgono i risultati e le metriche usate. La motivazione crittografica, i requisiti funzionali e la specifica completa di input e output sono riportati nella Sezione C.2 (Sez. C.2.1, C.2.2 e C.2.3).

2.1 Obiettivo e Perimetro del Lavoro

L'obiettivo generale è stabilire quali strategie rendano più affidabile l'esecuzione di Shor in presenza di rumore, distinguendo tre livelli che non sono direttamente intercambiabili: recupero dei candidati dall'istogramma finale, correzione basata sulle sindromi e analisi di sensibilità a un errore logico residuo.

Il lavoro combina tre attività essenziali:

1. richiamare il period finding, la QFT, la stima di fase e i principali canali di rumore necessari a interpretare gli esperimenti;
2. implementare e validare Shor, quindi confrontare su simulatori rumorosi la baseline TOP-1, la ricerca TOP-4 e la variante con classificatore, usando gli stessi istogrammi;
3. costruire una progressione QEC — ripetizione, Steane $[[7, 1, 3]]$ e surface code — e misurare quando la codifica o un decoder appreso riducono l'errore logico. Una successiva analisi applica a Shor un proxy Pauli per gate compilato, senza identificarlo con il tasso di un memory experiment.

Le campagne rumorose complete riguardano l'istanza $N = 15$. Le istanze $N = 21$ e $N = 35$ sono impiegate soltanto per la validazione ideale dell'aritmetica e per

l'audit delle risorse: non vengono usate per inferire la scalabilità sotto rumore né per rappresentare una QPU reale.

2.2 Domande di Ricerca e Ipotesi

La domanda trasversale della prima fase è se il classificatore riduca il numero di iterazioni oltre quanto già ottenibile esaminando più candidati dello stesso istogramma. L'ipotesi principale richiede pertanto un vantaggio di M2 sia rispetto alla baseline M1 sia rispetto all'ablazione TOP-4; F1 e AUC del classificatore sono controlli predittivi secondari, non prove autonome di utilità operativa. Un eventuale vantaggio deve inoltre essere osservabile nei due preset $N = 15$, mentre la generalizzazione al crescere di N resta fuori dal perimetro rumoroso.

La progressione sperimentale risponde poi alle seguenti domande:

- DR1.** Come varia l'output di Shor al crescere dei diversi parametri di rumore, e quali conclusioni sono sostenute dagli sweep controllati?
- DR2.** Come si estrae una sindrome con un codice a ripetizione senza misurare direttamente lo stato logico?
- DR3.** Quali vantaggi e limiti offre Steane $[[7, 1, 3]]$ nella correzione di un errore Pauli arbitrario?
- DR4.** Il surface code mostra un regime di soglia nel quale aumentare la distanza riduce il logical error rate?
- DR5.** Come varia il successo di Shor sotto un proxy Pauli uniforme per gate e quale beneficio producono ripetizioni indipendenti?
- DR6.** In quale regime un decoder appreso dalla sindrome può migliorare il decoder analitico di riferimento?

Le ipotesi quantitative associate sono: soppressione quadratica dell'errore per i codici di distanza $d = 3$ nel regime perturbativo; esistenza di un incrocio delle curve $p_L(p)$ del surface code; andamento non crescente del successo di Shor al crescere del proxy per gate. Per la decodifica appresa il confronto è deliberatamente esplorativo: il modello deve essere valutato sugli stessi campioni e contro MWPM, senza presupporre la superiorità.

2.3 Contributo e Disegno Sperimentale

Il contributo metodologico è costituito da tre scelte coordinate. Primo, il confronto appaiato M1–TOP-4–M2 separa il valore della ricerca multi-candidato da quello del classificatore. Secondo, la progressione dai codici elementari al surface code usa metriche esplicite e dichiara il confine tra tasso per memoria, tasso per ciclo e proxy per gate. Terzo, il modello appreso viene valutato in due punti diversi della pipeline — output e sindrome — così che la sua utilità dipenda dall’informazione effettivamente disponibile, non dalla sola qualità predittiva.

Il protocollo mantiene distinti risultati, validazioni e assunzioni. Circuiti e aritmetica sono prima verificati idealmente; i metodi decisionali ricevono gli stessi campioni; ogni stima riporta seed, numerosità e incertezza; i preset uniformi sono trattati come scenari illustrativi e non come calibrazioni hardware. Il dettaglio implementativo e statistico è nel Capitolo 6, mentre gli esperimenti estesi sono raccolti nelle Appendici D e E.

2.4 Campagne Rumorose e Validazioni Ideali

Il disegno distingue due use case:

- **campagne rumorose N15**: stesso $N = 15$, base $a = 7$, circuito e budget di shot, con un preset di riferimento e uno di stress;
- **validazioni ideali N21/N35**: verifica dell’aritmetica reversibile, del ripristino delle ancille e della distribuzione QPE, senza simulazione rumorosa completa.

Parametro	Riferimento	Stress
N, a , periodo atteso r	15, 7, 4	15, 7, 4
n_{count}	8	8
M_{shots}	1024	1024
$\epsilon_{1q} = \lambda_{1q}$	1×10^{-3}	5×10^{-3}
$\epsilon_{2q} = \lambda_{2q}$	1×10^{-2}	5×10^{-2}
T_1 (ns)	100 000	50 000
T_2 (ns)	80 000	30 000
p_{ro}	2×10^{-2}	5×10^{-2}

Tabella 2.1: Preset uniformi delle campagne rumorose N15; non rappresentano la calibrazione di una QPU.

I parametri λ_{1q} e λ_{2q} sono quelli passati al canale depolarizzante di Aer; il readout è modellato mediante una matrice di confusione simmetrica. Il modello è uniforme e non comprende topologia, routing, crosstalk o deriva temporale.

2.4.1 Razionale per la Scelta delle Istanze

Per $N = 15$, la base $a = 7$ ha periodo atteso $r = 4$ e consente un circuito abbastanza contenuto da sostenere confronti appaiati e sweep. Per $N = 21$, $a = 2$, e $N = 35$, $a = 6$, l'aritmetica di Beauregard è invece validata idealmente sui vettori di base e mediante QPE. La separazione evita di trasformare un limite computazionale del simulatore in un'affermazione sul successo dell'algoritmo.

2.5 Metriche di Valutazione

2.5.1 Post-processing dell'Output

M1 usa il solo candidato più frequente; M_{TOP4} verifica i quattro candidati dominanti; M2 applica il classificatore prima della stessa ricerca TOP-4. Per ciascun metodo j si riportano il numero medio di iterazioni $\bar{M}_j$, la dispersione e la probabilità di successo entro il budget preregistrato $M_{\text{max}} = 50$. Il confronto descrittivo tra baseline e metodo con classificatore usa

$$\rho = \frac{\bar{M}_1}{\bar{M}_2}, \quad (2.1)$$

mentre l'ablazione richiede il confronto separato con M_{TOP4} . Un valore $\rho > 1$ indica soltanto una media inferiore per M2: l'inferenza è condotta sulle differenze appaiate complete. I fallimenti sono conservati mediante una sentinella oltre il budget; test, gestione degli zeri e tie-break deterministico sono specificati nel Capitolo 6. F1 e AUC descrivono il classificatore, ma non sostituiscono la metrica operativa.

2.5.2 Correzione d'Errore

La metrica centrale è il **logical error rate** p_L , stimato con intervallo di confidenza in funzione dell'errore fisico p e della distanza d . La pendenza γ in $p_L \propto p^\gamma$ verifica la soppressione a basso errore; il *break-even* è il regime $p_L < p$; la soglia p_{th} è individuata dall'incrocio delle curve a distanze diverse. Per l'analisi di Shor si misurano inoltre la probabilità per singola esecuzione e quella cumulativa $P(k) = 1 - (1 - P_s)^k$.

Il simbolo p_L compare in due contesti che restano distinti: nei benchmark QEC è un tasso riferito allo specifico circuito di memoria o correzione; nel proxy di Shor è la probabilità di un Pauli non identità dopo ciascun gate compilato incluso nel modello. Una conversione tra i due richiederebbe una compilazione fault-tolerant per operazione e non viene assunta.

2.6 Approccio Metodologico

La trattazione procede dalla teoria alla verifica ideale, quindi alle campagne appaiate e alla QEC. Architettura software, modelli di rumore, protocolli statistici e implementazione dei metodi sono descritti nel Capitolo 6; questo capitolo ne fissa soltanto obiettivi, perimetro e criteri di valutazione.

Capitolo 3

L'Algoritmo di Shor

3.1 Richiami di Calcolo Quantistico

Questa sezione introduce soltanto il formalismo necessario a seguire l'algoritmo di Shor, i modelli di rumore e i codici correttori. I postulati, l'insieme esteso delle porte e il modello di complessità sono raccolti nella Sezione A.1.

3.1.1 Qubit, misura ed entanglement

Un **qubit** è un vettore unitario di $\mathbb{C}^2$,

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1. \quad (3.1)$$

La fase globale non è osservabile; ogni stato puro può quindi essere parametrizzato come

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\phi} \sin\left(\frac{\theta}{2}\right) |1\rangle, \quad \theta \in [0, \pi], \quad \phi \in [0, 2\pi), \quad (3.2)$$

e rappresentato sulla sfera di Bloch. Questa rappresentazione rende intuitiva l'azione del rumore: un'evoluzione ideale ruota il vettore, mentre un canale rumoroso può contrarlo o spostarlo.

La misura in base computazionale restituisce 0 o 1 con probabilità rispettivamente $|\alpha|^2$ e $|\beta|^2$ (**regola di Born**) e proietta lo stato sull'esito osservato. Per un registro di n qubit lo spazio degli stati è il prodotto tensoriale $(\mathbb{C}^2)^{\otimes n}$, di dimensione 2^n . Uno stato è **entangled** quando non può essere scritto come prodotto degli stati dei sottosistemi; per esempio

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

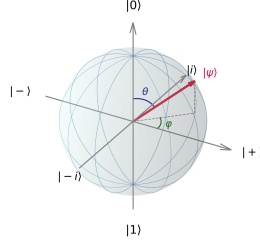


Figura 3.1: Rappresentazione di uno stato puro secondo la parametrizzazione dell'Eq. (3.2); i poli corrispondono a $|0\rangle$ e $|1\rangle$.

In Shor l'entanglement correla il registro di controllo con i valori della funzione modulare; la successiva interferenza rende osservabile il periodo. L'accoppiamento con l'ambiente degrada tali coerenze, collegando direttamente questo richiamo al Capitolo 4.

3.1.2 Operazioni essenziali e matrici di Pauli

L'evoluzione di un sistema isolato è descritta da una matrice unitaria U , con $U^\dagger U = I$. Le porte necessarie nel seguito sono: Hadamard H , che crea sovrapposizione; le rotazioni di fase R_ϕ , usate nella QFT; e la Controlled-NOT (CNOT), che correla due qubit ed è anche la principale porta a due qubit nel conteggio delle risorse.

Le matrici di Pauli

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

descrivono rispettivamente un bit flip, una combinazione di bit e phase flip e un phase flip. Insieme a I formano una base degli operatori su un qubit: è perciò sufficiente caratterizzare gli errori di Pauli per trattare, per linearità, un errore arbitrario. Le relazioni essenziali sono

$$X^2 = Y^2 = Z^2 = I, \quad XY = iZ, \quad YZ = iX, \quad ZX = iY, \quad \{X, Y\} = \{Y, Z\} = \{Z, X\} = 0, \quad (3.3)$$

dove $\{A, B\} = AB + BA$. L'anticommutazione permette ai codici stabilizzatori di rilevare un errore senza misurare direttamente l'informazione logica, come discusso nel Capitolo 8.

Infine, il vantaggio algoritmico non deriva dalla sola dimensione 2^n dello spazio degli stati. Occorre costruire il circuito affinché le ampiezze associate agli esiti incompatibili si cancellino e quelle utili si sommino. Questa **interferenza** è il principio sfruttato dalla QFT nel period finding.

3.2 Fattorizzazione e ricerca del periodo

Dato un intero composto N , la **fattorizzazione** richiede di determinarne fattori non banali. I migliori algoritmi classici generali noti, fra cui il *General Number Field Sieve* (GNFS), hanno complessità sub-esponenziale ma super-polinomiale nella lunghezza dell'input:

$$T_{\text{classico}}(N) = O\left(\exp\left((c + o(1))(\log N)^{1/3}(\log \log N)^{2/3}\right)\right). \quad (3.4)$$

L'algoritmo di Shor [1] riconduce invece il problema alla stima di un periodo e la risolve in tempo polinomiale. La conseguenza crittografica è diretta: una macchina quantistica sufficientemente grande e fault-tolerant invaliderebbe l'assunzione di sicurezza di RSA. Il contesto crittografico e la transizione post-quantum sono approfonditi nella Sezione C.2 dell'Appendice C.

Si scelga $a \in \{2, \dots, N-1\}$. Se $\gcd(a, N) > 1$, il massimo comune divisore è già un fattore; altrimenti si considera

$$f(x) = a^x \bmod N. \quad (3.5)$$

Poiché a è invertibile modulo N , la funzione è periodica. Il minimo intero positivo r tale che $a^r \equiv 1 \pmod{N}$ è l'**ordine** di a modulo N . Se r è pari e $a^{r/2} \not\equiv -1 \pmod{N}$, allora

$$(a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod{N}$$

e almeno uno fra

$$\gcd(a^{r/2} - 1, N), \quad \gcd(a^{r/2} + 1, N) \quad (3.6)$$

fornisce un fattore non banale. Quando le condizioni non sono soddisfatte si ripete la procedura con una nuova base a .

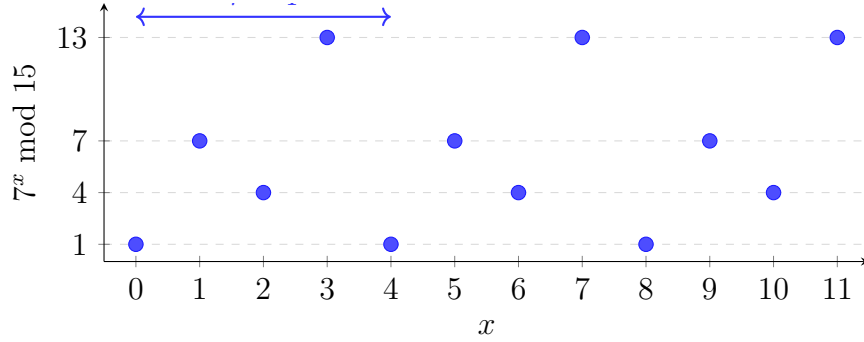


Figura 3.2: Per $N = 15$ e $a = 7$ la sequenza 1, 7, 4, 13 si ripete con periodo $r = 4$; i due MCD restituiscono 3 e 5.

3.3 QFT e stima quantistica del periodo

3.3.1 Quantum Fourier Transform

Su un registro di t qubit, con $Q = 2^t$, la QFT è definita da

$$\text{QFT } |j\rangle = \frac{1}{\sqrt{Q}} \sum_{k=0}^{Q-1} e^{2\pi i j k / Q} |k\rangle. \quad (3.7)$$

Per linearità trasforma le ampiezze di uno stato secondo la Trasformata Discreta di Fourier (*Discrete Fourier Transform*) (DFT). Il circuito usa porte di Hadamard e rotazioni di fase controllate e richiede $O(t^2)$ porte, contro le $O(Q \log Q)$ operazioni della Trasformata Rapida di Fourier (*Fast Fourier Transform*) (FFT) su Q dati espliciti. Il confronto non significa che la QFT restituisca più rapidamente l'intero spettro: i coefficienti restano codificati nelle ampiezze e una misura produce un solo esito. In Shor ciò è sufficiente perché interessano le posizioni dei picchi, non tutti i coefficienti.

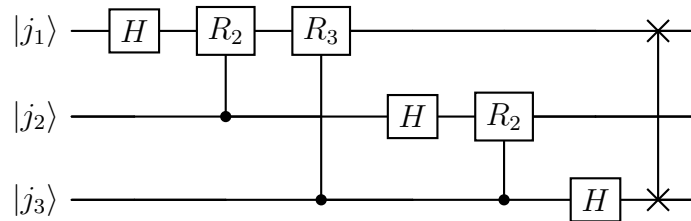


Figura 3.3: QFT su tre qubit: H , rotazioni controllate $R_k = \text{diag}(1, e^{2\pi i / 2^k})$ e riordinamento finale.

3.3.2 Phase estimation e periodicità

La *Quantum Phase Estimation* (QPE) stima la fase θ di un autovalore $e^{2\pi i\theta}$ di un'unitaria U . Nel period finding si usa

$$U_a |y\rangle = |ay \bmod N\rangle.$$

Sul sottospazio generato dall'orbita di 1, gli autovalori di U_a hanno fasi s/r , con $s \in \{0, \dots, r-1\}$. Un registro di controllo in sovrapposizione comanda le potenze $U_a^{2^j}$; la QFT inversa converte la fase in un intero misurabile y tale che y/Q approssima s/r . Scegliendo Q dell'ordine di N^2 , l'algoritmo delle frazioni continue ricostruisce r con probabilità sufficiente; esiti non informativi vengono gestiti ripetendo l'esecuzione.

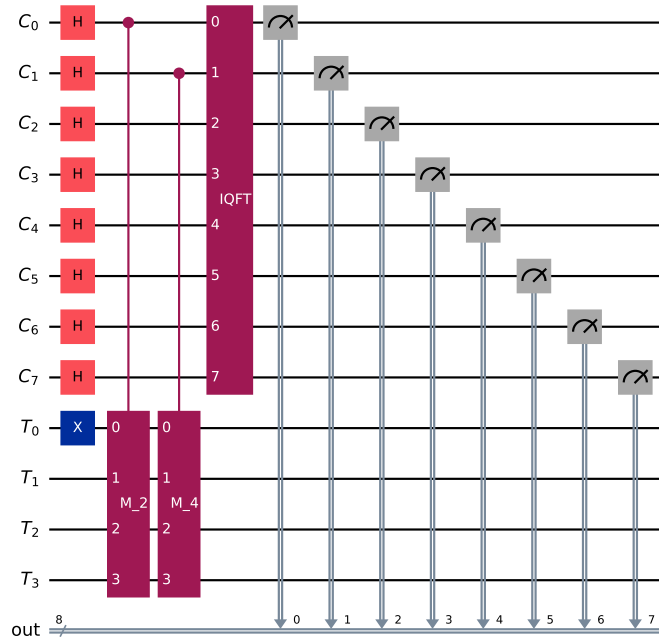


Figura 3.4: Struttura della QPE per $N = 15$: il registro di controllo comanda le potenze della moltiplicazione modulare e la QFT^{-1} rende misurabili le fasi s/r .

3.3.3 Ruolo dell'entanglement

Dopo l'esponentiazione modulare controllata, i registri sono nello stato

$$|\Psi\rangle = \frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle |a^x \bmod N\rangle. \quad (3.8)$$

I valori del secondo registro correlano classi di indici distanziate di r . Se lo si misurasse, il primo registro verrebbe proiettato su una sovrapposizione del tipo

$$|x_0\rangle + |x_0 + r\rangle + |x_0 + 2r\rangle + \cdots, \quad (3.9)$$

a meno della normalizzazione e del troncamento a Q . La misura intermedia non è necessaria: ignorare il registro di lavoro produce sul controllo la miscela delle stesse classi periodiche, e la QFT^{-1} conserva i picchi utili. Questo passaggio spiega sia il ruolo dell'entanglement sia la vulnerabilità dell'algoritmo alla perdita di coerenza.

3.4 Algoritmo completo

Posto $\ell = \lceil \log_2 N \rceil$, l'esecuzione può essere riassunta come segue:

1. scegliere a e calcolare $\text{gcd}(a, N)$; terminare se il MCD è non banale;
2. preparare circa 2ℓ qubit di controllo in sovrapposizione e il registro di lavoro in $|1\rangle$;
3. applicare le moltiplicazioni modulari controllate $U_a^{2^j}$;
4. applicare la QFT^{-1} e misurare il registro di controllo;
5. usare le frazioni continue per ricavare un candidato r ;
6. verificare il periodo e, se è valido, calcolare i due MCD; altrimenti ripetere con un nuovo campione o una nuova base.

Il controllo classico finale è indispensabile: la misura produce una stima da verificare, non direttamente i fattori. Per l'istanza $N = 15$ usata negli esperimenti, la costruzione dei blocchi di moltiplicazione modulare e del circuito completo è documentata nei Listati B.4 e B.5 dell'Appendice B.2. I diagrammi delle singole permutazioni controllate non sono necessari per comprendere l'algoritmo; ciò che conta sperimentalmente è la loro decomposizione nella base nativa, trattata nel Capitolo 6.

3.5 Complessità e limiti implementativi

Con aritmetica modulare elementare, l'esponenziazione controllata domina il costo e richiede $O(\ell^3)$ porte; la QFT aggiunge $O(\ell^2)$ porte e il post-processing classico resta

polinomiale. Pertanto

$$T_{\text{Shor}}(N) = O((\log N)^3), \quad (3.10)$$

mentre lo spazio quantistico è $O(\log N)$. Costruzioni in-place possono usare $2\ell + 3$ qubit [15], al prezzo di circuiti profondi. Rispetto alla complessità dell'Eq. (3.4), il vantaggio è super-polinomiale nella lunghezza dell'input; la praticabilità dipende però dal costo fault-tolerant delle operazioni logiche, non dal solo conteggio asintotico.

3.6 Perimetro sperimentale

Le dimostrazioni su hardware hanno finora usato istanze piccole e circuiti compilati o semplificati [5]. Non consentono di estrapolare direttamente le risorse necessarie per moduli crittografici. Implementazioni sperimentali, sviluppi algoritmici e confronto con Grover sono discussi nella Sezione A.3 dell'Appendice A. Nel seguito l'algoritmo resta quindi il carico di lavoro di riferimento, mentre l'oggetto misurato è la sua sensibilità al rumore entro modelli e circuiti dichiarati.

Capitolo 4

Rumore Quantistico nei Sistemi NISQ

4.1 Dal modello ideale alla macchina fisica

Shor è definito nel modello ideale di porte unitarie e misure perfette. I processori non fault-tolerant operano invece nel regime *Noisy Intermediate-Scale Quantum* (NISQ) [4]: coerenza finita, gate imperfetti, connettività limitata e lettura rumorosa restringono la profondità utile. Il numero di qubit, da solo, non determina quindi la capacità di eseguire un circuito.

La protezione non può consistere nella copia diretta dello stato, vietata dal teorema del no-cloning [20]. Inoltre un errore quantistico può essere una rotazione continua o un processo dissipativo, non soltanto l'inversione discreta di un bit. Per collegare fenomeno fisico e simulazione occorre distinguere la sorgente dell'errore dal *canale* matematico con cui la si approssima.

4.2 I vincoli hardware rilevanti

Per il seguito sono sufficienti tre proprietà delle Unità di Elaborazione Quantistica (*Quantum Processing Unit*) (QPU) superconduttive. Primo, i gate sono impulsi analogici: errori di calibrazione, accoppiamenti residui e leakage perturbano l'unitaria desiderata. Secondo, ogni operazione ha una durata, mentre rilassamento e perdita di fase avvengono su scale T_1 e T_2 ; la durata del cammino critico deve quindi restare piccola rispetto ai tempi di coerenza. Terzo, le interazioni a due qubit rispettano una topologia fisica: il routing aggiunge operazioni e l'esecuzione parallela può generare crosstalk. Anatomia criogenica, controllo e tecnologie di qubit sono approfonditi nella Sezione A.2 dell'Appendice A.

4.3 Fonti fisiche e canali adottati

Un canale quantistico è una mappa lineare *Completely Positive Trace-Preserving* (CPTP) sulla matrice densità. Nella rappresentazione di Kraus [7],

$$\mathcal{E}(\rho) = \sum_k K_k \rho K_k^\dagger, \quad \sum_k K_k^\dagger K_k = I. \quad (4.1)$$

La Tabella 4.1 riunisce i livelli fisico, matematico e algoritmico, evitando di trattare come equivalenti modelli che descrivono fenomeni diversi.

Fonte	Modello	Parametro	Impatto principale su Shor
Rilassamento energetico	amplitude damping / thermal relaxation	$\gamma = 1 - e^{-t/T_1}$	perdita della popolazione $ 1\rangle$ durante i blocchi profondi
Perdita di fase	phase flip / thermal relaxation	$p = \frac{1}{2}(1 - e^{-t/T_2})$	attenuazione delle coerenze usate dalla QPE
Imprecisione dei gate	canale depolarizzante a uno o due qubit	$\lambda_{1q}, \lambda_{2q}$	accumulo sulle porte entangling e distorsione delle fasi
Errore di lettura	matrice di confusione classica	e_0, e_1	alterazione dei bit da cui le frazioni continue stimano r
Crosstalk	errore correlato o coerente	dipendente da coppia e calendario	correlazioni non descritte dal modello uniforme indipendente

Tabella 4.1: Corrispondenza fra principali fonti di rumore, modelli e vulnerabilità dell'algoritmo.

4.3.1 Rilassamento e perdita di fase

Il tempo T_1 descrive il decadimento $|1\rangle \rightarrow |0\rangle$, mentre T_2 descrive la perdita della fase relativa. Separando la componente puramente dissipativa da quella di puro dephasing,

$$\frac{1}{T_2} = \frac{1}{2T_1} + \frac{1}{T_\phi}, \quad (4.2)$$

da cui $T_2 \leq 2T_1$. Il canale di amplitude damping è non unital: sposta lo stato verso $|0\rangle$. Il phase flip lascia invece invariate le popolazioni e moltiplica le coerenze per $1 - 2p$. Nella simulazione, Qiskit Aer combina i due contributi nel *thermal relaxation*

error, parametrizzato da T_1 , T_2 e dalla durata del gate. Il modello è Markoviano e non include automaticamente deriva, leakage o correlazioni temporali.

4.3.2 Canale depolarizzante

La campagna usa il canale depolarizzante per rappresentare l'errore stocastico associato ai gate. Per un qubit, indicando con p la probabilità totale di un Pauli non identità,

$$\mathcal{D}_p(\rho) = (1 - p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z). \quad (4.3)$$

Il canale contrae isotropicamente il vettore di Bloch e non descrive la direzione microscopica dell'errore. Aer usa invece il parametro λ nella convenzione

$$\mathcal{D}_\lambda^{(q)}(\rho) = (1 - \lambda)\rho + \lambda \frac{I}{2^q}, \quad (4.4)$$

per un gate su q qubit. La probabilità aggregata della componente Pauli non identità è dunque $3\lambda/4$ per $q = 1$ e $15\lambda/16$ per $q = 2$. Esplicitare questa convenzione è necessario per interpretare correttamente i parametri sperimentali; la definizione del noise model e i gate ai quali è applicato sono riportati nel Capitolo 6.

4.3.3 Lettura e rumore correlato

L'errore di misura è rappresentato, per ciascun qubit, dalla matrice di assegnazione

$$A = \begin{pmatrix} 1 - e_0 & e_1 \\ e_0 & 1 - e_1 \end{pmatrix}, \quad A_{ij} = P(\text{esito } i \mid \text{stato } j). \quad (4.5)$$

Per errori indipendenti su più qubit si usa il prodotto tensoriale delle matrici locali. Il crosstalk viola invece l'ipotesi di indipendenza: un'operazione può perturbare qubit vicini o dipendere dalle operazioni simultanee [21]. Il modello uniforme della prima campagna non pretende di riprodurre tali correlazioni; queste vengono introdotte separatamente negli esperimenti di decodifica, dove è possibile confrontare metodi analitici e appresi sullo stesso insieme di sindromi.

Le derivazioni degli operatori di Kraus, l'azione sulle componenti della matrice densità e la geometria sulla sfera di Bloch sono raccolte nella Sezione A.4 dell'Appendice A.

4.4 Perché Shor è sensibile al rumore

La vulnerabilità non deriva da un solo canale, ma dall'interazione fra profondità, precisione di fase e misura:

- l'**esponenziazione modulare** domina la profondità e contiene molte porte a due qubit; aumentano sia l'esposizione agli errori di gate sia il tempo durante il quale i registri devono restare coerenti;
- la **phase estimation** codifica il periodo nelle fasi relative. Dephasing ed errori coerenti riducono o spostano i picchi da cui si ricava s/r ;
- la **QFT inversa** contiene rotazioni controllate, comprese rotazioni ad angolo piccolo, sensibili alla precisione di controllo;
- la **misura** finale può modificare i bit dell'intero osservato e quindi la frazione continua selezionata;
- **routing e parallelismo** aggiungono gate e possono introdurre errori correlati dipendenti dalla topologia.

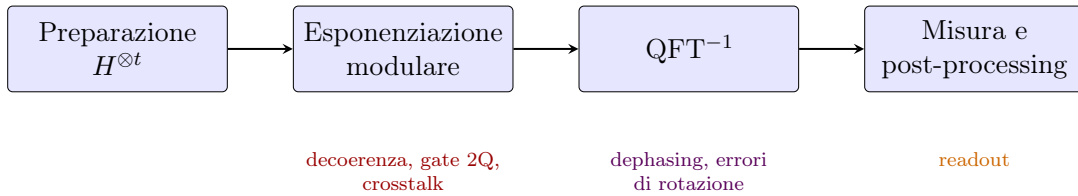


Figura 4.1: Principali sorgenti di errore lungo la pipeline; decoerenza e crosstalk possono agire trasversalmente, non soltanto nello stadio indicato.

Non è corretto trasformare il solo conteggio asintotico $O((\log N)^3)$ in una probabilità di successo: gate diversi hanno durate e tassi diversi, il circuito contiene parallelismo e alcuni errori non cambiano l'esito classico. Per questa ragione il seguito usa circuiti compilati e noise model dichiarati, separando conteggio di gate, profondità, parametri fisici e metrica di successo. Le dimostrazioni di fattorizzazione su istanze piccole e circuiti semplificati [5] non rimuovono questo limite di scalabilità.

Il capitolo successivo distingue quindi gli interventi possibili: riduzione fisica del rumore, mitigazione a posteriori, correzione/tolleranza ai guasti e metodi appresi. Nessuno di essi viene assunto efficace per definizione; ciascuno richiede un confronto entro il proprio livello operativo.

Capitolo 5

Strategie per la Riduzione del Rumore Quantistico

Il rumore può essere affrontato in momenti diversi della pipeline. Distinguere questi livelli evita due equivoci: una stima a posteriori dell'output ideale non corregge lo stato durante il calcolo, e un modello appreso non coincide con la correzione d'errore quantistica.

5.1 Panoramica e criteri di scelta

Strategia	Livello	Principio	Vincolo principale
Soppressione del rumore	hardware e controllo	ridurre l'errore prima o durante l'esecuzione	richiede accesso a pulse, calibrazione o dispositivo
Mitigazione	esecuzioni e post-processing	stimare l'osservabile ideale da circuiti rumorosi	aumenta il numero di campioni e dipende dal modello
QEC e fault tolerance	stato logico e architettura	rilevare errori tramite sindromi e limitarne la propagazione	richiede qubit, cicli di sindrome e operazioni logiche aggiuntive
Metodi appresi classici	output o decoder	apprendere una decisione da istogrammi o sindromi	necessita dati, validazione e un riferimento non appreso

Tabella 5.1: Livelli di intervento contro il rumore e relativi vincoli.

La **soppressione** comprende isolamento, calibrazione, pulse shaping e dynamical decoupling. Riduce l'errore fisico ma non lo annulla ed è in larga parte vincolata

all'hardware. La **mitigazione**, fra cui la *Zero-Noise Extrapolation* (ZNE), combina più esecuzioni rumorose senza codificare qubit logici; è utilizzabile nel regime NISQ, ma il suo costo di campionamento cresce con il rumore totale e con la tecnica adottata.

La **QEC** codifica l'informazione in un sottospazio logico e ricava una sindrome senza misurare direttamente lo stato protetto. La fault tolerance estende questo principio all'intero calcolo, imponendo che un singolo guasto non si propaghi in modo incontrollato. Sotto le ipotesi del threshold theorem e al di sotto della soglia pertinente, aumentare la distanza del codice può sopprimere l'errore logico [22]; soglia e overhead dipendono però da circuito, modello di rumore e decoder.

I **metodi appresi** sono trasversali. Possono filtrare un output già campionato oppure assistere il decoder di un codice, ma non rimuovono il costo di generare quei dati e non sostituiscono automaticamente una regola analitica. La loro utilità deve essere valutata sullo stesso input e con la stessa metrica dell'alternativa di confronto.

5.2 Machine learning classico su dati quantistici

In questa tesi Random Forest, *Support Vector Machine* (SVM) e Percettrone Multistrato (*Multi-Layer Perceptron*) (MLP) sono modelli eseguiti su hardware classico; il termine corretto è quindi **machine learning classico applicato a dati quantistici**, non *Quantum Machine Learning* (QML). Vengono considerati due contratti distinti:

- nel **post-processing di Shor**, il modello riceve feature dell'istogramma e decide se ampliare la ricerca dal candidato dominante ai primi K candidati;
- nella **decodifica QEC**, il modello riceve sindromi o detection event e propone una correzione, oppure modifica residualmente l'uscita di un decoder analitico.

Nel primo caso il riferimento naturale è una regola TOP- K priva di addestramento; nel secondo è un decoder analitico adatto al codice e al noise model. Separare i due contratti impedisce di attribuire al modello informazione già contenuta nella regola di selezione o nel decoder di base.

5.3 Correzione quantistica e decodifica

La QEC distribuisce un qubit logico su più qubit fisici. Gli stabilizzatori rivelano la classe dell'errore attraverso una sindrome, mentre il decoder sceglie un recupero compatibile. Questo è sufficiente per motivare gli esperimenti successivi; formalismo,

circuiti e curve dei codici a ripetizione e di Steane sono sviluppati nel Capitolo 8, mentre geometria, soglia e distanza del surface code sono trattate nel Capitolo 9. I riferimenti storici dei codici impiegati sono Shor [23], Steane [9] e il surface code [10].

5.3.1 Il modello appreso come componente del decoder

Il punto di contatto fra QEC e machine learning è la mappa classica dalla sindrome alla decisione. Un modello può apprendere correlazioni non rappresentate dal decoder analitico oppure operare come correzione residua [24, 17]. Ciò non garantisce un vantaggio: la qualità dipende dalla distribuzione di addestramento, dalla stabilità del rumore e dal costo d’inferenza. Per questo il Capitolo 9 confronta decoder appreso e analitico sugli stessi campioni e mantiene separati addestramento, selezione e valutazione.

5.4 Strategia sperimentale della tesi

Il lavoro adotta una progressione in due fasi. La prima studia ciò che si può recuperare *dopo* la misura di Shor: TOP-1, TOP- K e filtro appreso sono confrontati sugli stessi istogrammi, mentre la ZNE costituisce un confronto di mitigazione separato. Metodo e verifica sono descritti nei Capitoli 6 e 7.

La seconda fase interviene *durante* la protezione dell’informazione: caratterizza codici di complessità crescente, stima l’errore logico e valuta il ruolo di un decoder appreso. Il Capitolo 10 mantiene infine distinta la sensibilità di Shor a un proxy per gate dai tassi logici per ciclo dei codici. Questa separazione fra livelli — output, sindrome e circuito logico — è il criterio che rende confrontabili le strategie senza confonderne obiettivi e metriche.

Capitolo 6

Metodo e Implementazione

Questo capitolo definisce il metodo con cui vengono verificate le due domande sperimentali della tesi: se un filtro appreso migliori il post-processing di Shor e se un modello appreso possa assistere la decodifica di una sindrome quantistica. Gli strumenti sono scelti in funzione del circuito da simulare e della metrica da stimare.

6.1 Strumenti e Ambiente di Simulazione

Shor e prima fase. Il circuito generale di Shor, il modello di rumore e il campionamento degli istogrammi sono implementati con Qiskit e Qiskit Aer [11]. La campagna rumorosa usa lo statevector soltanto per l'istanza didattica $N = 15$; le istanze $N = 21$ e $N = 35$ sono riservate alla validazione ideale dell'aritmetica e all'audit del costo circuitale.

Correzione d'errore. I laboratori su repetition e Steane usano Qiskit Aer. Per il surface code, che richiede molti campioni di circuiti stabilizer, si adottano Stim [12] e PyMatching [13]: il primo genera i *detection events*, il secondo applica la decodifica MWPM. Questa divisione evita di simulare con uno statevector circuiti per i quali esiste una rappresentazione classica efficiente.

Modelli appresi e riproducibilità. I classificatori sono realizzati con scikit-learn [14]. Le campagne canoniche vengono eseguite su CPU in un ambiente Python versionato; seed, dipendenze, parametri e manifest del circuito sono registrati con i risultati. Le motivazioni estese delle scelte tecnologiche, il confronto tra framework e il dettaglio dell'ambiente WSL/GPU sono riportati nella Sezione B.1; i listati sono raccolti nella Sezione B.2.

Il disegno sperimentale separa tre livelli che non devono essere confusi: correttezza ideale dell'aritmetica, comportamento sotto un modello di rumore controllato e capacità delle regole di post-processing di utilizzare l'istogramma ottenuto. Gli esiti numerici sono presentati nel Capitolo 7; qui vengono fissati perimetro, variabili e criteri di confronto.

6.2 Disegno e perimetro sperimentale

La prima fase confronta tre strategie sullo *stesso* istogramma: **M1**, che verifica il solo candidato più frequente (TOP-1); **TOP-4**, che verifica i primi quattro candidati; e **M2**, in cui un classificatore decide se attivare la medesima ricerca TOP-4. TOP-4 è l'ablazione necessaria per distinguere il contributo del modello appreso da quello della semplice ricerca multi-candidato.

Le campagne rumorose sono limitate a $N = 15$, $a = 7$, con due preset uniformi e illustrativi. Essi non rappresentano la calibrazione di una QPU. Le istanze $N = 21$, $a = 2$, e $N = 35$, $a = 6$, sono validate idealmente mediante truth table, pulizia delle ancille e distribuzione QPE, ma sono escluse dal campionamento rumoroso per il costo della simulazione.

Tabella 6.1: Contratto essenziale delle campagne rumorose su $N = 15$.

Parametro	Riferimento	Stress
a / periodo atteso	7/4	7/4
qubit di controllo / shot	8/1024	8/1024
$\lambda_{1q} / \lambda_{2q}$	$10^{-3}/10^{-2}$	$5 \times 10^{-3}/5 \times 10^{-2}$
T_1/T_2 (ns)	100 000/80 000	50 000/30 000
p_{ro}	0.02	0.05

6.3 Pipeline appaiata

Il circuito viene compilato una sola volta nella base `rz/sx/x/cx`, con `optimization_level=2` e seed del transpiler fissato. La configurazione di riferimento contiene 224 porte `cx` e ha profondità 412; hash e conteggi completi della netlist sono registrati nel manifest. Per ogni coppia (replica, iterazione) Aer produce un solo istogramma, poi consegnato a M1, TOP-4 e M2. I pareggi di frequenza sono risolti mediante una priorità SHA-256 dipendente dal seed e dalla bitstring, così l'ordine non dipende dal dizionario o dal valore numerico dell'esito.

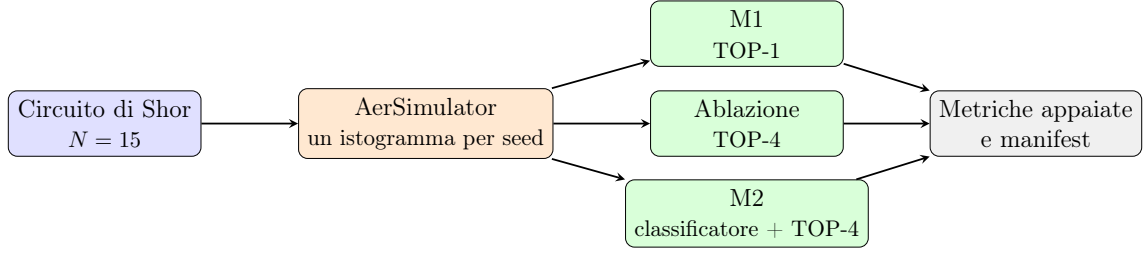


Figura 6.1: Ogni istogramma è riutilizzato dalle tre strategie, eliminando differenze dovute a realizzazioni Monte Carlo diverse.

6.4 Modello di rumore e controlli

Le rotazioni $\mathbf{rz}$ sono trattate come cambi di frame virtuali. Su $\mathbf{sx}/\mathbf{x}$ e $\mathbf{cx}$ vengono composti il canale depolarizzante e il rilassamento termico, con durate rispettivamente di 50 e 300 ns; il readout è una matrice di confusione simmetrica. Nel costruttore Aer il parametro λ definisce

$$\mathcal{E}(\rho) = (1 - \lambda)\rho + \lambda I/2^n.$$

Per i gate a due qubit la probabilità aggregata della componente Pauli non identità è $15\lambda/16$. Il corrispondente indicatore circuitale è

$$P_{\text{no-Pauli-2Q}} = \left(1 - \frac{15\lambda}{16}\right)^{224}. \quad (6.1)$$

È un proxy di nessun evento sulle CX, non una probabilità di successo di Shor, e non comprende errori 1Q, rilassamento o readout.

Il contratto base non inverte a posteriori la matrice di readout. Readout, T_1/T_2 , errori 1Q/2Q, numero di shot, valore di K e livello di ottimizzazione sono variati in campagne di sensibilità. La griglia completa e il confronto con la ZNE sono riportati nella Sezione D.1; nel corpo resta soltanto il loro bilancio.

6.5 Regole decisionali e analisi statistica

Per ogni istogramma M1 prova il primo candidato ordinato; TOP-4 ne prova fino a quattro; M2 tenta gli stessi quattro candidati soltanto quando il classificatore accetta l'istogramma. Il dataset conserva sia l'etichetta TOP-1 sia l'audit TOP-16, ma la pipeline usa esclusivamente il modello TOP-1 compatibile con il manifest.

Architetture, addestramento e metriche del classificatore sono documentati nella Sezione D.2.

Ogni scenario comprende 30 repliche e un budget massimo di 50 iterazioni. La prima iterazione di successo è registrata separatamente per le tre strategie; un fallimento al budget assume la sentinella 51. I confronti $M1 > \text{TOP-4}$ e $M1 > M2$ usano il test dei ranghi con segno di Wilcoxon appaiato, unilaterale, con `zero_method='pratt'`; $\text{TOP-4}-M2$ è bilaterale. Le correzioni per confronti multipli sono applicate entro la famiglia dichiarata. Medie e tassi di successo sono descrittivi, mentre l'inferenza usa i vettori appaiati completi.

Questa sezione completa il metodo descrivendo soltanto le decisioni implementative che incidono sulla validità degli esperimenti. I listati riproducibili sono raccolti nella Sezione B.2; setup, dipendenze e limiti dell'ambiente sono documentati nella Sezione B.1.

6.6 Implementazione e tracciabilità

L'implementazione di Shor costruisce la QPE con un registro di controllo, un registro di lavoro, moltiplicazioni modulari controllate e QFT inversa. Per $N = 15$, $a = 7$, il blocco applica realmente $y \mapsto 7y \bmod 15$ il numero di volte richiesto dall'esponente; per $N = 21$ e $N = 35$ viene usata l'aritmetica reversibile di Beauregard. Truth table, ripristino delle ancille e distribuzione QPE ideale sono verificati prima di qualunque campagna rumorosa.

Compilazione, training, baseline e sweep condividono la stessa funzione di transpilazione e lo stesso manifest. In questo modo una revisione del circuito, del gate set o del noise model rende incompatibili gli artefatti precedenti invece di mescolarli ai risultati correnti.

6.7 Rumore e strategie della prima fase

Il costruttore del rumore associa depolarizzazione e rilassamento termico soltanto alle istruzioni fisiche dichiarate e aggiunge separatamente la matrice di readout. $M1$, TOP-4 e $M2$ vengono valutati mediante un'unica funzione che riceve l'istogramma comune e restituisce i tre esiti decisionali. Il modello appreso include nei metadati definizione dell'etichetta, revisione, preset e hash del circuito; l'inferenza rifiuta un modello incompatibile.

6.8 Raccolta dati e implementazione QEC

Seed di simulazione e tie-break sono derivati in modo deterministico dalla coppia (replica, iterazione), con intervalli superiori al numero di shot. Ogni risultato conserva input, seed, versioni software, vettori appaiati e manifest. Le figure leggono questi artefatti senza trascrizione manuale dei valori.

La seconda fase applica lo stesso principio di separazione delle responsabilità: Qiskit Aer verifica repetition e Steane; Stim genera i circuiti di memoria del surface code; PyMatching decodifica gli stessi *detection events* consegnati ai modelli appresi. Il proxy per gate di Shor converte esplicitamente la probabilità totale di Pauli non identità nel parametro Aer, senza identificarla con il tasso per round o per memoria dei benchmark QEC. Protocolli, assunzioni e numerosità sono descritti nei capitoli che presentano i rispettivi risultati.

Capitolo 7

Verifica Sperimentale dell'Ipotesi Iniziale: dal Classificatore ML alla Strategia TOP-K

La prima fase verifica se un classificatore applicato all'istogramma rumoroso riduca il numero di iterazioni necessario a fattorizzare $N = 15$. Il confronto appaiato separa il modello dalla regola TOP-4 e produce un risultato netto: la ricerca multi-candidato è utile, mentre il filtro appreso non aggiunge valore alla medesima ricerca.

7.1 Perimetro e controlli di validità

Circuito, noise model, numerosità e test statistici sono quelli fissati nel Capitolo 6. Tutte le strategie ricevono lo stesso istogramma e lo stesso ordinamento dei pareggi. La campagna usa soltanto gli artefatti generati dopo la verifica del moltiplicatore $\times 7 \bmod 15$; manifest e hash impediscono di combinare revisioni diverse.

Le implementazioni Beauregard per $N = 21$ e $N = 35$ superano i controlli ideali di truth table, pulizia delle ancille e distribuzione QPE. Questo non le rende però adatte alla stessa campagna rumorosa: per $N = 21$ la compilazione raggiunge 21 036 CX e profondità 23 081. L'esclusione documenta un limite della simulazione classica adottata, non un limite dell'algoritmo quantistico.

7.2 Ablazione M1–TOP-4–M2

Gli istogrammi conservano in genere massa sui quattro picchi teorici $\{0, 64, 128, 192\}$, ma la loro moda non è stabile: nella diagnostica indipendente la moda è $y = 0$ in metà dei casi, mentre TOP-4 include sempre almeno un candidato utile. La differenza tra le strategie dipende quindi da come utilizzano una struttura ancora osservabile, non dalla produzione di un circuito diverso.

Tabella 7.1: Confronto appaiato delle tre strategie, 30 repliche per scenario.

Scenario	Strategia	Successo	$\bar{M}$	ρ vs M1	p_{Holm} vs M1
UC1	M1	30/30	1.37	—	—
UC1	TOP-4	30/30	1.00	1.367	0.0024
UC1	M2	30/30	1.50	0.911	0.7314
UC2	M1	30/30	1.50	—	—
UC2	TOP-4	30/30	1.00	1.500	0.0015
UC2	M2	30/30	1.37	1.098	0.0874

TOP-4 converge al primo tentativo in tutte le repliche ed è migliore di M1 in entrambi gli scenari. M2 non migliora significativamente M1 e, rispetto alla propria ablazione, richiede più iterazioni ($p_{\text{Holm}} = 0.0033$ in UC1 e 0.0096 in UC2). Il classificatore scarta dunque istogrammi che TOP-4 avrebbe risolto: il guadagno appartiene alla ricerca multi-candidato.

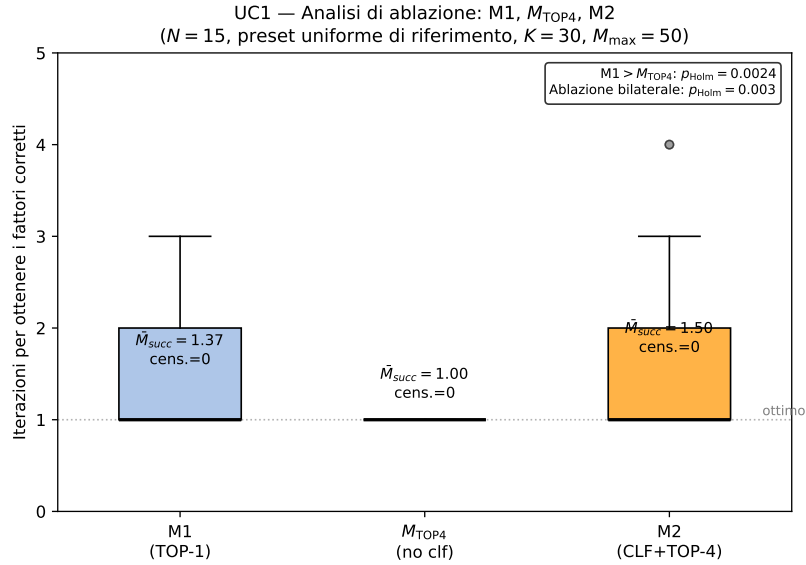


Figura 7.1: Iterazioni richieste da M1, TOP-4 e M2 nelle repliche appaiate dei due scenari.

L’audit delle etichette completa l’interpretazione. TOP-1 definisce un problema addestrabile, ma predice soltanto la riuscita del candidato dominante; TOP-16 produce 2 000/2 000 positivi in entrambi gli scenari. Le metriche elevate del classificatore non implicano quindi un vantaggio operativo. Architetture, curve ROC e analisi combinatoria sono riportate nella Sezione D.2.

7.3 Robustezza e confronti secondari

Otto famiglie di sweep variano K , errore 2Q, numero di shot, T_1/T_2 , errore 1Q, readout e livello di ottimizzazione. TOP-4 resta a una iterazione in tutti i punti salvo il livello più severo $\lambda_{2q} = 0.1$, dove sale a 1.07; le oscillazioni di M1 non sono monotone e, con 30 repliche, non vengono interpretate causalmente. Questi risultati sono esplorativi, poiché non è applicata una correzione globale fra tutte le famiglie.

La massa sui picchi QPE diminuisce con il rumore molto più lentamente del proxy di nessun evento sulle CX. Ciò conferma che un evento Pauli non equivale necessariamente al fallimento dell’algoritmo. Definizione dell’indice, griglia completa e figure sono raccolte nella Sezione D.1.

Nello stesso perimetro la ZNE a due e tre livelli non migliora M1 e moltiplica il costo in shot; TOP-4 è l’unica strategia che riduce significativamente le iterazioni. Il risultato riguarda questa istanza e questo modello uniforme, non costituisce una valutazione generale della ZNE su hardware calibrato.

7.4 Bilancio e passaggio alla correzione d’errore

La prima fase stabilisce che una regola semplice può utilizzare meglio l’informazione già presente nell’istogramma, mentre il classificatore considerato non aggiunge informazione alla regola TOP-4. Nessun post-processing può però ricostruire coerenza e periodicità già distrutte durante il circuito. Questo limite motiva il passaggio alla QEC, dove l’informazione diagnostica è esposta dalla sindrome prima della misura finale.

Capitolo 8

La Correzione d'Errore Quantistico: dal Codice a Ripetizione al Codice di Steane

Il post-processing del Capitolo 7 recupera candidati ancora presenti nell'istogramma, ma non ripristina informazione distrutta durante l'evoluzione. La QEC interviene invece durante il calcolo. Questo capitolo ne introduce gli elementi operativi — stabilizzatori, sindrome e decoder — e li verifica sul codice a ripetizione e sul codice di Steane $[[7, 1, 3]]$; il Capitolo 9 passa poi al surface code scalabile.

8.1 Dalla Ridondanza Ingenua alla Codifica

L'idea di proteggere l'informazione replicandola è classica: si trasmettono tre copie di un bit e si decide a maggioranza. Sul piano quantistico questa strada è sbarrata due volte. Il teorema di no-cloning [20] (Sezione 3.1) impedisce di copiare uno stato ignoto; e la misura diretta di un qubit, necessaria per un ipotetico “voto di maggioranza”, distrugge la sovrapposizione che si voleva proteggere. Per questo la pratica NISQ più diffusa si limita a una ridondanza *a livello di esecuzione* — eseguire istanze gemelle dello stesso circuito su regioni diverse del dispositivo e usare l'esito della copia sopravvissuta — che paga un raddoppio di qubit fisici senza agire sulla causa degli errori.

La QEC risolve il problema in modo strutturalmente diverso [23, 9, 25]: l'informazione di un **qubit logico** (*logical qubit*) viene *codificata* — non copiata — nelle correlazioni di entanglement di più **qubit fisici** (*physical qubits*), e gli errori

vengono diagnosticati misurando operatori ausiliari che rivelano *che cosa è andato storto* senza rivelare — e quindi senza distruggere — *che cosa è codificato*. È questa distinzione, non la semplice ridondanza, a rendere possibile la correzione durante il calcolo.

8.2 Il Ciclo della Correzione: Stabilizzatori, Sindrome, Decoder

Il funzionamento di ogni codice correttore quantistico segue lo stesso ciclo concettuale:

1. **Codifica:** lo stato logico $|\psi_L\rangle$ viene preparato su n qubit fisici;
2. **Rumore:** i canali fisici del Capitolo 4 agiscono sui qubit fisici;
3. **Estrazione della sindrome:** qubit ausiliari (*ancilla*) misurano gli stabilizzatori del codice; l'esito — una stringa di bit classica detta **sindrome** (*syndrome*) — identifica la classe di errore senza toccare l'informazione logica;
4. **Decodifica:** un algoritmo classico, il **decoder**, interpreta la sindrome e propone la correzione più probabile;
5. **Correzione:** la correzione viene applicata fisicamente, oppure registrata in un registro classico — il **Pauli frame** — che tiene memoria delle correzioni da interpretare sulle misure successive, riducendo le operazioni fisiche e la propagazione di errori.

Stabilizzatori. Uno **stabilizzatore** (*stabilizer*) [26] è un operatore S che lascia invariati tutti gli stati validi del codice: $S|\psi_L\rangle = |\psi_L\rangle$. Finché lo stato resta nello spazio del codice, ogni misura di S restituisce $+1$; un esito -1 segnala che un errore ha portato lo stato fuori dal sottospazio consentito. La collezione degli esiti di tutti i generatori del gruppo stabilizzatore costituisce la sindrome. Il punto cruciale è che gli stabilizzatori commutano con l'informazione logica: la loro misura proietta lo stato in un autospazio dell'errore, ma non distingue — e quindi non collassa — i coefficienti della sovrapposizione logica.

Distanza del codice. Un codice si denota $[[n, k, d]]$: n qubit fisici, k qubit logici, distanza d . La distanza è il peso minimo di un errore capace di trasformare uno stato

logico valido in un altro senza essere rilevato; un codice di distanza d corregge fino a

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor \quad (8.1)$$

errori arbitrari. Lo Steane code ($d = 3$) corregge un errore; un surface code di distanza 5 ne corregge due, di distanza 7 tre. La soglia oltre la quale aggiungere distanza *conviene* — il regime sotto-soglia in cui l'errore logico decresce all'aumentare di d — è l'oggetto quantitativo del Capitolo 9, e la sua esistenza è garantita in linea di principio dal teorema della soglia per il calcolo fault-tolerant [22].

8.3 Il Panorama dei Codici e la Scelta Metodologica

La progressione adottata è funzionale all'esperimento: il *repetition code* rende leggibile l'estrazione di sindrome; lo Steane $[[7, 1, 3]]$ corregge un errore arbitrario; il surface code introduce distanza variabile, cicli rumorosi e un decoder scalabile. Le implementazioni dei primi due codici sono riportate nei Listati B.11 e B.12 dell'Appendice B.

8.4 Laboratorio I: il Codice a Ripetizione

8.4.1 Il codice bit-flip a 3 qubit

Il codice a ripetizione contro errori X codifica $|0_L\rangle = |000\rangle$ e $|1_L\rangle = |111\rangle$: uno stato arbitrario $\alpha|0\rangle + \beta|1\rangle$ diventa $\alpha|000\rangle + \beta|111\rangle$ — uno stato entangled, non tre copie. Gli stabilizzatori sono Z_1Z_2 e Z_2Z_3 : misurano la *parità* tra coppie di qubit senza misurare i singoli qubit. Un bit-flip su un qubit altera una o entrambe le parità, e la sindrome a 2 bit identifica univocamente quale dei tre qubit ha subito l'errore — che viene corretto riapplicando X. La misura di parità avviene tramite ancilla: due CNOT copiano la parità sull'ancilla, che viene misurata al posto dei qubit dati.

Il limite è strutturale: il codice non rileva gli errori Z (phase-flip), che commutano con tutti gli stabilizzatori. Il duale — codificare nella base di Hadamard, $|\pm_L\rangle = |\pm\pm\pm\rangle$, con stabilizzatori X_1X_2 e X_2X_3 — protegge dai phase-flip ma non dai bit-flip. La dualità X/Z è la ragione per cui la correzione quantistica è più difficile di quella classica, e la concatenazione delle due protezioni è esattamente la costruzione del codice di Shor a 9 qubit [23].

8.4.2 Risultati del laboratorio repetition

Il primo controllo inietta deterministicamente un errore su ciascun qubit e verifica la lookup table

$$00 \rightarrow I, \quad 10 \rightarrow X_0, \quad 11 \rightarrow X_1, \quad 01 \rightarrow X_2.$$

Le quattro sindromi osservate coincidono con la previsione; per coniugazione di Hadamard vale la stessa mappatura per gli errori Z del codice duale. Il test completo e il circuito di estrazione sono documentati nel Listato B.11.

Il secondo test misura il beneficio della correzione. Si prepara lo stato logico, si applica a ciascun qubit dato un errore indipendente con probabilità p (canale X per il bit-flip, Z per il phase-flip), si estrae la sindrome, si applica la correzione e si decodifica: la frazione di esiti logici errati su 20 000 ripetizioni (seed = 42) stima l'errore logico p_L . La Figura 8.1 confronta i punti Monte Carlo con la previsione analitica del voto di maggioranza: il codice sbaglia quando almeno due dei tre qubit subiscono un errore, cioè

$$p_L = 3p^2(1 - p) + p^3 = 3p^2 - 2p^3. \quad (8.2)$$

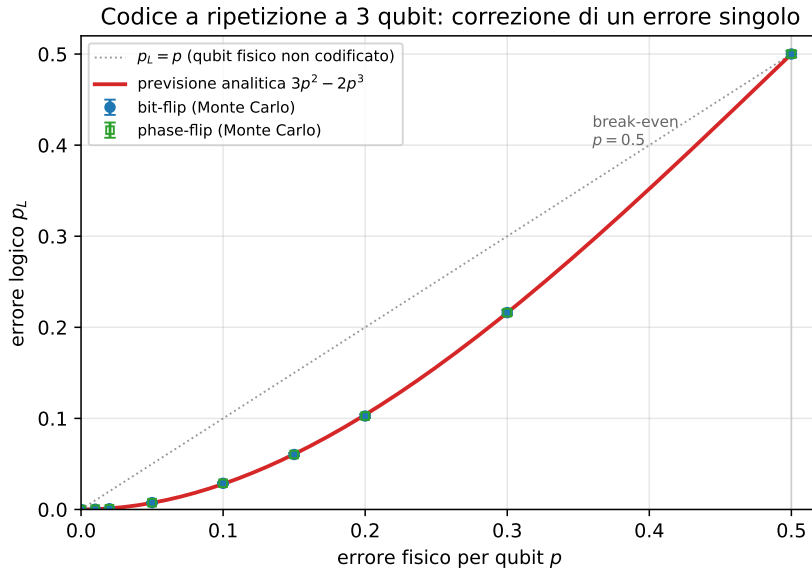


Figura 8.1: I punti Monte Carlo (20 000 campioni per valore) seguono $p_L = 3p^2 - 2p^3$; sotto $p = 0.5$ la codifica riduce l'errore.

L'accordo è quantitativo: a $p = 0.10$ si osserva $p_L = 0.0288 \pm 0.0012$ contro 0.0280 teorico. La soppressione è quadratica, ma il codice protegge un solo tipo di errore

per volta; serve quindi un codice capace di correggere tutte le componenti di Pauli.

8.5 Laboratorio II: il Codice di Steane [[7, 1, 3]]

8.5.1 Struttura del codice

Lo Steane code [9] codifica un qubit logico in sette qubit fisici, ha distanza 3 e corregge un errore arbitrario (X, Z o Y) su un singolo qubit. È un codice Calderbank-Shor-Steane (CSS), costruito a partire dal codice classico di Hamming [7, 4, 3]: la stessa matrice di parità classica genera sia gli stabilizzatori di tipo X sia quelli di tipo Z, il che permette di trattare separatamente le due famiglie di errori.

I generatori degli stabilizzatori, nella convenzione adottata, sono:

$$\begin{aligned} \text{tipo X: } & IIIXXXX, \quad IXXIIXX, \quad XIXIXIX \\ \text{tipo Z: } & IIIZZZZ, \quad IZZIIZZ, \quad ZIZIZIZ \end{aligned} \tag{8.3}$$

Gli stabilizzatori di tipo Z rilevano gli errori X; quelli di tipo X rilevano gli errori Z; un errore Y, essendo proporzionale a XZ , produce entrambe le sindromi. La struttura di Hamming rende la decodifica trasparente: i tre bit di sindrome di ciascuna famiglia, letti come numero binario, indicano direttamente la *posizione* del qubit colpito (sindrome 011 $\rightarrow$ qubit 3, sindrome 110 $\rightarrow$ qubit 6, e così via), e una lookup table sindrome $\rightarrow$ correzione completa il decoder.

8.5.2 Protocollo di laboratorio

Il protocollo prepara $|0_L\rangle$, verifica la sindrome nulla, inietta X , Z e Y sui sette qubit e costruisce la lookup table; infine stima $p_L(p)$. L'estrazione di sindrome implementata non è fault-tolerant: il modello Monte Carlo applica gli errori ai qubit dati e assume la sindrome ideale. I risultati caratterizzano quindi il codice entro questo contratto, non un dispositivo protetto completo.

8.5.3 Risultati del laboratorio Steane

Lo stato $|0_L\rangle$ è preparato con una rete di Hadamard e CNOT; i sei stabilizzatori restituiscono sindrome nulla sui 2000 campioni, verificando l'appartenenza al *code space*. Il circuito completo è nel Listato B.12.

La syndrome table è biiettiva. Il secondo controllo verifica i 21 errori singoli X , Z e Y . Le sette sindromi non nulle $001, \dots, 111$ identificano in modo biiettivo i sette qubit: gli stabilizzatori Z diagnosticano X , quelli X diagnosticano Z , mentre Y attiva entrambe le famiglie. La tabella completa è quindi la rappresentazione binaria della posizione e non richiede un float dedicato.

Il qubit logico batte il fisico: soppressione quadratica. Sotto un canale depolarizzante di intensità p per qubit, una simulazione Monte Carlo nel formalismo di Pauli (200 000 campioni per punto) applica la correzione a peso minimo e verifica l'eventuale residuo logico. La Figura 8.2 mostra $p_L \propto p^2$ con pendenza log-log 1.94, come atteso per un codice di distanza 3 che fallisce a partire da errori di peso due.

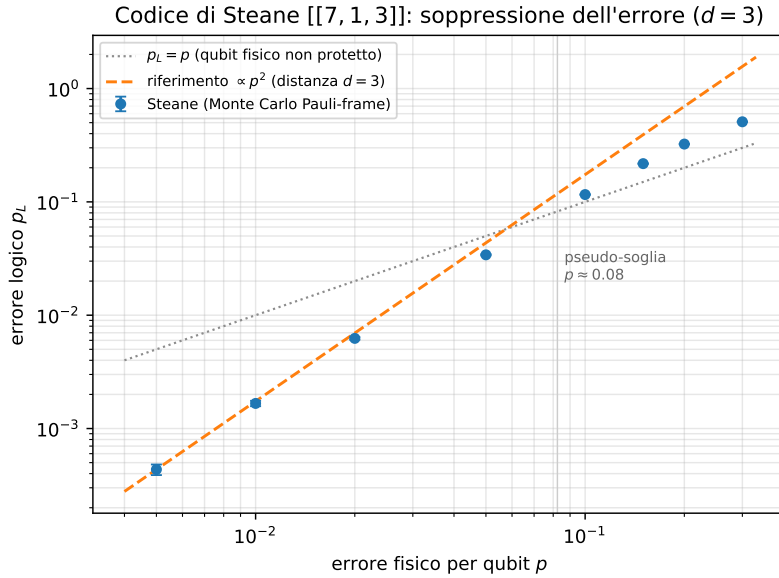


Figura 8.2: La pendenza 1.94 conferma la soppressione quadratica; l'intersezione con $p_L = p$ dà una pseudo-soglia vicina a 0.08.

La pseudo-soglia è circa $p = 0.08$: sotto tale valore il qubit logico è più affidabile del fisico. Rispetto al repetition code, Steane protegge simultaneamente da X , Z e Y , ma resta a distanza fissa e l'estrazione qui simulata non include guasti delle ancille. Il surface code affronta questo limite mediante cicli ripetuti di sindrome rumorosa e distanza variabile.

Capitolo 9

Il Surface Code e la Stima dell'Errore Logico

Lo Steane code del Capitolo 8 ha distanza fissa. Il **surface code** [27, 10] introduce invece una protezione scalabile su griglia bidimensionale: aumentando la distanza d cresce il costo fisico ma, sotto soglia, diminuisce l'errore logico. Il capitolo definisce il decoder MWPM, stima $p_L(p, d)$ con Stim e PyMatching e riassume i due controlli che ne delimitano la validità: rumore non-Pauli e decodifica appresa.

9.1 Il Codice su Griglia: Concetto Operativo

Il surface code dispone su una griglia bidimensionale due popolazioni di qubit: i **qubit dati** (*data qubits*), che portano l'informazione logica, e i **qubit di misura** (*measure qubits*), che estraggono ciclicamente stabilizzatori *locali* — ciascuno coinvolge solo i pochi qubit dati adiacenti sulla griglia. Questa località è il vantaggio architetturale decisivo: ogni operazione richiesta dal codice è un'interazione tra vicini fisici, compatibile con le topologie reali delle QPU superconduttive descritte nel Capitolo 4.

Un patch planare ruotato di distanza d usa d^2 qubit dati per codificare un qubit logico, oltre a un numero dello stesso ordine di ancille di misura: il costo fisico totale cresce quindi come $O(d^2)$. Una distanza maggiore richiede una griglia più grande ma corregge più errori (Eq. 8.1). L'estrazione degli stabilizzatori viene ripetuta per più cicli (*rounds*) consecutivi, perché anche le misure di sindrome sono rumorose: la diagnosi non può basarsi su un singolo ciclo. Una *variazione* dell'esito di uno

stabilizzatore tra due cicli successivi costituisce un **detection event**: è il segnale grezzo da cui il decoder deve ricostruire cosa è accaduto.

9.2 La Decodifica: Minimum Weight Perfect Matching

Nel bulk gli errori fisici sul surface code producono tipicamente detection events *in coppie*; vicino a un bordo, invece, un estremo della catena può terminare sul bordo stesso. In generale le catene di errore sono osservate attraverso i loro estremi nello spazio-tempo. Il problema del decoder è quindi: dati gli eventi osservati, inferire la configurazione di errori *più probabile* che li ha generati. La formulazione standard è un problema di accoppiamento su grafo, includendo i nodi di bordo: il MWPM associa gli eventi minimizzando un peso legato alla probabilità degli errori, e la libreria PyMatching [13] lo implementa in modo efficiente a partire dal modello di errore del circuito.

Se la catena di correzione proposta dal decoder e la catena di errori reale differiscono per un ciclo che attraversa la griglia da parte a parte, la correzione fallisce *silenziosamente*: l'informazione logica è stata alterata senza che alcuna sindrome lo riveli. La frequenza di questi fallimenti è il logical error rate p_L — la metrica centrale di tutta la parte QEC.

Perché servono strumenti dedicati. La stima di p_L richiede milioni di esecuzioni di circuiti con centinaia di qubit: fuori portata per un simulatore statevector, il cui costo cresce come 2^n (Sezione 6.1). I circuiti del surface code, però, sono interamente *stabilizer* (composti di sole operazioni Clifford e misure), e per questa classe il teorema di Gottesman-Knill garantisce la simulabilità classica efficiente. Stim [12] sfrutta esattamente questa proprietà: campiona detection events di circuiti stabilizer a velocità di milioni di shot al secondo. Il rovescio della medaglia — il fatto che Shor *non* sia un circuito stabilizer — è il punto di partenza del Capitolo 10.

La struttura del codice che realizza questa stima — generazione parametrica del circuito, campionamento dei detection events, costruzione del *Detector Error Model* (DEM) e decodifica — è riportata nel Listato B.14 (Sezione B.2).

9.3 Disegno Sperimentale

La campagna usa memory experiment circuit-level con la seguente griglia:

Variabile	Piano	Eseguito
d	3, 5, 7, 9	3, 5, 7, 9
rounds	$d, 2d$	d
p	10^{-4} – 2×10^{-2}	2×10^{-3} – 10^{-2}
base	memory X/Z	memory X/Z

Tabella 9.1: Griglia pianificata e sottoinsieme effettivamente eseguito.

L’ipotesi è un ordinamento opposto delle curve ai due lati della soglia: sotto soglia una distanza maggiore riduce p_L , sopra soglia lo aumenta. Il valore osservato resta specifico di circuito, decoder e modello di rumore.

9.4 Risultati

Stim campiona i circuiti stabilizer e PyMatching applica MWPM. Il modello circuit-level assegna lo stesso parametro p a depolarizzazione dopo i gate Clifford e ai flip di reset e misura. Il numero di campioni è adattato per raccogliere circa duecento fallimenti logici per punto, da 2×10^5 a 4×10^7 shot. La Figura 9.1 mostra la base Z; la base X fornisce lo stesso comportamento qualitativo.

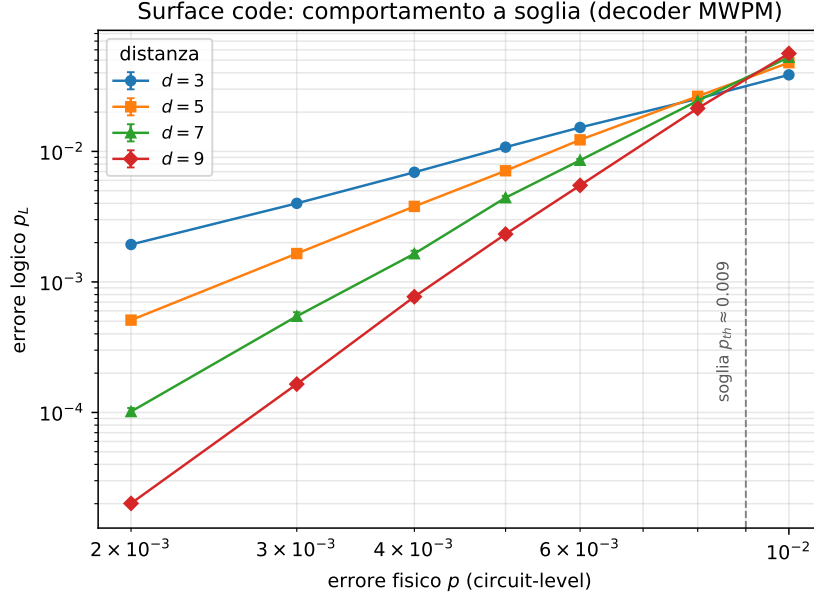


Figura 9.1: Errore logico per $d = 3, 5, 7, 9$ in base Z. Le curve si incrociano vicino a $p_{th} \simeq 0.9\%$: sotto soglia aumentare d sopprime l'errore, sopra soglia lo amplifica.

La soglia è osservata. A $p = 2 \times 10^{-3}$, p_L scende da 1.94×10^{-3} per $d = 3$ a 2.01×10^{-5} per $d = 9$; a $p = 10^{-2}$ l'ordine si inverte (3.9% contro 5.6%). Gli incroci cadono attorno allo 0.9% in base Z e allo 0.7% in base X. Quattro distanze non dimostrano il regime asintotico, ma l'inversione dell'ordinamento verifica il comportamento threshold-like nel modello studiato.

Dalla constatazione alla legge di scala. Sotto soglia si adatta la legge di scala

$$p_L(d, p) = A \left(\frac{p}{p_{th}} \right)^{\lfloor (d+1)/2 \rfloor}, \quad (9.1)$$

dove l'esponente è il numero minimo di errori necessario al fallimento. Sui punti con $p \leq 6 \times 10^{-3}$ si ottengono $A = 3.5 \times 10^{-2}$ e $p_{th} = 0.86\%$ in base Z; in base X, $A = 3.4 \times 10^{-2}$ e $p_{th} = 0.81\%$. Il residuo RMS è 0.08 in $\ln p_L$, coerente con l'incrocio osservato.

La legge può essere invertita per stimare la distanza richiesta *entro questo memory experiment*. Non viene invece trasformata in un numero di qubit per Shor: il p_L di memoria e il proxy per gate del Capitolo 10 hanno unità differenti. Una stima end-to-end richiederebbe il circuito fault-tolerant e una mappatura per operazione

logica. Seed, numerosità adattiva e intervalli binomiali sono registrati negli artefatti; una replica indipendente riproduce i punti entro l'incertezza.

9.5 Oltre il Modello di Pauli: Errori Coerenti e Leakage

Stim rappresenta circuiti Clifford con errori Pauli stocastici; non descrive direttamente sovrarotazioni coerenti o leakage. Le derivazioni dei canali e il loro significato fisico sono raccolti nella Sezione A.4. Ne segue che le soglie appena stimate sono baseline entro il modello circuit-level adottato, non previsioni calibrate di hardware [16].

Un controllo full-state a distanza 3 confronta, su quattro cicli, errori X stocastici e sovrarotazioni R_X con la stessa probabilità marginale. A $p = 3 \times 10^{-2}$ si ottiene $p_L = 0.0767$ nel primo caso e 0.0789 nel secondo: uno scarto relativo del 2.9%, sufficiente a mostrare che i canali non sono intercambiabili ma non a estrapolare segno o ampiezza ad altre distanze. Sul decoder ibrido la differenza fra rumore Pauli e coerente resta dell'ordine di una deviazione standard e non è significativa. Il vantaggio appreso osservato nel seguito dipende quindi soprattutto da errori correlati non decomponibili nel grafo, non dal solo carattere non-Pauli.

9.6 La Decodifica Appresa: Sintesi della Campagna

Il MWPM rappresenta ogni meccanismo d'errore come un arco che collega al più due detection events. Errori correlati su qubit adiacenti possono attivarne quattro e violare questa ipotesi di **decomponibilità**, pur restando errori di Pauli. La campagna completa E1–E12, con protocolli, tabelle e test, è riportata nella Sezione E.1 dell'Appendice E; qui restano i risultati necessari al filo della tesi.

Una rete densa usata *al posto* del matching non vince: a $d = 3$ richiede 10^5 – 10^6 campioni per pareggiare e a $d = 5$ resta inferiore. Usata come residuo, cioè per decidere quando ribaltare l'esito di MWPM, riduce invece p_L del 18–21% a $d = 3$ sotto rumore correlato; il vantaggio scende all'1–3% a $d = 5$ e si annulla a $d = 7$. Nel modello previsto dal matching l'ibrido si astiene e non produce un peggioramento risolto statisticamente.

I controlli E6 ed E7 delimitano l'interpretazione. Una calibrazione perturbata cambia il matching solo dell'1–3% e non apre un vantaggio alla rete; un decoder analitico più espressivo, BP+OSD, recupera invece il 17–27% a $d = 5$. Gli esperimenti

E8–E11 mostrano inoltre che impilare BP+OSD e residuo non somma i guadagni: i metodi convergono allo stesso pavimento nel regime correlato.

Il criterio conclusivo è quindi strutturale: un modello appreso aggiunge valore quando sfrutta una classe d'errore che il metodo analitico non può rappresentare, non quando ne corregge soltanto parametri imprecisi. Prima di introdurre apprendimento occorre dunque confrontarsi con il miglior decoder analitico disponibile; nel protocollo studiato la nicchia utile è reale ma limitata alle distanze piccole.

Capitolo 10

Analisi di Sensibilità: Shor sotto un Proxy di Errore per Gate

I Capitoli 8 e 9 studiano la correzione d'errore su circuiti dedicati, mentre il Capitolo 7 misura la degradazione di Shor sotto un modello di rumore fisico. Collegare direttamente i due insiemi di risultati richiederebbe una compilazione fault-tolerant completa: codifica dei registri, estrazione ripetuta della sindrome, decoder, operazioni logiche e distillazione delle risorse non-Clifford. Questa tesi non simula tale architettura. Il presente capitolo svolge invece un esperimento di **sensibilità fenomenologica**: applica al circuito compilato un errore Pauli indipendente per gate, indicato con p_L , e misura come varia la probabilità di fattorizzazione. Il pedice “L” identifica il ruolo concettuale del parametro, ma non autorizza a sostituirvi direttamente le metriche logiche aggregate dei capitoli precedenti.

10.1 L'architettura a quattro livelli

La parte sperimentale separa quattro oggetti: Shor ideale, Shor sotto rumore fisico, benchmark QEC isolati e proxy Pauli per gate compilato. La separazione è necessaria perché Stim/PyMatching caratterizza efficientemente circuiti stabilizer, mentre Shor contiene aritmetica e rotazioni non-Clifford; sostituire i suoi dodici qubit con blocchi codificati, ancille e cicli di sindrome richiederebbe una simulazione fault-tolerant completa. Il quarto livello risponde quindi a una domanda di sensibilità e non riceve automaticamente i tassi aggregati del terzo.

10.2 Definizione operativa del proxy

In M8, p_L è definito come la **probabilità totale di applicare un Pauli non identità dopo ciascun gate compilato incluso nel modello**. Per un gate su q qubit il canale distribuisce p_L uniformemente sui $4^q - 1$ Pauli non identità. Poiché Qiskit Aer parametrizza il canale depolarizzante con λ_q , lo script usa

$$\lambda_1 = \frac{4}{3}p_L, \quad \lambda_2 = \frac{16}{15}p_L. \quad (10.1)$$

Il rumore è applicato a $\mathbf{sx}/\mathbf{x}$ e $\mathbf{cx}$; le rotazioni $\mathbf{rz}$ sono considerate virtuali. Si tratta quindi di un proxy uniforme *per gate nella base compilata*, non di una probabilità per ciclo di correzione, per round di sindrome o per intero blocco logico.

Il circuito è quello canonico $N = 15$, $a = 7$, con otto qubit di conteggio e quattro di lavoro, compilato deterministicamente nella base $\mathbf{rz}/\mathbf{sx}/\mathbf{x}/\mathbf{cx}$. Ogni punto M8 aggrega 20 repliche da 4096 shot; il successo vale uno quando frazioni continue e massimo comun divisore restituiscono fattori non banali. Manifest e hash della netlist fissano l'identità del circuito.

Questa definizione rende riproducibile la domanda sperimentale:

come cambia il successo di questa specifica implementazione di Shor al crescere di un errore Pauli indipendente e uniforme per gate?

Non consente invece di rispondere, senza un ulteriore modello architetturale, a quale distanza di surface code corrisponda un dato punto della curva.

10.3 Il contesto delle stime fault-tolerant

Le stime crittografiche includono routing, cicli di sindrome, operazioni logiche e fabbriche di stati magici; non si ricavano estrapolando $N = 15$ [6]. La curva seguente è quindi locale al circuito e al canale dichiarati, non una previsione di risorse hardware.

10.4 Risultati M8 e M13

M8: successo per singola misura

La griglia preregistrata copre $p_L = 0\text{--}0.5$. Gli intervalli sono Wilson al 95%; la monotonia è controllata con copertura simultanea di Bonferroni sui confronti adiacenti.

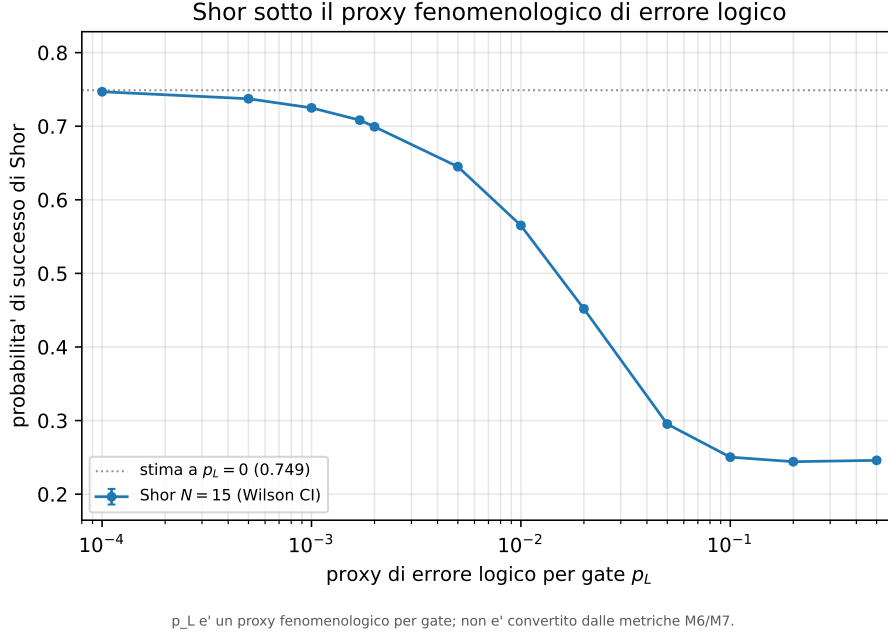


Figura 10.1: Successo per singola misura in funzione dell'errore Pauli per gate compilato. Le metriche aggregate dei benchmark QEC non sono proiettate sulla curva perché non hanno la stessa unità.

La curva è compatibile con una funzione non crescente, ma non ogni passo adiacente è significativo. Il successo scende da 0.7489 a $p_L = 0$ a 0.4518 per $p_L = 0.02$ e raggiunge, oltre 0.1, un plateau vicino a 0.245. Quest'ultimo dipende dagli esiti che il post-processing di $N = 15$ accetta e non è universale.

Ripetizioni indipendenti

Il successo per misura non è il successo dopo più tentativi indipendenti. Se ciascun tentativo ha probabilità P_s ,

$$P(k) = 1 - (1 - P_s)^k. \quad (10.2)$$

Due tentativi portano così la stima a 0.937 nel caso ideale e a circa 0.811 per $p_L = 0.01$. Una soglia dell'80% è pertanto cumulativa, non un requisito sul singolo run.

M13: TOP-4 con pareggi espliciti

M13 separa successo per misura, TOP-1 e TOP-4 sullo stesso istogramma. TOP-4 resta 1.000 fino a $p_L = 0.05$, ma scende a 0.915 per $p_L = 0.1$ e a 0.670 per $p_L = 0.2$:

recupera candidati su questa piccola istanza, senza certificare la conservazione dello stato. Protocollo, tabella completa e analisi dei pareggi sono nella Sezione E.2 dell'Appendice E.

10.4.1 Discussione

M8 mostra che ridurre l'errore Pauli per gate riporta il successo verso il limite ideale dell'istanza. Non stabilisce quale codice produca quel parametro: errore Steane, errore del memory experiment surface-code e guadagno del decoder sono grandezze aggregate diverse e non possono essere inserite direttamente sulla curva. Il passaggio richiederebbe gate logici, numero di cicli per operazione, decoder e canali residui espliciti, comprese le operazioni non-Clifford.

$N = 15$ resta inoltre un'istanza didattica, nella quale TOP-K beneficia di pochi picchi verificabili classicamente. Il capitolo caratterizza quindi la sensibilità del circuito controllato; non dimostra la fattibilità fault-tolerant di Shor su scala crittografica.

Capitolo 11

Sviluppi Futuri

I risultati della tesi valgono entro un perimetro preciso: le campagne rumorose riguardano $N = 15$, mentre $N = 21$ e $N = 35$ sono validati idealmente. Le estensioni più utili devono quindi ridurre la distanza fra circuito dimostrativo, correzione logica e dispositivo reale, senza estrapolare i dati oltre il modello che li ha generati.

11.1 Scalabilità e Validazione su Hardware Reale

La prima priorità è ridurre il costo dell'aritmetica modulare. Approximate QFT con soglia esplicita, addizionatori alternativi, sintesi orientata al gate set e riduzione delle operazioni controllate sono direzioni concrete; ciascuna variante deve però conservare la verità funzionale della moltiplicazione, il ripristino delle ancille, la coerenza di fase e la distribuzione QPE ideale. Il solo conteggio delle porte non dimostra la correttezza.

Una campagna di scalabilità dovrebbe variare separatamente larghezza dell'istanza e ordine moltiplicativo r : moduli più grandi possono avere ordini più semplici e non costituiscono automaticamente test più difficili. Solo dopo la riduzione del circuito diventerebbe informativa una campagna rumorosa su $N = 21/35$.

In parallelo, il protocollo $N = 15$ può essere ripetuto su una QPU registrando backend, timestamp, calibrazione, layout, routing e job identifier. Un modello costruito da uno snapshot reale permetterebbe di misurare il *simulation gap* rispetto ai preset uniformi; non eliminerebbe però profondità e connettività del circuito. Il confronto deve mantenere budget di shot e regole di analisi comuni.

11.2 Dalla QEC a uno Shor Fault-Tolerant

L'analisi di sensibilità usa un parametro fenomenologico per gate, mentre i benchmark surface-code misurano un memory experiment. Il passo necessario per collegarli è una compilazione fault-tolerant che specifichi, per ogni operazione logica, canale residuo e costo: operazioni Clifford e non-Clifford, magic-state distillation, cicli di correzione, latenza del decoder e feed-forward. Solo questa mappatura consentirebbe una previsione end-to-end e una stima credibile delle risorse.

Anche la decodifica appresa può essere estesa, privilegiando modelli che incorporino la località del reticolo, la storia temporale delle sindromi e l'informazione analogica di misura. Il confronto deve restare appaiato con MWPM e includere distanze maggiori, deriva del rumore e costo di inferenza: un guadagno sul logical error rate è utile solo se compatibile con la latenza del ciclo QEC.

11.3 Trasferibilità del Post-processing e dell'Apprendimento

TOP-K è trasferibile quando ogni candidato dispone di una verifica classica economica e il costo delle K prove è esplicito. Grover e altri problemi di ricerca possono offrire casi adatti; VQE e QAOA richiedono invece strategie rivolte alla stima degli osservabili, non una semplice lista di candidati. Ogni estensione deve confrontarsi con una baseline TOP-1 sugli stessi campioni e controllare che l'aumento di K non renda il successo quasi certo per puro effetto combinatorio.

Per il classificatore sull'output non basta cambiare architettura: occorre una variabile obiettivo legata al costo operativo, separazione del test anche per backend o periodo di calibrazione e confronto con regole semplici basate su massa dei picchi ed entropia. La direzione più promettente resta l'uso dell'apprendimento sulla sindrome, dove il modello può sfruttare correlazioni non rappresentate dal decoder analitico prima che l'informazione venga persa nella misura finale.

Capitolo 12

Conclusioni

Questo lavoro ha chiesto quanta informazione di Shor possa essere recuperata dall'output rumoroso e quando sia necessario intervenire durante il calcolo. La risposta è un confine operativo: TOP-K valorizza candidati ancora presenti nell'istogramma, mentre gli errori che ne distruggono la struttura richiedono sindromi, decodifica e operazioni logiche protette.

12.1 Le Domande di Ricerca

DR1 — Effetto dei canali di rumore. Nel modello uniforme per $N = 15$, l'aumento dell'errore a due qubit attenua la struttura utile della QPE; il proxy di nessun evento decade più rapidamente e non coincide con la probabilità di successo. Gli altri sweep, con 30 repliche, non sostengono un ordinamento causale fra le fonti di rumore. TOP-4 resta più robusto perché verifica più candidati dello stesso istogramma.

DR2 — Estrazione della sindrome. Il codice a ripetizione misura parità tramite gli stabilizzatori Z_1Z_2 e Z_2Z_3 , senza misurare direttamente lo stato logico. La simulazione riproduce $p_L = 3p^2 - 2p^3$ entro l'incertezza, ma protegge un solo tipo di errore per volta.

DR3 — Codice di Steane. Steane $[[7, 1, 3]]$ corregge un errore Pauli arbitrario su un singolo qubit e la corrispondenza sindrome-errore è verificata sui 21 casi. La pendenza log-log misurata, 1.94, è coerente con la soppressione quadratica; distanza fissa ed estrazione non fault-tolerant ne limitano la scalabilità.

DR4 — Soglia del surface code. Nel memory experiment circuit-level, le curve per $d = 3, 5, 7, 9$ mostrano incroci attorno allo 0.9% in base Z e allo 0.7% in base X. Sotto tale regione aumentare d riduce p_L ; il parametro fisico della simulazione non è però direttamente confrontabile con quello della campagna Shor.

DR5 — Sensibilità di Shor all'errore logico. Il successo per singola esecuzione diminuisce al crescere del proxy Pauli per gate, mentre le ripetizioni indipendenti aumentano la probabilità cumulativa. La curva è una sensibilità fenomenologica: un tasso di memory experiment non può essere assegnato a ogni gate di Shor senza una compilazione fault-tolerant.

DR6 — Decodifica appresa. Il vantaggio emerge quando il rumore contiene correlazioni non rappresentate dal grafo di matching. Il decoder residuale può allora correggere una parte degli errori lasciati da MWPM; con la rete densa adottata il beneficio si riduce all'aumentare della distanza, quindi il risultato identifica un regime utile e non una sostituzione generale del decoder analitico.

12.2 Verifica delle Ipotesi

Tabella 12.1: Esito delle ipotesi nel perimetro sperimentale della tesi.

Ipotesi	Esito	Evidenza essenziale
Il classificatore riduce le iterazioni	smentita	M2 non migliora M1 e TOP-4 migliora M2 nei due preset.
$F1 > 0.85$ e $AUC > 0.90$	confermata, non sufficiente	Le soglie sono superate, ma misurano la predizione TOP-1 e non il guadagno operativo.
Il vantaggio è robusto oltre il preset di riferimento	non supportata	Il vantaggio ML non compare nei due preset; $N = 21/35$ non sono testati sotto rumore.
Soppressione quadratica a distanza 3	confermata nel modello	Ripetizione segue la curva teorica e Steane ha pendenza 1.94.
Comportamento di soglia del surface code	confermato nel modello	Le curve a distanza crescente si incrociano e invertono l'ordinamento.
Trasferimento diretto di p_L a Shor	non dimostrato	Il proxy per gate e i tassi dei memory experiment hanno granularità differenti.

Il risultato della prima ipotesi chiarisce perché le sole metriche del classificatore non bastino: un'etichetta può essere prevedibile senza corrispondere a una decisione utile. L'ablazione TOP-4 è quindi parte necessaria della verifica, non una baseline accessoria.

12.3 Contributi del Lavoro

La tesi fornisce una demo che separa esecuzione ideale e rumorosa; una validazione funzionale dell'aritmetica per le istanze considerate; un confronto appaiato M1–TOP-4–M2 che attribuisce il guadagno alla corretta regola decisionale; e una progressione QEC da ripetizione a surface code con limiti e unità di misura dichiarati. Il contributo trasversale è il confronto dell'apprendimento in due sedi diverse, output e sindrome, usando controfattuali semplici e decoder analitici come riferimenti.

12.4 Limiti

La campagna Shor usa un simulatore, preset uniformi e l'istanza didattica $N = 15$; non comprende topologia, routing, crosstalk, leakage o deriva di calibrazione. TOP-K beneficia di una verifica classica economica e gli sweep, limitati a 30 repliche, non permettono una gerarchia causale completa dei canali. Le istanze $N = 21/35$ sono validate soltanto in ideale. Nella parte QEC, Steane non impiega un'estrazione fault-tolerant, il decoder appreso è una rete densa e i memory experiment non condividono la definizione di p_L del proxy per gate. Ne segue che le curve ottenute non sono una stima delle risorse crittografiche di uno Shor fault-tolerant.

12.5 Il Criterio che Unifica il Lavoro

Un metodo aggiuntivo produce valore quando usa informazione che il riferimento non sfrutta e il beneficio supera il costo introdotto. TOP-4 verifica candidati già osservati; il decoder residuale interviene sulle correlazioni che il matching non rappresenta. Un modello più complesso non è invece utile se replica una regola semplice, apprende un'etichetta degenerare o opera dopo che l'informazione è stata distrutta.

Applicato a Shor, il criterio separa nettamente post-processing e correzione. Il primo recupera informazione presente; la seconda protegge il processo che la genera. La scalabilità richiede quindi operazioni logiche fault-tolerant, mentre

l'apprendimento può contribuire dove dispone di struttura diagnostica ulteriore e viene verificato contro una baseline forte.

Appendice A

Trattazione Teorica Estesa

A.1 Fondamenti del Calcolo Quantistico

Questa sezione sviluppa in forma estesa i richiami di meccanica quantistica che la Sezione 3.1 del Capitolo 3 usa nella misura minima necessaria a seguire l'algoritmo.

A.1.1 Dalla Meccanica Classica alla Meccanica Quantistica

La meccanica quantistica descrive il comportamento della materia e dell'energia alle scale atomiche e subatomiche, dove i principi della fisica classica cessano di essere validi. I suoi postulati, formulati nella prima metà del Novecento da Heisenberg, Schrödinger, Dirac e Bohr, introducono concetti radicalmente diversi da quelli classici: lo stato di un sistema fisico non è determinato in modo assoluto, ma è descritto da una **funzione d'onda** la cui evoluzione è governata dall'equazione di Schrödinger.

Il calcolo quantistico traduce questi principi fisici in risorse computazionali. La **sovrapposizione** consente a un bit quantistico di trovarsi in una combinazione lineare di 0 e 1 simultaneamente; l'**entanglement** crea correlazioni non classiche tra qubit distinti; l'**interferenza** permette di amplificare le soluzioni corrette e cancellare quelle errate.

A.1.2 I Postulati della Meccanica Quantistica

Il formalismo matematico della meccanica quantistica poggia su quattro postulati fondamentali, che costituiscono l'assiomatica della teoria e il punto di partenza rigoroso per la descrizione di qualsiasi sistema quantistico [7].

1. **Postulato dello spazio degli stati.** Ad ogni sistema fisico isolato è associato uno **spazio di Hilbert** $\mathcal{H}$, detto *spazio degli stati* del sistema. Lo stato del sistema è completamente descritto da un vettore unitario $|\psi\rangle \in \mathcal{H}$, detto *vettore di stato* o *ket*. Per un sistema a due livelli (qubit), $\mathcal{H} = \mathbb{C}^2$ e lo stato generale è $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ con $|\alpha|^2 + |\beta|^2 = 1$.
2. **Postulato dell'evoluzione.** L'evoluzione di un sistema quantistico chiuso è descritta da una **trasformazione unitaria**: se lo stato del sistema all'istante t_1 è $|\psi(t_1)\rangle$, allora all'istante t_2 esso si trova nello stato

$$|\psi(t_2)\rangle = U(t_1, t_2) |\psi(t_1)\rangle \quad (\text{A.1})$$

dove $U(t_1, t_2)$ è un operatore unitario ($U^\dagger U = I$) che dipende dagli istanti t_1 e t_2 . In forma differenziale, l'evoluzione è governata dall'**equazione di Schrödinger**:

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = \hat{H} |\psi(t)\rangle \quad (\text{A.2})$$

dove $\hat{H}$ è l'**Hamiltoniano** del sistema — un operatore hermitiano ($\hat{H} = \hat{H}^\dagger$) che rappresenta l'energia totale — e $\hbar = h/(2\pi)$ è la costante di Planck ridotta.¹

3. **Postulato della misura.** Le misure quantistiche sono descritte da un insieme $\{M_m\}$ di **operatori di misura**, indicizzati dagli esiti m , soggetti alla relazione di completezza:

$$\sum_m M_m^\dagger M_m = I \quad (\text{A.3})$$

Se il sistema si trova nello stato $|\psi\rangle$ prima della misura, la probabilità di ottenere l'esito m è:

$$p(m) = \langle \psi | M_m^\dagger M_m | \psi \rangle \quad (\text{A.4})$$

¹Nel calcolo quantistico a porte, ogni porta è la realizzazione discreta di questo postulato: la matrice unitaria U descrive l'evoluzione del registro di qubit in un passo computazionale. L'Hamiltoniano fisico che implementa una porta specifica dipende dall'architettura hardware (impulsi a microonde per qubit superconduttori, impulsi laser per ioni intrappolati, ecc.).

e lo stato immediatamente dopo la misura con esito m è il vettore normalizzato:

$$|\psi'\rangle = \frac{M_m |\psi\rangle}{\sqrt{p(m)}} \quad (\text{A.5})$$

Nel caso della **misura proiettiva in base computazionale** — il caso standard nel calcolo quantistico — gli operatori sono i proiettori $M_m = |m\rangle \langle m|$, e la probabilità di ottenere m si riduce alla **regola di Born**: $p(m) = |\langle m|\psi\rangle|^2$. La misura è un'operazione *non unitaria* e *irreversibile*: il collasso dello stato distrugge irrecuperabilmente le ampiezze che non corrispondono all'esito osservato.

4. **Postulato dei sistemi composti.** Lo spazio degli stati di un sistema fisico composto da n sottosistemi con spazi $\mathcal{H}_1, \dots, \mathcal{H}_n$ è il **prodotto tensoriale**:

$$\mathcal{H} = \mathcal{H}_1 \otimes \mathcal{H}_2 \otimes \dots \otimes \mathcal{H}_n \quad (\text{A.6})$$

Se ciascun sottosistema i si trova nello stato $|\psi_i\rangle$, lo stato del sistema composto è $|\psi_1\rangle \otimes |\psi_2\rangle \otimes \dots \otimes |\psi_n\rangle$. Esistono tuttavia stati del sistema composto — gli stati **entangled** — che non possono essere scritti in questa forma fattorizzata per nessuna scelta degli stati individuali: la misura di un sottosistema modifica istantaneamente le probabilità degli esiti di misura sull'altro, indipendentemente dalla distanza fisica tra i due.

Questi quattro postulati forniscono il quadro assiomatico completo entro cui si collocano tutti i concetti che seguono: il qubit (Postulato 1), le porte quantistiche come trasformazioni unitarie (Postulato 2), la misura e il collasso dello stato (Postulato 3), i registri multi-qubit e l'entanglement (Postulato 4).

A.1.3 Il Qubit

Definizione Matematica

L'unità fondamentale del calcolo quantistico è il **qubit** (*quantum bit*). Mentre un bit classico assume uno dei due valori $\{0, 1\}$, un qubit è descritto da un vettore unitario nello spazio di Hilbert bidimensionale $\mathcal{H} = \mathbb{C}^2$.²

Introducendo la **notazione di Dirac**³, i due stati base computazionali sono:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (\text{A.7})$$

Lo stato generico di un qubit è la **sovrapposizione**:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (\text{A.8})$$

dove $|\alpha|^2$ e $|\beta|^2$ rappresentano le probabilità di ottenere rispettivamente 0 e 1 al momento della misura. Il vincolo di normalizzazione $|\alpha|^2 + |\beta|^2 = 1$ garantisce che le probabilità sommino a 1.

La Sfera di Bloch

Ogni stato puro di un qubit può essere visualizzato come un punto sulla superficie della **sfera di Bloch**, una sfera unitaria in $\mathbb{R}^3$.⁴ Parametrizzando le ampiezze complesse tramite angoli reali $\theta \in [0, \pi]$ e $\phi \in [0, 2\pi]$:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\phi} \sin\left(\frac{\theta}{2}\right) |1\rangle \quad (\text{A.9})$$

²Lo **spazio di Hilbert** prende il nome dal matematico tedesco David Hilbert (1862–1943), uno dei più influenti matematici del XX secolo. In meccanica quantistica indica uno spazio vettoriale complesso dotato di prodotto interno, che fornisce il formalismo matematico per descrivere gli stati di un sistema fisico.

³La **notazione bra-ket** fu introdotta dal fisico britannico Paul Adrien Maurice Dirac (1902–1984), Premio Nobel per la Fisica nel 1933. Il simbolo $|\psi\rangle$ è detto *ket*, mentre $\langle\psi|$ è detto *bra*; il prodotto interno $\langle\phi|\psi\rangle$ forma il *bracket*, da cui il nome.

⁴La **sfera di Bloch** deve il suo nome al fisico svizzero-americano Felix Bloch (1905–1983), Premio Nobel per la Fisica nel 1952 (insieme a Edward Purcell) per lo sviluppo della spettroscopia a risonanza magnetica nucleare (Risonanza Magnetica Nucleare (*Nuclear Magnetic Resonance*) (NMR)). La stessa notazione T_1 e T_2 per i tempi di decoerenza proviene dal formalismo NMR di Bloch.

I poli nord e sud corrispondono agli stati $|0\rangle$ e $|1\rangle$, mentre i punti sull'equatore rappresentano sovrapposizioni equiprobabili con fasi diverse.

La figura corrispondente è riportata nel corpo (Figura 3.1).

A.1.4 Sistemi Multi-Qubit e Prodotto Tensoriale

Un sistema di n qubit è descritto da un vettore nello spazio di Hilbert $\mathcal{H}^{\otimes n} = \mathbb{C}^{2^n}$, ottenuto come prodotto tensoriale degli spazi individuali. Uno stato generico di n qubit è:

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1 \quad (\text{A.10})$$

La dimensione esponenziale 2^n dello spazio di Hilbert è la fonte del potere computazionale dei sistemi quantistici: con $n = 300$ qubit, lo spazio degli stati ha dimensione 2^{300} , superiore al numero di atomi nell'universo osservabile.

Entanglement

Due qubit sono detti **entangled** quando il loro stato congiunto non può essere scritto come prodotto tensoriale degli stati individuali. Gli stati di Bell costituiscono la base canonica degli stati entangled a due qubit:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (\text{A.11})$$

$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) \quad (\text{A.12})$$

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) \quad (\text{A.13})$$

$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \quad (\text{A.14})$$

Lo stato $|\Phi^+\rangle$ non può essere fattorizzato come $|\psi_A\rangle \otimes |\psi_B\rangle$ per nessuna scelta di $|\psi_A\rangle$ e $|\psi_B\rangle$: misurare il primo qubit come 0 garantisce che il secondo sia 0, indipendentemente dalla distanza fisica tra i due sistemi.

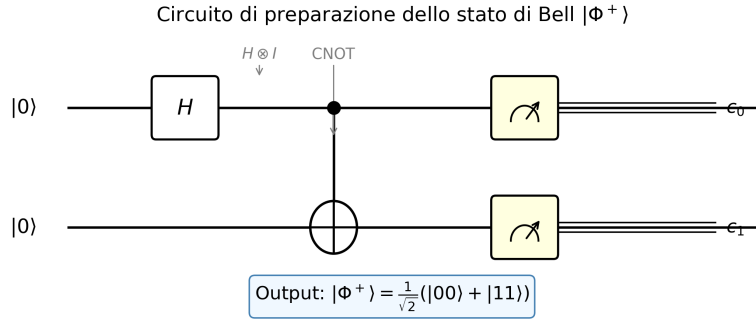


Figura A.1: Circuito quantistico per la preparazione dello stato di Bell $|\Phi^+\rangle$. Una porta di Hadamard H posta sul qubit di controllo crea la sovrapposizione $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$; la successiva porta CNOT introduce le correlazioni di entanglement, producendo lo stato $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.

L'Entanglement come Risorsa Fisica Fragile

L'entanglement non è soltanto una proprietà astratta degli stati quantistici: è la **risorsa fisica** che gli algoritmi quantistici consumano per ottenere il loro vantaggio computazionale. Nell'algoritmo di Shor, il registro di controllo e il registro di lavoro devono restare entangled per tutta la durata della phase estimation affinché la Quantum Fourier Transform possa estrarre il periodo corretto. La perdita anche parziale di questa correlazione è sufficiente a degradare il risultato.

La fragilità dell'entanglement ha un'origine fisica precisa. Un sistema quantistico reale non è mai perfettamente isolato dall'ambiente che lo circonda: campi elettromagnetici residui, vibrazioni termiche del reticolo cristallino e fluttuazioni di carica si accoppiano continuamente ai qubit. Quando questo accoppiamento avviene, il sistema entra in uno stato entangled *con l'ambiente*, trasferendo ad esso parte dell'informazione di fase. Dal punto di vista del solo sistema, le ampiezze di interferenza si riducono: il qubit non è più in una sovrapposizione pura $\alpha|0\rangle + \beta|1\rangle$, ma in un **miscuglio statistico** in cui la fase relativa tra $|0\rangle$ e $|1\rangle$ è persa. Questo processo prende il nome di **decoerenza** [28].

Il tempo caratteristico entro cui questa perdita di fase diventa apprezzabile è T_2 , il **tempo di coerenza di fase** (*dephasing time*). Esso misura per quanto tempo un qubit riesce a mantenere la propria sovrapposizione coerente prima che l'interazione con l'ambiente la degradi. La crescita della profondità di Shor rende questo budget critico, ma il solo ordine asintotico $O(n^3)$ non determina una soglia universale in bit: contano compilazione, parallelismo, idle e architettura fault-tolerant. La trattazione quantitativa e i limiti del confronto diretto con T_2 sono nel Capitolo 4.

A.1.5 Porte Quantistiche

Le operazioni su qubit sono rappresentate da **matrici unitarie**: $U^\dagger U = I$. La condizione di unitarietà garantisce la reversibilità del calcolo e la conservazione della norma (delle probabilità).

Porte a Singolo Qubit

Le principali porte a singolo qubit sono:

Le tre **matrici di Pauli** X , Y , Z costituiscono, insieme all'identità I , una base per lo spazio delle matrici 2×2 hermitiane a traccia nulla, e giocano un ruolo centrale sia nella descrizione degli stati del qubit sia nella caratterizzazione dei canali di rumore.

Porta di Pauli-X (NOT quantistico / Bit Flip):

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle \quad (\text{A.15})$$

È l'analogo quantistico del NOT classico: inverte i valori di $|0\rangle$ e $|1\rangle$.

Porta di Pauli-Y:

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle \quad (\text{A.16})$$

Combina un bit flip e un phase flip: è equivalente (a meno di una fase globale) alla composizione iXZ . Compare naturalmente nella decomposizione di rotazioni generali su qubit e nella descrizione del canale di *depolarizzazione*.

Porta di Pauli-Z (Phase Flip):

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle \quad (\text{A.17})$$

Lascia invariato $|0\rangle$ e aggiunge una fase -1 a $|1\rangle$: è un *phase flip*.

Le tre matrici di Pauli soddisfano le seguenti relazioni algebriche fondamentali:

$$X^2 = Y^2 = Z^2 = I \quad (\text{A.18})$$

$$XY = iZ, \quad YZ = iX, \quad ZX = iY \quad (\text{A.19})$$

$$\{X, Y\} = \{Y, Z\} = \{X, Z\} = 0 \quad (\text{A.20})$$

dove $\{A, B\} = AB + BA$ è l'**anticommutatore**. Le prime due relazioni mostrano che le matrici di Pauli formano l'algebra di Lie $\mathfrak{su}(2)$; la terza — l'anticommutazione — è alla base della struttura dei codici di correzione degli errori quantistici, dove X e Z rappresentano rispettivamente errori di bit flip e phase flip indipendenti.

Porta di Hadamard:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \equiv |+\rangle, \quad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \equiv |-\rangle \quad (\text{A.21})$$

La porta di Hadamard è fondamentale: applicata a n qubit inizializzati in $|0\rangle^{\otimes n}$, genera la **sovrapposizione uniforme** di tutti i 2^n stati base:

$$H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \quad (\text{A.22})$$

Porta di fase R_ϕ :

$$R_\phi = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix} \quad (\text{A.23})$$

Casi particolari: $T = R_{\pi/4}$ (T-gate) e $S = R_{\pi/2}$ (S-gate o Phase gate).

Porte a Due Qubit

Porta CNOT:

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad (\text{A.24})$$

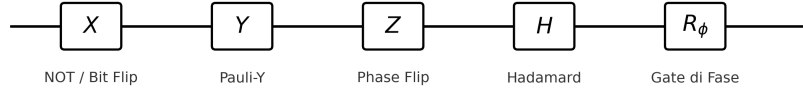
Applica una NOT sul qubit *target* se e solo se il qubit *control* è $|1\rangle$. È la porta a due qubit più comune e, insieme alle porte a singolo qubit, forma un insieme universale.

Porta Toffoli (Controlled-Controlled NOT (porta di Toffoli) (CCNOT)):

Generalizzazione a tre qubit; applica NOT sul terzo qubit solo se entrambi i qubit di controllo sono $|1\rangle$.

Principali porte quantistiche: simboli circuitali

Porte a singolo qubit:



Porta a due qubit:

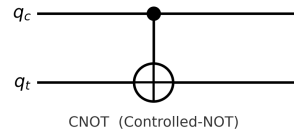


Figura A.2: Simboli circuitali delle principali porte quantistiche a singolo qubit (X , Y , Z , H , R_ϕ) e della porta CNOT a due qubit. Nella porta CNOT, il punto pieno indica il qubit di controllo q_c e il cerchio con croce indica il qubit target q_t .

A.1.6 Circuiti Quantistici

Un **circuito quantistico** è una sequenza ordinata di porte quantistiche applicata a un registro di qubit. I circuiti si leggono da sinistra a destra, con ogni filo orizzontale che rappresenta l'evoluzione temporale di un qubit. La composizione di porte corrisponde al prodotto matriciale degli operatori unitari associati.

L'equivalente quantistico del teorema di universalità classico afferma che qualunque trasformazione unitaria su n qubit può essere **approssimata con precisione arbitraria** da una sequenza finita di porte prese da un insieme universale, tipicamente $\{H, T, \text{CNOT}\}$. La distinzione fra approssimazione e decomposizione esatta non è pedanteria: le sequenze finite su un insieme di porte fissato sono numerabili, mentre le trasformazioni unitarie non lo sono, sicché una decomposizione esatta è impossibile in generale. Il teorema di Solovay–Kitaev garantisce che l'approssimazione a precisione ε richiede $O(\log^c(1/\varepsilon))$ porte, con c costante piccola: il costo dell'approssimazione cresce dunque solo polilogaritmicamente, ed è questa la ragione per cui l'universalità resta praticamente utile.

A.1.7 Misura Quantistica

La **misura** in base computazionale di uno stato $|\psi\rangle = \sum_x \alpha_x |x\rangle$ produce il risultato x con probabilità $|\alpha_x|^2$, e collassa irrevocabilmente lo stato in $|x\rangle$. La misura è una operazione *non unitaria* e *non reversibile*: una volta effettuata, l'informazione sulle ampiezze è irreversibilmente persa.

Questa caratteristica impone che gli algoritmi quantistici siano progettati per amplificare selettivamente l'ampiezza della risposta corretta prima della misura, in modo da ottenere il risultato voluto con alta probabilità.

A.1.8 Interferenza Quantistica

L'**interferenza** è il meccanismo fondamentale attraverso cui gli algoritmi quantistici realizzano il loro vantaggio computazionale. Le ampiezze quantistiche sono numeri complessi e possono combinarsi costruttivamente (quando hanno lo stesso segno/fase) o distruttivamente (quando hanno fase opposta).

Un algoritmo quantistico efficiente è costruito in modo che:

- Le ampiezze degli stati **sbagliati** si cancellino per **interferenza distruttiva**;
- Le ampiezze degli stati **corretti** si sommino per **interferenza costruttiva**.

Questo meccanismo è alla base sia dell'algoritmo di Grover sia dell'algoritmo di Shor (Capitolo 3), dove la Quantum Fourier Transform realizza l'interferenza necessaria al ritrovamento del periodo.

A.1.9 Modello di Complessità Quantistica

La complessità di un algoritmo quantistico è misurata in termini di numero di porte (equivalente delle operazioni elementari classiche) e di profondità del circuito (numero di livelli di porte parallele). Si definiscono le classi di complessità quantistiche più rilevanti:

- *Bounded-error Quantum Polynomial time* (BQP): classe dei problemi risolvibili in tempo polinomiale con probabilità di errore al più $1/3$ da un computer quantistico. È la classe centrale del calcolo quantistico.
- *Quantum Merlin-Arthur* (QMA): analogo quantistico di NP; problemi la cui soluzione può essere verificata efficientemente da un computer quantistico.

Le inclusioni $\mathbf{P} \subseteq \mathbf{BPP} \subseteq \mathbf{BQP} \subseteq \mathbf{PSPACE}$ sono *dimostrate*; ciò che resta aperto è se siano **strette**. Non è noto se BQP contenga propriamente P o BPP — cioè se esista davvero un problema che un computer quantistico risolva efficientemente e uno classico no — né se sia propriamente contenuto in PSPACE. Una dimostrazione di separazione stretta fra BQP e BPP implicherebbe $\mathbf{P} \neq \mathbf{PSPACE}$, un risultato che la teoria della complessità non ha ancora raggiunto: è questa la ragione per cui il vantaggio quantistico, per Shor come per gli altri algoritmi, resta una congettura ampiamente creduta e non un teorema.

Gerarchia delle classi di complessità nel calcolo quantistico

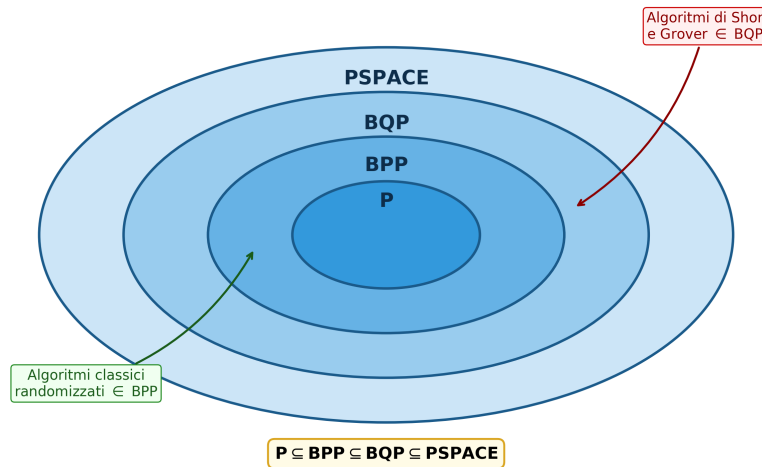


Figura A.3: Gerarchia delle classi di complessità nel calcolo quantistico. La classe BQP cattura i problemi efficientemente risolvibili da un computer quantistico; le inclusioni mostrate ($P \subseteq BPP \subseteq BQP \subseteq PSPACE$) sono dimostrate, mentre è congetturata — e non dimostrata — la loro *strettezza*. Gli algoritmi di Shor e Grover risiedono in BQP; che la fattorizzazione non appartenga a P è ritenuto probabile ma non provato.

A.2 L'Anatomia Fisica di una QPU

Il Capitolo [efchap:rumore](#) isola di questa materia i soli tre fatti che ricorrono nel seguito; questa sezione ne dà la trattazione completa.

A.2.1 L'Anatomia Fisica di una QPU

Per capire dove nascono gli errori bisogna prima capire com'è fatta la macchina. Una QPU superconduttiva è un chip criogenico che contiene *esclusivamente* i qubit e le linee fisiche che li collegano: contrariamente all'intuizione, **le porte logiche non risiedono nel chip**. Aprendo una CPU classica si trovano registri e porte logiche incise nel silicio; aprendo una QPU si trovano solo i qubit. Le operazioni sono realizzate dall'**elettronica di controllo esterna**, che invia impulsi a microonde convogliati ai qubit tramite guide d'onda: “far passare un qubit in un gate” significa in realtà inviargli un'onda sagomata che ne ruota lo spin dell'angolo desiderato (90°,

$30^\circ, 10^\circ, \dots$). La Figura A.4 schematizza questa architettura per una QPU a 5 qubit con topologia a catena lineare.

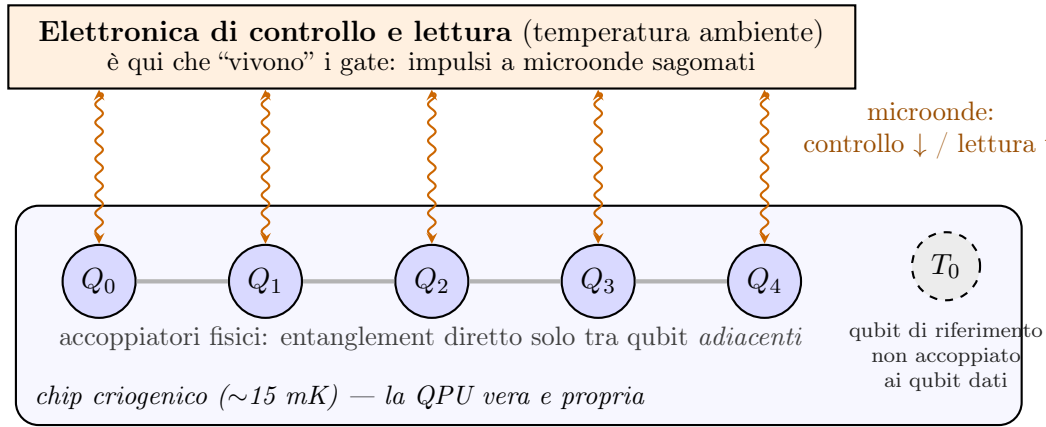


Figura A.4: Schema didattico di una QPU superconduttiva a 5 qubit con topologia a catena. Nel chip criogenico risiedono qubit, risonatori e accoppiatori; le operazioni sono pilotate dall’elettronica esterna mediante linee di controllo e lettura. L’interazione a due qubit diretta è disponibile solo lungo gli accoppiamenti disegnati (Q_1 – Q_2 sì, Q_0 – Q_2 mediante routing). T_0 rappresenta un qubit di riferimento non accoppiato alla catena, ma ancora dotato delle linee necessarie a preparazione e misura: non fornisce una misura “pura” di T_1 e non elimina perdite, Purcell effect o altre sorgenti di decoerenza.

La Figura A.5 mostra come questa architettura si presenta nella realtà: all’esterno il sistema integrato (criostato, elettronica e schermature racchiusi nella teca), all’interno della sala macchine i cilindri criogenici sospesi con il fascio di tubi e cavi coassiali che convogliano le microonde di controllo e lettura — le “guide d’onda” dello schema precedente.



(a) IBM Quantum System One (Ehningen, Germania)



(b) Criostati IBM Q con il cablaggio di controllo

Figura A.5: La QPU reale. (a) Il sistema integrato IBM Quantum System One: il cilindro nero al centro è il criostato che contiene il chip, circondato dall'elettronica di controllo. (b) Criostati IBM Q in sala macchine: dall'alto arrivano i tubi per la refrigerazione e i cavi coassiali che convogliano le microonde di controllo e lettura verso il chip, sospeso sul fondo del cilindro a ~ 15 mK. *Crediti: (a) IBM Research, licenza CC BY 2.0; (b) Connie Zhou per IBM, pubblico dominio; entrambe via Wikimedia Commons.*

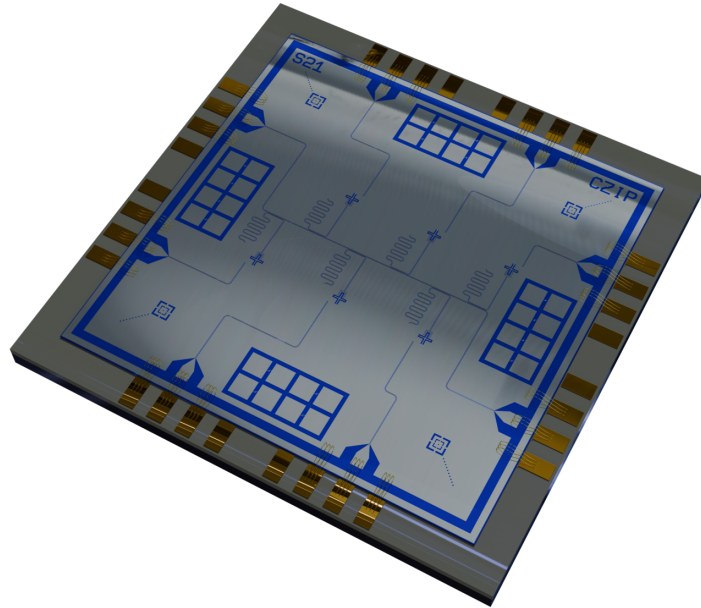


Figura A.6: Resa tridimensionale di un chip quantistico superconduttivo con 6 qubit transmon e 12 giunzioni di test. Si riconoscono gli elementi dello schema di Figura A.4: i qubit con i loro risonatori, le linee di accoppiamento, le piazzole dorate per il collegamento alle guide d'onda e — ai quattro angoli — le strutture di test isolate, l'analogo reale del qubit T_0 . *Credito: Onri Jay Benally, licenza CC BY 4.0, via Wikimedia Commons.*

Questa architettura ha tre conseguenze dirette per gli errori.

La lettura distrugge lo stato — fisicamente. Anche la misura avviene tramite microonde: per leggere un qubit lo si “bombarda” con un’onda di sonda e si osserva come essa torna indietro — un modo di risposta corrisponde a $|0\rangle$, un altro a $|1\rangle$. L’atto stesso di sondare blocca la dinamica del qubit: è per questo che la misura di uno stato quantistico lo distrugge *realmente*, non per convenzione matematica. Ed è per questo che nei circuiti le letture stanno tutte alla fine, in parallelo: misurare un solo qubit lasciando gli altri in sovrapposizione significa inviare un’onda che, per quanto focalizzata, perturba anche i vicini e può farli precipitare.

Il budget di operazioni: coerenza vs velocità dei gate. Le prestazioni di una QPU dipendono dal rapporto tra due tempi: quanto a lungo il qubit resta coerente e quanto dura una singola operazione. Con una finestra di coerenza di $100\ \mu\text{s}$ e gate da $\sim 25\ \text{ns}$, si eseguono $\sim 4\,000$ operazioni; una macchina con coerenza superiore ma gate

più lenti può eseguirne *meno*. È il rapporto che conta, non il singolo parametro — ed è questo rapporto a decidere se il circuito di Shor per una data istanza è fisicamente eseguibile. La Tabella A.1 confronta gli ordini di grandezza delle due principali tecnologie.

	Superconduttori (IBM, IQM, ...)	Ioni intrappolati (IonQ, Quantinuum, ...)
Tempo di coerenza	50–300 μ s	da secondi a minuti
Durata gate (1 qubit)	~ 15 –40 ns	~ 1 –100 μ s
Durata gate (2 qubit)	~ 60 –500 ns	~ 0.1 –1 ms
Operazioni per finestra	$\sim 10^3$ – 10^4	$\sim 10^3$
Lettura	più esposta a errori e interferenze	più stabile e affidabile
Crosstalk	presente (accoppiamento capacitivo)	ridotto
Velocità di esecuzione	molto alta	bassa (gate lenti)

Tabella A.1: Ordini di grandezza indicativi delle due principali tecnologie di QPU. La coerenza lunghissima degli ioni intrappolati non si traduce in più operazioni utili, perché i gate sono da 10^3 a 10^4 volte più lenti: il budget di operazioni per finestra di coerenza è simile, ma i superconduttori lo spendono molto più in fretta. Gli ioni sono invece più robusti su lettura e crosstalk.

Il bordo di coerenza. Gli errori non sono distribuiti uniformemente nel tempo: finché il calcolo occupa una piccola frazione della finestra di coerenza, la probabilità di decadimento è bassa; quando il circuito si spinge *verso la fine* della finestra — il “bordo di coerenza” — la probabilità che il qubit sia già decaduto al momento della misura cresce rapidamente. La Figura A.7 illustra il concetto: è la ragione per cui circuiti profondi come quello di Shor, che consumano gran parte del budget, producono letture inaffidabili anche quando “tecnicamente” rientrano nella finestra.

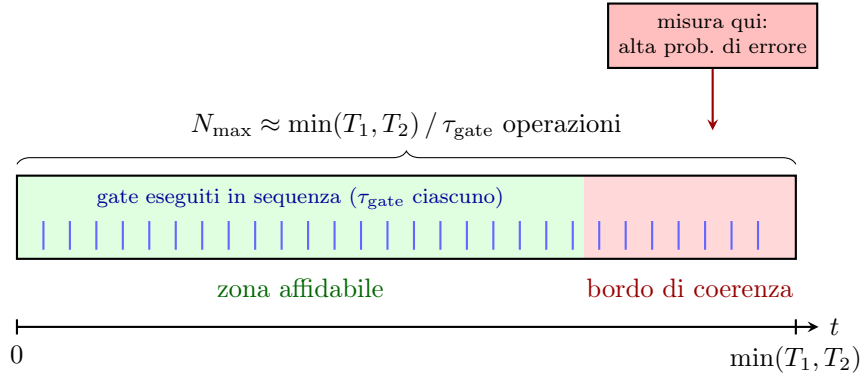


Figura A.7: Il “budget” di operazioni di un qubit: la finestra di coerenza $\min(T_1, T_2)$ divisa per la durata del singolo gate dà il numero massimo di operazioni eseguibili $N_{\max}$. Un circuito che consuma solo la prima parte della finestra opera in zona affidabile; un circuito profondo come quello di Shor spinge la misura verso il *bordo di coerenza*, dove la probabilità che il qubit sia già decaduto — e che la lettura restituisca un valore errato — è massima.

La topologia decide chi parla con chi. Le QPU differiscono anche per geometria: catene lineari, griglie (heavy-hex di IBM), connettività completa. Su una catena, l’entanglement tra qubit non adiacenti richiede un “cross-entanglement” attraverso i qubit intermedi: ogni passaggio disperde energia e accorcia la vita dello stato condiviso — più cross si fanno, prima lo stato decade. La topologia determina quindi sia *quali* circuiti si possono mappare in modo efficiente, sia *dove* il crosstalk colpirà. Questo aspetto — insieme al fatto che i parametri di coerenza variano da qubit a qubit dello stesso chip — è ripreso nel Capitolo 9, dove la località delle interazioni sulla griglia è il fondamento architetturale del surface code.

A.3 L’Algoritmo di Shor — Approfondimenti e Stato dell’Arte

Questa sezione raccoglie il materiale di contesto e di approfondimento dell’algoritmo di Shor: lo stato delle realizzazioni sperimentali, i miglioramenti algoritmici recenti, la roadmap verso il calcolo fault-tolerant e il confronto con l’algoritmo di Grover. Il Capitolo 3 ne riporta la sintesi.

A.3.1 Sviluppi Recenti e Stato dell'Arte

Implementazioni su Hardware Reale

Nonostante la solidità teorica, l'esecuzione di Shor su hardware reale rimane limitata a istanze molto piccole a causa dei requisiti di profondità circuitale. I principali risultati sperimentali sono:

- **2001 – NMR (Vandersypen et al.):** prima dimostrazione sperimentale di Shor su un sistema a 7 spin nucleari; fattorizzazione di $N = 15$ in 3×5 [29].
- **2016 – Ioni intrappolati (Monz et al.):** implementazione scalabile su 5 qubit ionici; fattorizzazione di $N = 15$ con un'architettura modulare [30].
- **2021 – Qubit superconduttori International Business Machines (IBM):** dimostrazione compilata della fattorizzazione di $N = 21$ su processori IBM Quantum utilizzando 5 qubit [5].

Queste istanze evidenziano quanto la distanza dalle chiavi crittograficamente rilevanti sia ancora enorme. Una specifica stima di risorse per fattorizzare un numero a L bit riporta circa $5L+1$ qubit logici e $72L^3$ gate [31]; costanti e overhead dipendono però dall'architettura aritmetica e dal protocollo fault-tolerant. In ogni caso, i valori per RSA-2048 restano fuori portata per i processori NISQ attuali.

Miglioramenti Algoritmici: Regev 2023

Un risultato teorico rilevante è stato pubblicato nel 2023 da Oded Regev [32]. La proposta fattorizza un intero di n bit eseguendo indipendentemente circa $\sqrt{n} + 4$ circuiti, ciascuno con $\tilde{O}(n^{3/2})$ gate, seguiti da post-processing classico polinomiale. Il confronto con Shor non può quindi essere ridotto al solo conteggio di gate di una singola esecuzione: aumentano il numero di run e, nella costruzione originale, anche lo spazio quantistico; inoltre l'articolo dichiara non chiaro l'effettivo vantaggio implementativo. Il risultato è un miglioramento asintotico importante, non una dimostrazione di praticabilità su dispositivi NISQ.

Roadmap IBM verso il Quantum Fault-Tolerant

IBM ha delineato una roadmap concreta verso il calcolo quantistico fault-tolerant [33]:

- **2025:** processori Loon e Nighthawk (120 qubit con 218 coupler sintonizzabili), con dimostrazione dei componenti chiave per la fault-tolerance tramite codici qLDPC;
- **2026:** target di vantaggio quantistico su problemi pratici;
- **2029:** sistema Quantum Starling con 200 qubit logici e oltre 10^8 operazioni quantistiche.

Questi traguardi sono obiettivi industriali dichiarati da IBM, soggetti a revisione, e non stabiliscono quando una macchina avrà le risorse logiche necessarie a rompere RSA-2048. Rafforzano la necessità di preparare la migrazione, ma non forniscono una previsione crittografica.

Le sfide hardware principali per l'esecuzione di Shor su istanze crittograficamente rilevanti sono analizzate in dettaglio nel Capitolo 4, mentre l'implementazione pratica con Qiskit e le tecniche di ottimizzazione basate su intelligenza artificiale sono presentate nel Capitolo 6.

A.3.2 Contesto: Confronto con l'Algoritmo di Grover

Per collocare correttamente l'algoritmo di Shor nel panorama dell'informatica quantistica, è utile confrontarlo con l'altro grande risultato algoritmico degli anni Novanta: l'algoritmo di Grover per la ricerca non strutturata [34].

Il Problema di Grover

Dato un insieme di N elementi e una funzione oracolo $f : \{0, \dots, N-1\} \rightarrow \{0, 1\}$, l'obiettivo è trovare x^* tale che $f(x^*) = 1$. Classicamente, nel caso peggiore, sono necessarie $O(N)$ interrogazioni. Grover (1996) dimostrò che un algoritmo quantistico, sfruttando il meccanismo di *amplitude amplification*, può trovare la soluzione in sole $O(\sqrt{N})$ interrogazioni – uno **speedup quadratico** che è stato dimostrato ottimale: nessun algoritmo quantistico può fare meglio [35, 36].

Differenze Fondamentali

I due algoritmi rappresentano due regimi distinti di vantaggio quantistico:

Caratteristica	Shor	Grover
Problema target	Fattorizzazione intera / period finding	Ricerca non strutturata
Speedup	Esponenziale rispetto al miglior classico	Quadratico: $O(\sqrt{N})$ vs $O(N)$
Tecnica quantistica core	QFT + Phase Estimation	Amplitude Amplification
Ottimalità	Non dimostrata in assoluto	Dimostrata ottimale [35]
Impatto crittografico	Rompe RSA, DH, ECDSA (chiave pubblica)	Dimezza sicurezza AES/SHA (simmetrica)
Requisiti hardware	$O(\log N)$ qubit, alta profondità	$O(\log N)$ qubit, bassa profondità

Tabella A.2: Confronto tra l'algoritmo di Shor e l'algoritmo di Grover

L'impatto sulla crittografia simmetrica di Grover è significativo ma gestibile: raddoppiare la lunghezza delle chiavi (da AES-128 a AES-256) è sufficiente a ripristinare il livello di sicurezza. Al contrario, non esiste un analogo rimedio per RSA: l'unica risposta è la migrazione verso gli standard post-quantum discussi nella sezione precedente. È per questa ragione che l'algoritmo di Shor rappresenta la minaccia quantistica più urgente e motivante per la ricerca sul calcolo quantistico applicato.

Tutta l'analisi svolta in questo capitolo ha però assunto un modello di calcolo *ideale*: porte unitarie perfette, coerenza illimitata, nessuna interazione con l'ambiente.

Su hardware reale questi presupposti non reggono, e la distanza tra il modello teorico e l'esecuzione fisica è precisamente il nodo critico che rende ancora irraggiungibile la fattorizzazione di numeri crittograficamente rilevanti. Comprendere le origini e gli effetti del rumore quantistico è quindi il passo necessario per valutare le reali prospettive dell'algoritmo di Shor.

A.4 I Canali di Rumore: Derivazioni Complete

I quattro canali quantistici che modellano il rumore sono mappe *Completely Positive Trace-Preserving* (CPTP) in rappresentazione di Kraus, secondo l'Eq. 4.1. Il Capitolo 4 ne riporta il canale depolarizzante per esteso, essendo quello su cui poggia la campagna sperimentale, e l'enunciato compatto degli altri tre; qui se ne danno le derivazioni complete.

Canale Bit Flip

Il canale **bit flip** modella un errore di tipo X discreto: con probabilità p il qubit viene invertito ($|0\rangle \leftrightarrow |1\rangle$), con probabilità $1 - p$ rimane invariato. Gli operatori di Kraus sono $K_0 = \sqrt{1 - p} I$ e $K_1 = \sqrt{p} X$, e il canale agisce come:

$$\mathcal{E}_{BF}(\rho) = (1 - p) \rho + p X \rho X \quad (\text{A.25})$$

Sulla sfera di Bloch, le componenti r_y e r_z vengono contratte di un fattore $(1 - 2p)$ mentre la componente r_x rimane invariata. Per $p = 1/2$ il canale annulla le componenti y e z , ma conserva r_x : equivale a una completa dephasing nella base degli autostati di X , non alla distruzione completa dello stato.

Canale Phase Flip

Il canale **phase flip** modella la perdita di informazione di fase senza cambiamento di popolazione: la sovrapposizione decade senza che l'energia venga dissipata. Con operatori di Kraus $K_0 = \sqrt{1 - p} I$ e $K_1 = \sqrt{p} Z$, il canale è:

$$\mathcal{E}_{PF}(\rho) = (1 - p) \rho + p Z \rho Z \quad (\text{A.26})$$

L'effetto sulla matrice densità è immediato:

$$\mathcal{E}_{PF} \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix} = \begin{pmatrix} \rho_{00} & (1-2p) \rho_{01} \\ (1-2p) \rho_{10} & \rho_{11} \end{pmatrix} \quad (\text{A.27})$$

Le popolazioni ρ_{00} e ρ_{11} rimangono invariate, mentre le **coerenze** ρ_{01} , ρ_{10} vengono sopprese di un fattore $(1-2p)$. Questo è il canale che descrive il decadimento T_2 : parametrizzando $p(t) = \frac{1}{2}(1 - e^{-t/T_2})$, si ottiene $\rho_{01}(t) = \rho_{01}(0) e^{-t/T_2}$.

Canale di Amplitude Damping

Il canale di **amplitude damping** modella il rilassamento energetico T_1 : il qubit decade spontaneamente dallo stato eccitato $|1\rangle$ allo stato fondamentale $|0\rangle$ con probabilità $\gamma = 1 - e^{-t/T_1}$. Gli operatori di Kraus sono:

$$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \quad K_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix} \quad (\text{A.28})$$

Il canale agisce sulla matrice densità come:

$$\mathcal{E}_{AD}(\rho) = \begin{pmatrix} \rho_{00} + \gamma \rho_{11} & \sqrt{1-\gamma} \rho_{01} \\ \sqrt{1-\gamma} \rho_{10} & (1-\gamma) \rho_{11} \end{pmatrix} \quad (\text{A.29})$$

La popolazione ρ_{11} decade verso ρ_{00} con tasso γ , mentre le coerenze vengono sopprese di $\sqrt{1-\gamma}$. A differenza dei canali precedenti, questo canale è **non simmetrico**: sulla sfera di Bloch contrae la sfera verso il polo nord $|0\rangle$, non verso il centro:

$$r_x \rightarrow \sqrt{1-\gamma} r_x, \quad r_y \rightarrow \sqrt{1-\gamma} r_y, \quad r_z \rightarrow (1-\gamma) r_z + \gamma \quad (\text{A.30})$$

Per $t \rightarrow \infty$ ($\gamma \rightarrow 1$), qualsiasi stato converge a $|0\rangle \langle 0|$, indipendentemente dallo stato iniziale.

Appendice B

Strumenti, Ambiente e Codice

B.1 Strumenti e Ambiente di Simulazione — Trattamento Esteso

Questa sezione estende la Sezione 6.1 del Capitolo 6, che ne riporta le sole conclusioni operative.

La scelta degli strumenti di simulazione non è neutrale. In informatica quantistica, il framework adottato determina quali modelli di rumore sono disponibili, verso quale hardware è possibile traslare i circuiti e con quali risultati della letteratura è possibile confrontarsi direttamente. Questo capitolo descrive gli strumenti utilizzati nella parte sperimentale di questa tesi, motivando ciascuna scelta e collocandola rispetto alle alternative disponibili.

B.1.1 Qiskit: il Framework Quantistico di Riferimento

Qiskit (*Quantum Information Science Kit*) è un *Software Development Kit* (SDK) open-source sviluppato da IBM Quantum per la programmazione, la simulazione e l'esecuzione di algoritmi quantistici [11]. Rilasciato pubblicamente nel 2017, offre integrazione diretta con l'hardware IBM ed è uno dei framework ampiamente usati nella ricerca su circuiti quantistici a corto termine [5, 37].

Architettura del Framework

Qiskit è organizzato in tre componenti principali, ciascuno con un ruolo distinto nel ciclo di vita di un esperimento quantistico:

- **Qiskit (core)** — il nucleo del framework, già noto come Qiskit Terra. Fornisce le primitive per costruire i circuiti, compilarli verso un backend target tramite `transpile` e accedere alle interfacce `Sampler` ed `Estimator`. Gestisce la pipeline dalla descrizione astratta fino all’invio al simulatore o all’hardware reale.
- **Qiskit Aer** — il modulo di simulazione classica ad alte prestazioni. Implementa diversi metodi di simulazione esatta e approssimata, con supporto nativo per modelli di rumore realistici. È il componente centrale di questa tesi e viene descritto in dettaglio nella Sezione B.1.2.
- **Qiskit IBM Runtime** — l’interfaccia per l’esecuzione su processori IBM Quantum reali tramite cloud (`qiskit-ibm-runtime`). Fornisce le primitive `Sampler` e `Estimator` ottimizzate per l’hardware reale, con supporto per tecniche di soppressione del rumore come il *Dynamical Decoupling* e il *Gate Twirling* [38].

Perché Qiskit e non un’alternativa

La scelta di Qiskit è stata formalizzata nell’incontro con il relatore del 18 marzo 2026, in seguito all’analisi del tutorial ufficiale IBM per l’algoritmo di Shor [38]. La decisione è motivata da quattro ragioni convergenti.

Prima ragione: controllo esplicito del modello. Qiskit Aer permette di comporre canali depolarizzanti, termici e di readout e di legarli a un gate set dichiarato. Questa flessibilità consente esperimenti controllati e riproducibili; non rende però i preset uniformi della tesi direttamente equivalenti alla calibrazione di una QPU reale.

Seconda ragione: copertura delle primitive necessarie. Qiskit Aer espone nello stesso ambiente i canali depolarizzanti, termici, di readout e coerenti richiesti dal disegno. Altri framework, fra cui Cirq [39] e PennyLane [40], offrono anch’essi simulazione rumorosa; la scelta non pretende di stabilire una graduatoria generale, ma riduce le conversioni rispetto al codice Qiskit adottato.

Terza ragione: disponibilità dell’implementazione di riferimento. Il circuito dell’algoritmo di Shor utilizzato in questa tesi è tratto da un’implementazione open-source in Qiskit [41]. Questo punto di partenza, validato dalla letteratura [5], ha reso naturale mantenere Qiskit come framework unico lungo l’intero pipeline sperimentale.

Quarta ragione: allineamento con le fonti implementative. Diversi lavori usati come riferimento adottano Qiskit [5, 37, 42]. Condividere il framework facilita il confronto delle API e dei circuiti, ma non rende direttamente comparabili risultati ottenuti con gate set, versioni, transpiler o noise model differenti.

Tabella B.1: Confronto tra i principali framework per la simulazione quantistica con rumore.

Framework	Sviluppatore	Simulatore con rumore	Target hardware primario
Qiskit + Aer	IBM Quantum	Completo (T1, T2, depolarz., readout)	IBM superconduttori
Cirq	Google Quantum AI	Parziale	Google superconduttori
PennyLane	Xanadu	Limitato	Vari (plug-in)
Forest / PyQuil	Rigetti	Parziale	Rigetti superconduttori

B.1.2 Qiskit Aer: il Simulatore

Qiskit Aer è il modulo di simulazione classica di Qiskit. Il suo componente principale è `AerSimulator`, un simulatore ad alte prestazioni che supporta più metodi di simulazione selezionabili a seconda della dimensione del circuito e del tipo di analisi richiesta.

Metodi di Simulazione

`AerSimulator` espone i seguenti metodi principali tramite il parametro `method`:

- `statevector` — simulazione del vettore di stato completo. Richiede 2^n ampiezze complesse in memoria ed è il metodo usato dalla campagna rumorosa principale su $N = 15$.
- `density_matrix` — evolve la matrice densità ρ anziché il vettore di stato, permettendo di modellare canali di rumore non unitari in modo esatto. Richiede 4^n elementi e riduce il limite pratico a circa 25 qubit.
- `stabilizer` — simulazione efficiente di circuiti Clifford puri (senza porte non-Clifford come T o R_z arbitrarie). Scala a migliaia di qubit ma non è applicabile all'algoritmo di Shor, che utilizza rotazioni di fase arbitrarie.

- `matrix_product_state` — rappresenta lo stato come rete di tensori; senza troncamenti imposti resta una rappresentazione controllata, ma il costo cresce con l'entanglement. È usato in M8/M13 per contenere il costo delle molte repliche.

La scelta è quindi per campagna, non globale: `statevector` per la prima fase N15 e `matrix_product_state` per la sensibilità M8/M13. Le istanze N21/N35 sono validate idealmente ma escluse dalle campagne rumorose generali.

Modelli di Rumore

Il modulo `qiskit_aer.noise` fornisce le primitive per costruire modelli di rumore compositi. Ogni errore è rappresentato come un canale quantistico che viene applicato ai gate specificati, con la possibilità di assegnare errori diversi a qubit diversi (rumore eterogeneo) o un errore uniforme a tutti i qubit (*all-qubit error*). I tipi di errore disponibili includono:

- `depolarizing_error` — errore depolarizzante su gate a 1 o 2 qubit, parametrizzato da λ nella convenzione Aer; la probabilità Pauli non-identità è $3\lambda/4$ o $15\lambda/16$.
- `thermal_relaxation_error` — modella il rilassamento verso lo stato fondamentale (T_1) e la perdita di coerenza di fase (T_2), con un tempo di gate configurabile.
- `ReadoutError` — matrice di confusione che descrive la probabilità di misurare $|0\rangle$ come $|1\rangle$ e viceversa.
- Errori coerenti — applicazione di una rotazione indesiderata su ogni gate, modellabile tramite operatori unitari arbitrari.

Il modello di rumore usato nella prima fase combina errori depolarizzanti, rilassamento termico e readout error con parametri uniformi. È un modello illustrativo per isolare i meccanismi, non uno snapshot di `ibm_marrakesh`: non include coupling map, routing, errori per qubit, crosstalk o deriva temporale. La configurazione dettagliata è riportata nel Capitolo 2.

Transpilazione

Prima dell'esecuzione, la funzione `transpile` riscrive il circuito nella base esplicita `rz/sx/x/cx` e applica ottimizzazioni strutturali. La campagna usa il livello di ottimizzazione 2 e il seed del transpiler 20260819; questa coppia, insieme all'hash della sequenza compilata, è registrata in ogni artefatto. La base esplicita impedisce che porte composte restino opache e sfuggano al canale di rumore previsto.

B.1.3 Accelerazione GPU con Qiskit Aer

La libreria `qiskit-aer-gpu` estende `AerSimulator` con il supporto per l'accelerazione tramite Unità di Elaborazione Grafica (*Graphics Processing Unit*) (GPU) NVIDIA attraverso CUDA. Il backend `statevector` è interamente eseguito sulla GPU: il vettore di stato viene allocato nella memoria VRAM e le operazioni di gate sono eseguite come moltiplicazioni matrice-vettore parallelizzate su migliaia di core CUDA.

Motivazione pratica. L'accelerazione era stata valutata per la generazione dei dataset e per le campagne replicate. Non vengono però riportati fattori di accelerazione ipotetici come risultati sperimentali, poiché l'ambiente canonico finale usa la Unità di Elaborazione Centrale (*Central Processing Unit*) (CPU).

Specifiche hardware. La GPU prevista è una NVIDIA GeForce RTX 4070 con 12 GB di memoria VRAM e 5888 core CUDA (architettura Ada Lovelace). La sola allocazione di uno statevector `complex128` avrebbe un tetto teorico di 29 qubit, inferiore nella pratica per memoria di lavoro e overhead. Questo limite dimensionale non garantisce inoltre la fattibilità: circuiti molto profondi e traiettorie rumorose moltiplicano il costo temporale, ed è proprio ciò che esclude N21/N35 dalla campagna rumorosa.¹

Esito: l'accelerazione non è stata utilizzabile. Quanto sopra descrive il piano iniziale. In sede di implementazione il backend GPU si è rivelato inutilizzabile nell'ambiente *Windows Subsystem for Linux* (WSL) adottato — il driver CUDA riconosce la scheda, ma il backend C++ di Aer solleva un errore a runtime sull'esecuzione dei circuiti — e **tutti gli esperimenti riportati in questa tesi sono stati eseguiti su CPU**. La diagnosi completa e le conseguenze sui tempi di esecuzione sono nella Sezione 6.6. Il vincolo di memoria resta quello indicato, riferito però alla memoria di sistema anziché alla VRAM.

¹La memoria del solo vettore è $2^n \times 16$ byte. Con 12 GB decimali, $\log_2(12 \times 10^9/16) \simeq 29.48$, quindi il massimo intero teorico è $n = 29$.

B.1.4 scikit-learn: la Libreria per il Classificatore

Il Metodo 2 richiede l'addestramento di un classificatore binario. La libreria scelta è scikit-learn [14], una delle librerie di machine learning più consolidate nell'ecosistema Python scientifico.

La motivazione principale è pragmatica: scikit-learn espone un'interfaccia unificata (`fit` / `predict`) per un ampio catalogo di classificatori — Random Forest, SVM, MLP, e altri — permettendo di confrontarli sullo stesso dataset con minime modifiche al codice. Questo facilita la selezione sperimentale del modello migliore senza dover introdurre dipendenze da framework più pesanti (TensorFlow, PyTorch) per un classificatore che, per il problema in esame, si prevede non richieda architetture profonde.

B.1.5 Ambiente di Esecuzione: WSL con Ubuntu

Le simulazioni vengono eseguite in ambiente Linux tramite WSL (versione 2) su Windows 11. La scelta è motivata dalla necessità di mantenere la compatibilità con l'ecosistema Linux di Qiskit e delle librerie scientifiche Python, garantendo al contempo l'integrazione con i driver CUDA per WSL forniti da NVIDIA.

WSL 2 esegue un kernel Linux completo in una macchina virtuale leggera. L'ambiente Python è isolato in un virtual environment dedicato, garantendo la riproducibilità delle dipendenze e la compatibilità delle versioni tra le librerie usate; le versioni esatte sono versionate nel file `requirements.txt` della repository. È in questo stesso ambiente che l'accesso alla GPU si è rivelato non praticabile (Sezione 6.6).

Con l'ambiente così definito, il Capitolo 6 descrive come Qiskit, Aer e scikit-learn si integrano nell'architettura del pipeline sperimentale: dalla costruzione del circuito di Shor alla generazione del dataset di addestramento, fino all'inferenza del classificatore.

B.2 Codice Sorgente dei Componenti Implementati

Questa sezione raccoglie gli estratti necessari a verificare le scelte implementative del Capitolo 6. Gli output numerici non sono codificati nei listati: ogni campagna li salva in un artefatto JSON schema 2 corredato di manifest.

```
python3 -m venv ~/quantum-env
```

```
source ~/quantum-env/bin/activate
python -m pip install -r requirements.txt
python -c "import qiskit, qiskit_aer; print(qiskit.__version__)"
```

Listing B.1: Installazione dell'ambiente Python e delle librerie necessarie

```
RuntimeError: Simulation device "GPU" is not supported on this
system
```

Listing B.2: Errore GPU osservato su WSL durante i test

```
def inverse_qft(n_qubits):
    qc = QuantumCircuit(n_qubits, name='QFT_dagger')
    for q in range(n_qubits // 2):
        qc.swap(q, n_qubits - q - 1)
    for j in range(n_qubits):
        for m in range(j):
            qc.cp(-np.pi / 2 ** (j - m), m, j)
        qc.h(j)
    return qc
```

Listing B.3: Implementazione della Quantum Fourier Transform inversa

```
def c_amod15(a, power):
    if a not in [2, 7, 8, 11, 13]:
        raise ValueError("base non supportata per N=15")
    U = QuantumCircuit(4)
    for _ in range(power):
        if a in [2, 13]:
            U.swap(0, 1); U.swap(1, 2); U.swap(2, 3)
        if a in [7, 8]:
            U.swap(2, 3); U.swap(1, 2); U.swap(0, 1)
        if a == 11:
            U.swap(1, 3); U.swap(0, 2)
        if a in [7, 11, 13]:
            U.x(range(4))
    return U.control(annotated=False)
```

Listing B.4: Moltiplicazione modulare N15 corretta: l'operazione $\times a$ viene ripetuta $power$ volte

```

def shor_circuit(N, a, n_count):
    if N != 15:
        return shor_circuit_beauregard(N, a, n_count)
    n_work = 4
    qc = QuantumCircuit(n_count + n_work, n_count)
    qc.h(range(n_count))
    qc.x(n_count + 3)    #  $|1\rangle$  nella convenzione del registro
                        textbook
    for j in range(n_count):
        power = 2 ** j
        if power % 4 != 0:    #  $U^4 = I$  per  $a=7 \bmod 15$ 
            qc.append(c_amod15(a, power),
                      [j] + list(range(n_count, n_count +
                                         n_work)))
    qc.barrier()
    qc.append(inverse_qft(n_count), range(n_count))
    qc.measure(range(n_count), range(n_count))
    return qc

def extract_factors(measured_value, n_count, N, a):
    if measured_value == 0:
        return None, None
    phase = measured_value / 2 ** n_count
    r = Fraction(phase).limit_denominator(N).denominator
    if r % 2:
        return None, None
    for candidate in (gcd(a ** (r // 2) - 1, N),
                      gcd(a ** (r // 2) + 1, N)):
        if 1 < candidate < N:
            return candidate, N // candidate
    return None, None

```

Listing B.5: Circuito N15 e post-processing classico

```

BASIS_GATES = ('rz', 'sx', 'x', 'cx')
GATE_TIME_1Q_NS = 50.0
GATE_TIME_2Q_NS = 300.0

def compose_errors(errors):

```

```

out = None
for error in errors:
    if error is not None:
        out = error if out is None else out.compose(error)
return out

def build_noise_model(eps_1q, eps_2q, t1_ns, t2_ns, p_ro=0.02)
:
    # eps_1q ed eps_2q sono i lambda di depolarizing_error.
    thermal_1q = thermal_relaxation_error(
        t1_ns, t2_ns, GATE_TIME_1Q_NS)
    thermal_one_2q = thermal_relaxation_error(
        t1_ns, t2_ns, GATE_TIME_2Q_NS)
    thermal_2q = thermal_one_2q.tensor(thermal_one_2q)
    error_1q = compose_errors([
        depolarizing_error(eps_1q, 1) if eps_1q else None,
        thermal_1q,
    ])
    error_2q = compose_errors([
        depolarizing_error(eps_2q, 2) if eps_2q else None,
        thermal_2q,
    ])
    nm = NoiseModel()
    nm.add_all_qubit_quantum_error(error_1q, ['sx', 'x'])
    nm.add_all_qubit_quantum_error(error_2q, ['cx'])
    if p_ro:
        nm.add_all_qubit_readout_error(
            ReadoutError([[1-p_ro, p_ro], [p_ro, 1-p_ro]]))
    return nm

```

Listing B.6: Noise model uniforme illustrativo: RZ virtuale, durate 1Q/2Q separate

```

def compile_shor_circuit(N, a, n_count):
    return transpile(
        shor_circuit(N, a, n_count),
        basis_gates=['rz', 'sx', 'x', 'cx'],
        optimization_level=2,
        seed_transpiler=20260819,
    )

```

```

def rank_measurements(counts, tie_seed):
    def priority(bitstring):
        payload = f'{int(tie_seed)}:{bitstring}'.encode('ascii')
        return hashlib.sha256(payload).digest()
    return sorted(counts.items(),
                  key=lambda item: (-item[1], priority(item[0])))

def method1_from_counts(counts, tie_seed, n_count, N, a):
    bitstring = rank_measurements(counts, tie_seed)[0][0]
    return extract_factors(int(bitstring, 2), n_count, N, a)

```

Listing B.7: Compilazione deterministica, tie-break SHA-256 e baseline TOP-1

```

LABEL_TOP_KS = (1, 16)
labels = {top_k: [] for top_k in LABEL_TOP_KS}

for sample_index in range(n_samples):
    sample_seed = seed * 1_000_000 + sample_index * 10_000
    counts = simulator.run(
        compiled, noise_model=sample_noise, shots=shots,
        seed_simulator=sample_seed,
    ).result().get_counts()
    ranked = rank_measurements(counts, tie_seed=sample_seed)
    for top_k in LABEL_TOP_KS:
        useful = any(
            extract_factors(int(bits, 2), n_count, N, a)[0] is
            not None
            for bits, _ in ranked[:top_k]
        )
        labels[top_k].append(int(useful))
    X.append(normalized_histogram(counts, n_count, shots))

# Ogni modello salva schema_version=2.0, label_top_k e
  manifest.

```

Listing B.8: Dataset condiviso per l'audit delle etichette TOP-1 e TOP-16

```

def evaluate_same_histogram(counts, simulation_seed,
                             classifier,
                             n_count, N, a, shots):
    ranked = rank_measurements(counts, simulation_seed)
    valid = [
        extract_factors(int(bits, 2), n_count, N, a)[0] is not
        None
        for bits, _ in ranked[:4]
    ]
    m1_success = valid[0]
    top4_success = any(valid)
    feature = normalized_histogram(counts, n_count, shots)
    m2_success = bool(classifier.predict([feature])[0]) and
    top4_success
    return m1_success, top4_success, m2_success

```

Listing B.9: Decisione M2 sullo stesso istogramma usato da M1 e TOP-4

```

PRESETS = {
    'reference': dict(N=15, a=7, n_count=8,
                      eps_1q=1e-3, eps_2q=1e-2,
                      t1_ns=100_000, t2_ns=80_000, p_ro=0.02),
    'stress': dict(N=15, a=7, n_count=8,
                   eps_1q=5e-3, eps_2q=5e-2,
                   t1_ns=50_000, t2_ns=30_000, p_ro=0.05),
}

K_REPETITIONS, SHOTS, MAX_ITER = 30, 1024, 50

for rep in range(K_REPETITIONS):
    first = {'m1': None, 'top4': None, 'm2': None}
    for iteration in range(1, MAX_ITER + 1):
        simulation_seed = rep * 1_000_000 + iteration * 10_000
        counts = simulator.run(compiled, shots=SHOTS,
                               seed_simulator=simulation_seed).result().
            get_counts()
        flags = evaluate_same_histogram(counts,
                                         simulation_seed, ...)
        update_first_success(first, flags, iteration)

```

```

        encoded = {name: value if value is not None else MAX_ITER
                    + 1
                    for name, value in first.items()}

# Inferenza: wilcoxon(paired_a, paired_b,
#                     zero_method='pratt', alternative=...)
# Output: schema_version='2.0', input espliciti e manifest/
#        hash.

```

Listing B.10: Orchestrazione appaiata delle sole campagne rumorose N15

```

if logical_value == 1:
    qc.x(0)
qc.cx(0, 1); qc.cx(0, 2)
if basis == 'X':
    qc.h(DATA)
for q in DATA:
    qc.id(q)                                # punto di iniezione del
    rumore
if basis == 'Z':
    qc.cx(0, 3); qc.cx(1, 3)
    qc.cx(1, 4); qc.cx(2, 4)
else:
    qc.h(3); qc.cx(3, 0); qc.cx(3, 1); qc.h(3)
    qc.h(4); qc.cx(4, 1); qc.cx(4, 2); qc.h(4)
qc.measure(ANC[0], 0); qc.measure(ANC[1], 1)

```

Listing B.11: Codifica ed estrazione di sindrome del codice a ripetizione.

```

def encode_zero_L(qc):
    for seed in SEED:
        qc.h(seed)
    for seed, support in SEED.items():
        for data in support:
            if data != seed:
                qc.cx(seed, data)

def measure_syndrome(qc):
    for k, support in enumerate(GZ):
        for data in support:

```



```

        qc.cx(data, ANC_Z[k])
    for k, support in enumerate(GX):
        qc.h(ANC_X[k])
        for data in support:
            qc.cx(ANC_X[k], data)
        qc.h(ANC_X[k])

```

Listing B.12: Preparazione di $|0_L\rangle$ e misura degli stabilizzatori di Steane.

```

def inject(block, pairs, p_ct):
    out = stim.Circuit()
    first = True
    for inst in block:
        if isinstance(inst, stim.CircuitRepeatBlock):
            out += inject(inst.body_copy(), pairs, p_ct) *
                inst.repeat_count
            continue
        out.append(inst)
        if inst.name == 'DEPOLARIZE1':
            if first:
                out.append('DEPOLARIZE2', pairs, p_ct)
                first = not first
    return out

```

Listing B.13: Iniezione ricorsiva del crosstalk nel circuito del surface code.

```

circuit = stim.Circuit.generated(
    'surface_code:rotated_memory_x', distance=d, rounds=rounds
    ,
    after_clifford_depolarization=p,
    after_reset_flip_probabilty=p,
    before_measure_flip_probabilty=p,
)
sampler = circuit.compile_detector_sampler()
events, observables = sampler.sample(shots,
    separate_observables=True)
model = circuit.detector_error_model(decompose_errors=True)
matching = pymatching.Matching.from_detector_error_model(model
)
predictions = matching.decode_batch(events)

```

```
p_L = (predictions != observables).any(axis=1).mean()
```

Listing B.14: Stima di p_L con Stim e PyMatching.

Appendice C

Contesto e Motivazione

C.1 Contesto Esteso dell'Introduzione

Questa appendice raccoglie in forma estesa il contesto richiamato sinteticamente nel Capitolo 1: i principi fisici del calcolo quantistico e la loro implementazione hardware, la rivoluzione algoritmica degli anni Novanta, il protocollo RSA e la sua vulnerabilità all'algoritmo di Shor, la crittografia post-quantistica e lo stato dell'arte dell'hardware quantistico.

C.1.1 Dal Bit al Qubit: i Principi Fisici del Calcolo Quantistico

Per comprendere appieno le potenzialità e i limiti del calcolo quantistico, è necessario esaminare le differenze fisiche fondamentali tra l'unità di informazione classica e quella quantistica.

Il Qubit e la Sovrapposizione

Un computer classico elabora l'informazione attraverso **bit** – unità fisiche che possono trovarsi in uno di due stati discreti, convenzionalmente 0 e 1, realizzati come livelli di tensione in circuiti elettronici. L'unità fondamentale del calcolo quantistico è invece il **qubit** (*quantum bit*): un sistema fisico a due livelli che obbedisce alle leggi della meccanica quantistica.

Matematicamente, lo stato di un qubit è descritto da un vettore nello spazio di Hilbert $\mathcal{H} \cong \mathbb{C}^2$:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (\text{C.1})$$

dove $|0\rangle$ e $|1\rangle$ sono i due stati della base computazionale, analoghi ai valori 0 e 1 del bit classico. La differenza fondamentale è che α e β possono essere simultaneamente diversi da zero: il qubit si trova in una **sovrapposizione** dei due stati base, con probabilità $|\alpha|^2$ di essere misurato in $|0\rangle$ e probabilità $|\beta|^2$ di essere misurato in $|1\rangle$.

È cruciale sottolineare che la sovrapposizione non equivale a ignoranza sullo stato del sistema: il qubit non è “segretamente” in uno stato definito che non conosciamo. Si tratta di una proprietà fisica genuina, verificata sperimentalmente con precisione straordinaria, che non ha analogo nella fisica classica.

La Sfera di Bloch

Poiché la norma è unitaria e la fase globale è fisicamente irrilevante, lo stato di un qubit puro può essere parametrizzato da soli due angoli reali:

$$|\psi\rangle = \cos\frac{\theta}{2} |0\rangle + e^{i\phi} \sin\frac{\theta}{2} |1\rangle, \quad \theta \in [0, \pi], \quad \phi \in [0, 2\pi) \quad (\text{C.2})$$

Questo definisce un punto sulla superficie di una sfera unitaria – la **sfera di Bloch** – dove il polo nord corrisponde a $|0\rangle$, il polo sud a $|1\rangle$, e i punti equatoriali a sovrapposizioni equiprobabili con fasi diverse. Questa rappresentazione geometrica è di grande utilità pratica: le operazioni unitarie su un singolo qubit corrispondono a rotazioni, mentre i canali rumorosi trasformano in generale la sfera in un ellissoide traslato. Un canale unital può contrarre verso il centro; l’amplitude damping trasla invece gli stati verso $|0\rangle$.

L’Entanglement

Quando due o più qubit interagiscono tramite porte quantistiche opportune, possono formare stati **entangled**: stati del sistema composto che non possono essere scritti come prodotto tensoriale di stati individuali. Il prototipo è lo stato di Bell:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (\text{C.3})$$

In questo stato, se la misura del primo qubit produce 0, una misura del secondo nella stessa base produce anch'essa 0, e analogamente per 1, indipendentemente dalla distanza fisica; la correlazione non consente però di trasmettere informazione più velocemente della luce. Con opportune scelte delle basi di misura, gli stati entangled violano disuguaglianze di Bell e producono correlazioni incompatibili con modelli a variabili nascoste locali [43, 44]. Un registro di n qubit descrive ampiezze in uno spazio $\mathbb{C}^{2^n}$, ma questa dimensione non basta da sola a garantire un vantaggio: alcune famiglie entangled, come gli stati stabilizer, restano simulabili classicamente in modo efficiente. Il vantaggio algoritmico nasce dalla struttura con cui preparazione, entanglement e interferenza trasformano tali ampiezze.

L'Interferenza Quantistica

Il terzo pilastro del calcolo quantistico è l'**interferenza**: la capacità di far sommare costruttivamente o annullare distruttivamente le ampiezze di probabilità di percorsi computazionali diversi. Le ampiezze α e β in (C.1) sono numeri complessi, e possono quindi avere segni opposti o sfasamenti arbitrari: quando si sommano, possono amplificarsi o cancellarsi esattamente come onde fisiche.

Un algoritmo quantistico ben progettato sfrutta questo meccanismo per modificare la distribuzione degli esiti: Grover amplifica l'ampiezza degli elementi marcati, mentre in Shor la QFT concentra la probabilità vicino alle frequenze legate al periodo. Non tutti gli esiti prodotti sono quindi “corretti”, e resta necessario un post-processing classico probabilistico.

Implementazione Fisica: i Qubit Superconduttori di IBM

Un qubit può essere realizzato con qualunque sistema fisico a due livelli: la polarizzazione di un fotone, lo spin di un elettrone, i livelli energetici di un atomo intrappolato. I **qubit superconduttori** utilizzati da IBM sono circuiti elettrici macroscopici che operano nel regime quantistico grazie al raffreddamento a temperature criogeniche estreme. In questa tesi costituiscono l'architettura di riferimento per la snapshot FakeSherbrooke e per la discussione hardware; gli esperimenti principali restano simulazioni classiche.

Il componente chiave è la **giunzione di Josephson** [45]: una sottile barriera isolante (spessa $\sim 1\text{--}2\text{ nm}$) interposta tra due materiali superconduttori. Questo elemento introduce una non-linearità nell'energia del circuito che rompe l'equidistanza dei livelli energetici, permettendo di indirizzare selettivamente i due stati

computazionali $|0\rangle$ e $|1\rangle$. Il risultato è il **transmon** [46] – il tipo di qubit superconduttore adottato da IBM – un oscillatore anarmonico con frequenza tipica 4–6 GHz, controllato da impulsi a microonde.

Per operare nel regime quantistico, questi circuiti devono essere raffreddati a temperature dell'ordine di 15 mK (circa -273.135°C), molto inferiori ai 2.7 K della radiazione cosmica di fondo, tramite refrigeratori a diluizione. Il raffreddamento abilita il regime superconduttivo e sopprime fortemente le eccitazioni termiche, ma non elimina la dissipazione né le altre sorgenti di decoerenza.

I parametri operativi tipici dei processori IBM (serie Eagle, Heron) sono:

- **Temperatura operativa:** ~ 15 mK
- **Tempi di coerenza:** $T_1, T_2 \sim 50\text{--}500\ \mu\text{s}$
- **Durata gate a singolo qubit:** ~ 50 ns
- **Durata gate a due qubit (ECR/CNOT):** $\sim 300\text{--}500$ ns
- **Fedeltà gate a singolo qubit:** $\sim 99.9\text{--}99.97\%$
- **Fedeltà gate a due qubit:** $\sim 99\text{--}99.7\%$

Il rapporto tra il tempo di coerenza e la durata di un gate contribuisce a determinare quante operazioni utili possano essere eseguite prima che il rumore domini; insieme agli errori di controllo, al crosstalk, al readout e alla connettività, è uno dei principali colli di bottiglia per algoritmi profondi come Shor sui processori attuali.

C.1.2 La Rivoluzione Algoritmica degli Anni Novanta

Negli anni successivi alla proposta di Deutsch, la comunità scientifica si interrogò sulla effettiva esistenza di problemi che un computer quantistico potesse risolvere in modo *probabilmente* più efficiente di qualunque macchina classica. La risposta arrivò in forma dirompente nel corso di pochi anni.

Nel 1994, Daniel Simon¹ identificò un problema oracle per il quale un algoritmo quantistico richiede esponenzialmente meno interrogazioni di qualunque algoritmo classico probabilistico. Nello stesso anno Peter Shor pubblicò algoritmi quantistici polinomiali per fattorizzazione e logaritmo discreto; nella formulazione textbook della fattorizzazione, il costo è spesso riassunto come $O(n^3)$ [1]. Le implicazioni furono immediate: RSA dipende dalla difficoltà della fattorizzazione, mentre Diffie–Hellman ed *Elliptic Curve Digital Signature Algorithm* (ECDSA) dipendono da varianti del logaritmo discreto. Una macchina quantistica sufficientemente grande comprometterebbe quindi molte delle primitive a chiave pubblica oggi impiegate, pur senza rendere insicura ogni componente simmetrica dei protocolli.

¹L'**algoritmo di Simon**, presentato nel 1994 e pubblicato in forma estesa nel 1997, dimostrò nel modello oracle una separazione esponenziale rispetto agli algoritmi classici probabilistici. Shor riconobbe che la sua tecnica si ispirava direttamente alle idee di Simon.

Due anni dopo, nel **1996**, Lov Grover presentò un algoritmo di tutt'altra natura, ma di eguale importanza [34]. Mentre Shor aveva risolto un problema specifico della teoria dei numeri, Grover affrontò una sfida universale: la **ricerca non strutturata**. Dato un insieme di N elementi senza alcuna struttura o ordinamento, trovare quello che soddisfa una certa proprietà richiede, nel caso classico, un numero di interrogazioni proporzionale a N . Grover dimostrò che un algoritmo quantistico può fare lo stesso lavoro in sole $O(\sqrt{N})$ interrogazioni – uno *speedup quadratico* – e che questo risultato è *ottimale*: nessun algoritmo quantistico può fare di meglio [36]. Sebbene meno spettacolare dello speedup esponenziale di Shor, il vantaggio di Grover è universale, applicabile a qualunque problema di ricerca, e porta con sé implicazioni profonde per la crittografia simmetrica, l'ottimizzazione combinatoria e la ricerca in grandi basi di dati.

Questi due algoritmi – Shor e Grover – rappresentano ancora oggi i pilastri dell'informatica algoritmica quantistica: il primo come dimostrazione del potere *esponenziale* del calcolo quantistico su problemi strutturati, il secondo come strumento *universale* di accelerazione quadratica applicabile al problema computazionale più fondamentale.

C.1.3 RSA e le Implicazioni Crittografiche dell'Algoritmo di Shor

Per comprendere appieno la portata dell'algoritmo di Shor è necessario esaminare la struttura matematica del sistema crittografico che esso minaccia. Il sistema **RSA**, proposto nel 1977 da Rivest, Shamir e Adleman e pubblicato nel 1978 [3], è uno degli schemi storicamente centrali della crittografia a chiave pubblica. Continua a essere impiegato in certificati e firme, ma i protocolli moderni possono adottare anche primitive diverse, in particolare su curve ellittiche.

Struttura del Protocollo RSA

La sicurezza di RSA si basa sulla **asimmetria computazionale** tra due operazioni inverse: moltiplicare due numeri primi grandi p e q per ottenere $N = p \cdot q$ è un'operazione elementare (complessità $O(n^2)$ per numeri a n bit), mentre fattorizzare N per recuperare p e q è computazionalmente intrattabile con gli algoritmi classici noti. La generazione delle chiavi segue i passi seguenti:

1. Scegliere due primi grandi p e q (tipicamente a 1024–2048 bit ciascuno) e calcolare $N = p \cdot q$;
2. Calcolare la funzione di Eulero $\phi(N) = (p-1)(q-1)$;
3. Scegliere e coprimo con $\phi(N)$ (convenzionalmente $e = 65537 = 2^{16} + 1$, primo di Fermat);
4. Calcolare $d \equiv e^{-1} \pmod{\phi(N)}$ tramite l'algoritmo di Euclide esteso.

La **chiave pubblica** è la coppia (N, e) ; la **chiave privata** è d . Cifrare un messaggio $m < N$ produce il crittogramma $c = m^e \bmod N$; decifrarlo richiede $m = c^d \bmod N$. La correttezza si fonda sul **teorema di Eulero** — la generalizzazione del piccolo teorema di Fermat a moduli composti, secondo cui $m^{\phi(N)} \equiv 1 \pmod{N}$ per m coprimo con N : essendo $ed \equiv 1 \pmod{\phi(N)}$, si ha $ed = 1 + k\phi(N)$ e quindi $(m^e)^d = m^{1+k\phi(N)} \equiv m \pmod{N}$. Tutta la sicurezza risiede nell'impossibilità di calcolare d da (N, e) senza conoscere la fattorizzazione di N .

Perché l'Algoritmo di Shor Rompe RSA

Il miglior algoritmo classico per la fattorizzazione di un intero a n bit è il *General Number Field Sieve* (GNFS) [2], con complessità sub-esponenziale:

$$T_{\text{GNFS}}(N) = O(\exp[c \cdot (\log N)^{1/3} (\log \log N)^{2/3}]) \quad (\text{C.4})$$

Per N a 2048 bit, questa espressione implica circa 2^{112} operazioni classiche — un numero astronomico, ben oltre la capacità di qualunque infrastruttura computazionale esistente o prevedibile in ambito classico. I migliori risultati pratici al 2020 hanno raggiunto la fattorizzazione di RSA-250 (829 bit) [47], impiegando circa 2700 anni-CPU di calcolo distribuito.

L'algoritmo di Shor riduce questa complessità a $O(n^3)$ — un salto da sub-esponenziale a **polinomiale** — rendendo la fattorizzazione di chiavi RSA-2048 un problema risolvibile in ore su un computer quantistico fault-tolerant con qualche migliaio di qubit logici. La struttura dell'algoritmo, descritta in dettaglio nel Capitolo 3, sfrutta la Quantum Fourier Transform per trovare il periodo di una funzione modulare, problema che si riduce efficientemente alla fattorizzazione.

Implicazioni Pratiche per la Sicurezza Informatica

Le conseguenze sono sistemiche e riguardano più in generale la crittografia a chiave pubblica vulnerabile a Shor: RSA, Diffie–Hellman e le primitive su curve ellittiche. Questi algoritmi compaiono, in combinazioni diverse, in:

- **HTTPS e TLS:** per autenticazione e scambio delle chiavi di sessione, a seconda della versione e della configurazione;
- **Certificati X.509:** nell’infrastruttura a chiave pubblica usata per autenticare servizi e soggetti;
- **SSH:** il protocollo standard per l’accesso remoto sicuro ai server;
- **Firme digitali:** documenti legali, transazioni finanziarie, aggiornamenti software firmati.

Un computer quantistico crittograficamente rilevante renderebbe vulnerabili anche comunicazioni registrate in precedenza quando la riservatezza della chiave di sessione dipende da una primitiva a chiave pubblica attaccabile da Shor. È il rischio “*harvest now, decrypt later*”: acquisire oggi dati cifrati ancora sensibili e conservarli per una futura decifratura. Il *National Institute of Standards and Technology* (NIST) sottolinea però che non esiste una data affidabile per la comparsa di una macchina di questo tipo; le stime spaziano da pochi anni a decenni. Le scadenze 2030–2035 presenti nelle roadmap istituzionali sono obiettivi di migrazione e dismissione degli algoritmi vulnerabili, non previsioni della data in cui RSA-2048 verrà violato [48, 49].

C.1.4 Crittografia Post-Quantistica: la Risposta dell’Industria

La consapevolezza della minaccia quantistica ha spinto il NIST a lanciare nel 2016 un processo di standardizzazione internazionale per la **crittografia post-quantistica** (*Post-Quantum Cryptography* (PQC)) – algoritmi crittografici resistenti agli attacchi di computer quantistici, progettati per essere eseguiti su hardware classico convenzionale e quindi immediatamente distribuibili.

Gli Standard NIST 2024

Dopo otto anni di valutazione pubblica, crittoanalisi internazionale e tre successivi turni di selezione tra 69 candidati iniziali, nell’agosto 2024 il NIST ha pubblicato i primi tre standard definitivi [50]:

- **ML-KEM** (FIPS 203, ex CRYSTALS-Kyber): meccanismo di incapsulamento di chiavi basato sul problema *Module Learning With Errors* (MLWE) su reticoli. È il sostituto designato per lo scambio di chiavi RSA e Diffie-Hellman nei protocolli TLS;
- **ML-DSA** (FIPS 204, ex CRYSTALS-Dilithium): schema di firma digitale basato sullo stesso fondamento matematico di ML-KEM. Sostituisce RSA e ECDSA per le firme digitali;
- **SLH-DSA** (FIPS 205, ex SPHINCS+): schema di firma basato esclusivamente su funzioni hash, senza assunzioni algebriche. Più conservativo per sicurezza, adottato come backup contro potenziali attacchi futuri ai reticoli.

La sicurezza di questi schemi si fonda su problemi matematici – trovare vettori corti in reticoli ad alta dimensione, o invertire funzioni hash – per i quali non sono noti algoritmi quantistici con speedup esponenziale. In particolare, l’algoritmo di Grover fornisce al più uno speedup quadratico sui problemi basati su hash, motivando l’aumento dei parametri di sicurezza (es. passaggio da 128 a 256 bit di sicurezza classica equivalente).

La Sfida della Migrazione

La transizione verso la crittografia post-quantistica è un’operazione di ingegneria su scala globale, paragonabile per complessità alla migrazione da IPv4 a IPv6, ma con urgenza maggiore. Le principali sfide includono:

- **Eterogeneità dei sistemi:** miliardi di dispositivi embedded, sensori industriali e infrastrutture legacy con cicli di vita decennali che non riceveranno aggiornamenti software;
- **Overhead di banda:** il materiale crittografico post-quantistico è sensibilmente più voluminoso. Al livello di sicurezza più basso (ML-KEM-512), la chiave di incapsulamento occupa 800 byte e il crittogramma 768 byte, contro i circa 256 byte di una chiave pubblica RSA-2048: un fattore di crescita che pesa sui protocolli a bassa latenza e sui dispositivi con banda limitata;
- **Migrazione a doppio binario:** nella fase transitoria, molte implementazioni adottano schemi *ibridi* (classico + post-quantistico in parallelo) per mantenere compatibilità con sistemi non aggiornati.

Aziende come Google (Chrome 131), Apple (iMessage PQ3) [51] e Cloudflare hanno già avviato implementazioni ibride nei loro protocolli di produzione. La coesistenza di questa transizione con la ricerca attiva su algoritmi quantistici come Shor definisce il contesto strategico in cui si inserisce il presente lavoro: comprendere i limiti reali di Shor su hardware NISQ è rilevante non solo teoricamente, ma anche per calibrare con maggiore precisione le tempistiche e le priorità della transizione crittografica globale.

C.1.5 Lo Stato dell'Arte: dall'Era NISQ ai Computer Fault-Tolerant

Nonostante la solidità teorica di questi risultati, la realizzazione pratica di computer quantistici di scala sufficiente ad eseguirli su istanze realmente significative incontra ostacoli formidabili. I sistemi quantistici fisici sono intrinsecamente fragili: qualunque interazione non controllata con l'ambiente esterno – vibrazioni termiche, campi elettromagnetici, imperfezioni nei materiali – provoca fenomeni di **decoerenza** – ossia la perdita progressiva della coerenza quantistica causata dall'entanglement indesiderato tra il qubit e i gradi di libertà ambientali, che sopprime le interferenze quantistiche riducendo il sistema a una miscela statistica classica – degradando lo stato quantistico in modo irreversibile prima che il calcolo possa completarsi. A differenza di un bit classico, che può essere protetto dalla ridondanza, un qubit non può essere copiato (teorema del no-cloning [20]) né letto senza distruggerlo: questo rende la correzione degli errori quantistici un problema radicalmente diverso e molto più complesso di quella classica.

Il panorama tecnologico attuale è dominato da un insieme variegato di piattaforme hardware in competizione, ciascuna con vantaggi e svantaggi distinti:

- **Qubit superconduttori** (IBM, Google, Rigetti): alta velocità dei gate ($\sim 50\text{--}500\text{ ns}$), scalabilità su chip in silicio, ma tempi di coerenza limitati a $\sim 100\text{--}500\text{ }\mu\text{s}$ e necessità di raffreddamento a 15 mK ;
- **Ioni intrappolati** (IonQ, Quantinuum): tempi di coerenza eccellenti ($> 1\text{ s}$), fedeltà dei gate superiore al 99.9% anche su due qubit, ma gate lenti ($\sim 10\text{--}100\text{ }\mu\text{s}$) e scalabilità più complessa;
- **Fotoni** (PsiQuantum, Xanadu): operano a temperatura ambiente, naturalmente adatti alla comunicazione quantistica, ma la generazione deterministica di fotoni entangled rimane una sfida aperta;
- **Spin in silicio** (Intel, UNSW): compatibili con i processi di fabbricazione CMOS esistenti, potenzialmente scalabili a milioni di qubit, ma con tecnologia di controllo ancora in fase di sviluppo.

Un momento simbolico di questa corsa fu raggiunto nel **2019**, quando il team di Google rivendicò il raggiungimento della cosiddetta *quantum supremacy*² con il processore Sycamore a 53 qubit [52]: un calcolo specifico fu completato in 200 secondi, mentre le stime attribuivano al più potente supercomputer classico un tempo di circa 10.000 anni per lo stesso compito. Nel 2023, IBM ha presentato il processore *Condor* a 1.121 qubit e ha delineato una roadmap [33] verso sistemi con architettura fault-tolerant entro la fine del decennio, con il target *Quantum Starling* previsto per il 2029.

Tuttavia, la distanza tra i dispositivi attuali e un computer quantistico *fault-tolerant* — in grado di eseguire Shor su istanze RSA reali — è ancora enorme. I processori odierni sono comunemente descritti nel regime NISQ, termine introdotto da John Preskill nel 2018 [4]: il numero di qubit fisici, da solo, non li rende fault-tolerant, perché errori, connettività e assenza di correzione scalabile limitano la profondità affidabile. In questo scenario, la ricerca si concentra su due direzioni parallele e complementari:

- **QEC:** tecniche per codificare qubit logici in qubit fisici ridondanti, in modo da rilevare e correggere gli errori durante il calcolo, a costo di un overhead fisico significativo (centinaia o migliaia di qubit fisici per qubit logico);
- **Noise mitigation:** tecniche che, senza richiedere overhead hardware, riducono l'effetto del rumore a livello di post-processing classico, estendendo la portata dei circuiti eseguibili su hardware NISQ.

C.2 Dettaglio degli Obiettivi e del Piano Sperimentale

Questa appendice raccoglie il materiale di dettaglio del Capitolo 2: la motivazione crittografica estesa, i requisiti funzionali del sistema sperimentale e la specifica completa dei parametri di input e di output.

²Il termine *quantum supremacy*, coniato da John Preskill nel 2012, indica il punto in cui un computer quantistico esegue un calcolo specifico in un tempo che nessun supercomputer classico potrebbe eguagliare in modo pratico. La rivendicazione di Google con il processore Sycamore è rimasta controversa: IBM sostenne che il calcolo avrebbe potuto essere eseguito classicamente in 2.5 giorni anziché 10.000 anni, con un algoritmo migliorato. Questo dibattito evidenzia la difficoltà di definire e misurare il vantaggio quantistico.

C.2.1 Motivazione: la Vulnerabilità della Crittografia Attuale

La quasi totalità delle comunicazioni digitali sicure — dalle transazioni bancarie alle connessioni *HyperText Transfer Protocol Secure* (HTTPS), dai protocolli *Secure Shell* (SSH) ai certificati digitali — è protetta da sistemi crittografici la cui sicurezza si fonda su un presupposto computazionale: la difficoltà di fattorizzare il prodotto di due grandi numeri primi. Il sistema RSA, ad esempio, costruisce la propria chiave pubblica come $N = p \cdot q$, dove p e q sono primi di centinaia o migliaia di bit. Finché non esiste un algoritmo classico efficiente per recuperare p e q a partire da N , la chiave privata rimane al sicuro.

Per comprendere perché i sistemi crittografici attuali siano strutturalmente vulnerabili all'algoritmo di Shor, è necessario mettere a confronto le complessità computazionali in gioco. Il miglior algoritmo classico per la fattorizzazione, il GNFS, richiede un tempo:

$$T_{\text{classico}}(N) = O\left(\exp\left((c + o(1))(\log N)^{1/3}(\log \log N)^{2/3}\right)\right) \quad (\text{C.5})$$

che, pur crescendo più lentamente di un esponenziale puro, rimane del tutto impraticabile per i valori di N usati in crittografia. A titolo di esempio, fattorizzare un numero RSA a 2048 bit con algoritmi classici richiederebbe un tempo stimato superiore all'età dell'universo, anche impiegando l'intera potenza di calcolo disponibile sul pianeta.

Nel 1994, Peter Shor dimostrò che un computer quantistico è in grado di eseguire la stessa operazione in tempo **polinomiale** [1]:

$$T_{\text{Shor}}(N) = O((\log N)^3) \quad (\text{C.6})$$

Lo scarto tra le due espressioni non è una differenza di grado, ma una differenza di *natura*: il tempo classico cresce in modo super-polinomiale con la lunghezza della chiave, mentre il tempo quantistico cresce polinomialmente. **Aumentare la lunghezza della chiave RSA non è quindi una contromisura strutturale** contro Shor: raddoppiare n porta il costo asintotico textbook da $O(n^3)$ a $8O(n^3)$. Sul piano concreto questo fattore può richiedere molte più risorse fault-tolerant e ritardare un attacco, ma non ripristina un problema computazionalmente difficile per un avversario quantistico scalabile. La stessa vulnerabilità asintotica si estende ai sistemi basati sul logaritmo discreto (Diffie–Hellman, ECDSA).

La minaccia ha già una conseguenza operativa: nel modello *harvest now, decrypt*

later, dati cifrati oggi e destinati a restare sensibili vengono conservati in vista di una futura macchina crittograficamente rilevante. Non esiste però una previsione affidabile della sua data di arrivo. Le roadmap dei produttori descrivono obiettivi ingegneristici e il NIST sottolinea che le stime spaziano da pochi anni a decenni; proprio l'incertezza, unita ai tempi lunghi di migrazione, motiva l'adozione anticipata della PQC [48, 33].

Gli ostacoli pratici sono la scala, la profondità fault-tolerant e il **rumore**. I processori attuali operano in regime NISQ: dispongono di molti qubit fisici, ma decoerenza, errori di gate, readout, connettività e overhead di correzione impediscono di sostenere il numero di operazioni logiche richiesto da Shor su chiavi crittografiche. Comprendere questi limiti e stabilire quali strategie li riducano — e in quale punto della pipeline i modelli di machine learning contribuiscano davvero — è la ragione d'essere del presente lavoro.

C.2.2 Requisiti Funzionali

Definiti gli obiettivi e il contributo originale, è possibile formalizzare cosa il sistema sperimentale deve concretamente fare. I requisiti seguenti traducono le ipotesi di ricerca in vincoli tecnici verificabili, e costituiscono il riferimento rispetto al quale l'implementazione descritta nel Capitolo 6 verrà valutata.

Il sistema sperimentale deve soddisfare i seguenti requisiti funzionali:

1. **Esecuzione configurabile entro il perimetro validato.** La campagna rumorosa e la demo pubblica usano l'istanza canonica $N = 15$, $a = 7$ e un numero configurabile di shot entro limiti dichiarati. Le implementazioni Beauregard per $N = 21$ e $N = 35$ sono sottoposte a test ideali e audit di risorse, ma non sono esposte come campagne rumorose equivalenti.
2. **Modello di rumore parametrizzabile.** Il sistema permette di configurare separatamente i parametri Aer λ_{1q} e λ_{2q} , i tempi T_1/T_2 , il readout asimmetrico della demo e l'eventuale sovrarotazione coerente. I preset della campagna restano uniformi e illustrativi; non riproducono una calibrazione hardware nominata.
3. **Raccolta e archiviazione degli output.** Il sistema registra la distribuzione di misura del registro di controllo per ogni esecuzione e la archivia in formato strutturato. La riproducibilità usa un seed base e calendari derivati separati per replica, iterazione, simulatore e risoluzione dei pareggi.

4. **Analisi statistica del Metodo 1.** Il sistema implementa la pipeline del Metodo 1: selezione della misura più frequente dell'istogramma (strategia TOP-1, senza sogliatura preliminare), stima del periodo r tramite frazioni continue, verifica della correttezza dei fattori estratti. Misura e registra il numero di iterazioni necessarie per raggiungere il primo risultato corretto.
5. **Training, validazione e inferenza del Metodo 2.** Il sistema genera il dataset dal simulatore rumoroso, addestra il classificatore con divisione training/validation/test e ne valuta le metriche. Il modello corrente apprende l'etichetta TOP-1: decide se il candidato dominante conduce ai fattori; se accetta l'istogramma, M2 applica poi la ricerca TOP-4. L'audit TOP-16 è a classe singola e non produce un classificatore.
6. **Confronto sistematico sui use case previsti.** M1, TOP-4 e M2 vengono eseguiti sugli stessi istogrammi nei due preset $N = 15$, producendo metriche appaiate secondo la Sezione 2.5. Per $N = 21$ e $N = 35$ si riportano correttezza ideale e costo compilato; non si afferma che la probabilità rumorosa sia zero, perché la campagna rumorosa non viene eseguita.
7. **Riproducibilità.** Tutti gli esperimenti sono riproducibili: seed fisso, parametri esplicitati, codice disponibile nel repository del progetto. Questo requisito è condizione necessaria per la validazione accademica dei risultati [37].

C.2.3 Parametri di Input e Output

Parametri di Input

La Tabella C.1 elenca i parametri di configurazione del sistema. I loro valori vengono fissati per ciascun use case prima dell'esecuzione e rimangono costanti per tutta la durata dell'esperimento corrispondente.

Parametro	Unità	Descrizione
N	intero	Numero da fattorizzare
a	intero	Base della moltiplicazione modulare ($\gcd(a, N) = 1$)
n_{count}	intero	Qubit del registro di controllo
M_{shots}	intero	Shot per esecuzione del simulatore
λ_{1q}	adimensionale	Parametro Aer del canale depolarizzante 1Q
λ_{2q}	adimensionale	Parametro Aer del canale depolarizzante 2Q
T_1	ns	Tempo di rilassamento (amplitude damping)
T_2	ns	Tempo di dephasing
p_{ro}	adimensionale	Probabilità di readout error

Tabella C.1: Parametri principali del sistema sperimentale. λ segue la convenzione Aer e non coincide con la probabilità aggregata di Pauli non identità.

Output del Sistema

Per ogni use case e per ciascuno dei due metodi, il sistema produce le seguenti quantità:

- I fattori trovati p e q tali che $p \cdot q = N$, con verifica esplicita della correttezza ($p \neq 1, q \neq 1, p \cdot q = N$);
- Il numero di iterazioni necessarie per il primo risultato corretto;
- La distribuzione di frequenza completa degli output del registro di controllo su M_{shots} esecuzioni;
- Le metriche di confronto tra i due metodi definite nella Sezione 2.5.

Per il solo Metodo 2, vengono prodotte in aggiunta:

- L'accuratezza del classificatore e il punteggio F1 sul test set;
- La curva ROC con l'area sottostante (Area Sotto la Curva ROC (*Area Under the Curve*) (AUC)).

Appendice D

Fase 1: la Campagna Classica nel Dettaglio

D.1 Campagna Parametrica e Confronto con la Zero-Noise Extrapolation

Questa appendice riporta la campagna parametrica schema 2 sulla strategia TOP-K, di cui il Capitolo [efchap:risultati](#) riporta la sintesi. Il protocollo usa $N = 15$, $a = 7$, $n_{\text{count}} = 8$, 30 repliche, budget massimo 50 e, salvo diversa indicazione, 1 024 shot. Il preset di riferimento è $\lambda_{1q} = 10^{-3}$, $\lambda_{2q} = 10^{-2}$, $T_1/T_2 = 100/80 \mu\text{s}$ e readout simmetrico 2%.

M1 e TOP-K condividono lo stesso istogramma per ogni coppia replica-iterazione. I fallimenti sono codificati con sentinella 51 e i pareggi con una priorità SHA-256 dipendente dal seed. I confronti usano Wilcoxon appaiato-Pratt. Poiché non è applicata una correzione globale fra le otto famiglie, i p di questa appendice hanno finalità esplorativa.

D.1.1 Variazione di K

Tabella D.1: M1 e TOP-K sui medesimi 30 vettori di iterazioni. $K = 1$ coincide con il controllo M1.

K	$\bar{M}_1$	$\bar{M}_{\text{TOP}K}$	successo	ρ	p
1	1.37	1.37	100%	1.000	1.000
2	1.37	1.00	100%	1.367	< 0.001
3	1.37	1.00	100%	1.367	< 0.001
4	1.37	1.00	100%	1.367	< 0.001
6	1.37	1.00	100%	1.367	< 0.001
8	1.37	1.00	100%	1.367	< 0.001

La soglia osservata è $K = 2$, non $K = r = 4$. Questo risultato dipende dalla banda di esiti che la post-elaborazione mappa su un periodo utile e non costituisce una regola generale per altre istanze.

D.1.2 Variazione di λ_{2q}

Per il canale Aer a due qubit, $\mathcal{E}(\rho) = (1 - \lambda)\rho + \lambda I/4$, la probabilità aggregata della componente Pauli non-identità è $15\lambda/16$. Sul circuito da 224 CX il proxy di nessun evento è quindi $(1 - 15\lambda_{2q}/16)^{224}$; non è una probabilità di successo.

Tabella D.2: Il tasso di successo è 100% per tutte le 30 repliche di ciascun punto.

λ_{2q}	proxy	$\bar{M}_1$	$\bar{M}_{\text{TOP}4}$	ρ	p
10^{-3}	8.11×10^{-1}	1.93	1.00	1.933	< 0.001
5×10^{-3}	3.49×10^{-1}	1.90	1.00	1.900	< 0.001
10^{-2}	1.21×10^{-1}	1.37	1.00	1.367	< 0.001
2×10^{-2}	1.44×10^{-2}	1.33	1.00	1.333	0.001
5×10^{-2}	2.14×10^{-5}	1.33	1.00	1.333	< 0.001
10^{-1}	2.65×10^{-10}	1.57	1.07	1.469	0.001

Al punto estremo TOP-4 richiede in media 1.07 iterazioni: la robustezza non è assoluta, pur restando il successo 30/30 entro il budget.

D.1.3 Variazione degli Shot

Tabella D.3: Risultati su 30 repliche per punto.

shot	$\bar{M}_1$	$\bar{M}_{\text{TOP4}}$	ρ	p
128	1.37	1.00	1.367	0.001
256	1.57	1.00	1.567	< 0.001
512	1.57	1.00	1.567	0.001
1 024	1.37	1.00	1.367	< 0.001
2 048	1.70	1.00	1.700	< 0.001

TOP-4 è stabile già a 128 shot; M1 oscilla senza andamento monotono, coerentemente con una competizione campionaria fra quattro picchi.

D.1.4 Analisi Congiunta $K \times \lambda_{2q}$

Tabella D.4: Ogni cella è $\bar{M}_{\text{TOP}K}$ su 30 repliche e ha successo 100%.

K	5×10^{-3}	10^{-2}	2×10^{-2}	5×10^{-2}
2	1.00	1.00	1.00	1.00
4	1.00	1.00	1.00	1.00
6	1.00	1.00	1.00	1.00
8	1.00	1.00	1.00	1.00

D.1.5 Parametri Secondari

RZ è virtuale (0 ns), SX/X dura 50 ns e CX 300 ns. T_2 viene variato insieme a T_1 mantenendo $T_2 = 0.8T_1$.

Tabella D.5: TOP-4 ottiene $\bar{M} = 1.00$ e successo 100% in tutti i punti.

T_1/T_2 (μs)	$\bar{M}_1$	$\bar{M}_{\text{TOP4}}$	ρ	p
20/16	1.47	1.00	1.467	< 0.001
50/40	2.20	1.00	2.200	< 0.001
100/80	1.37	1.00	1.367	< 0.001
200/160	1.60	1.00	1.600	0.001
500/400	1.80	1.00	1.800	< 0.001

Tabella D.6: TOP-4 ottiene $\bar{M} = 1.00$ e successo 100% in tutti i punti.

λ_{1q}	$\bar{M}_1$	ρ	p
10^{-4}	1.60	1.600	< 0.001
5×10^{-4}	1.43	1.433	< 0.001
10^{-3}	1.37	1.367	< 0.001
5×10^{-3}	1.67	1.667	< 0.001
10^{-2}	1.57	1.567	< 0.001
2×10^{-2}	1.47	1.467	< 0.001

Tabella D.7: TOP-4 ottiene $\bar{M} = 1.00$ e successo 100% in tutti i punti.

p_{ro}	$\bar{M}_1$	ρ	p
0%	1.33	1.333	0.004
1%	1.50	1.500	< 0.001
2%	1.37	1.367	< 0.001
5%	1.47	1.467	< 0.001
10%	1.30	1.300	0.002
20%	1.47	1.467	< 0.001

Le oscillazioni non monotone di M_1 sconsigliano inferenze causali sui singoli parametri con questa numerosità. Il risultato supportato è più limitato: TOP-4 non varia nei punti osservati.

D.1.6 Livello di Ottimizzazione

Tabella D.8: Conteggi della compilazione effettiva e metriche su 30 repliche.

livello	CX	profondità	$\bar{M}_1$	$\bar{M}_{\text{TOP4}}$	ρ
0	242	427	1.47	1.00	1.467
1	236	421	1.50	1.00	1.500
2	224	412	1.37	1.00	1.367
3	224	412	1.37	1.00	1.367

D.1.7 Diagnostiche della struttura QPE

La sola iterazione di successo non descrive quanta struttura periodica resti nell'istogramma. La Figura D.1 mostra, fra le 30 repliche nominali UC1, i casi con massa massima e minima sui quattro picchi teorici $\{0, 64, 128, 192\}$.

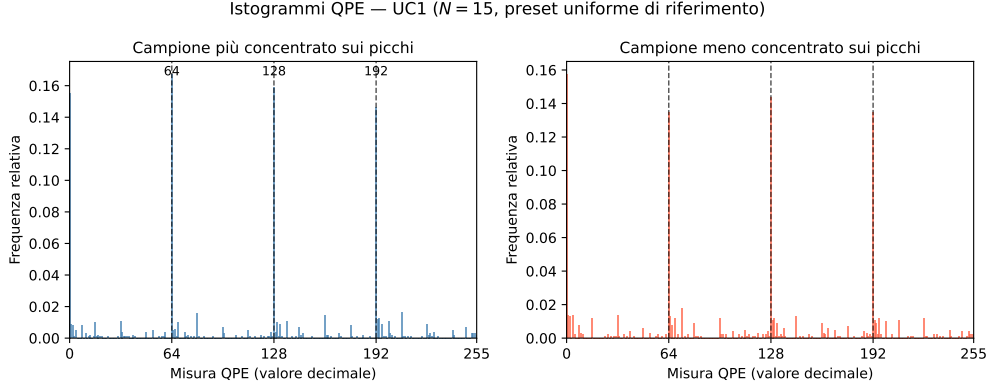


Figura D.1: Repliche UC1 con massa massima e minima sui quattro picchi QPE teorici.

La massa media m sui picchi è convertita nell'indice descrittivo

$$f = \frac{m - 4/256}{1 - 4/256}, \quad (\text{D.1})$$

che vale zero per il fondo uniforme e uno per una distribuzione concentrata sui picchi. Non è una fedeltà. La Tabella D.9 la confronta con il proxy $P_0 = (1 - 15\lambda_{2q}/16)^{224}$.

Tabella D.9: Massa efficace sui picchi e proxy di nessun evento 2Q.

λ_{2q}	f efficace	P_0 proxy
10^{-3}	0.8036	8.11×10^{-1}
5×10^{-3}	0.7039	3.49×10^{-1}
10^{-2}	0.5938	1.21×10^{-1}
2×10^{-2}	0.4286	1.44×10^{-2}
5×10^{-2}	0.1688	2.14×10^{-5}
10^{-1}	0.0421	2.65×10^{-10}
2×10^{-1}	0.0047	6.32×10^{-21}
3×10^{-1}	0.0015	7.47×10^{-33}

D.1.8 Confronto con Zero-Noise Extrapolation

La ZNE digitale scala λ_{1q} e λ_{2q} , mantenendo fissi coerenza e readout. ZNE-2 usa due livelli e ZNE-3 tre livelli condivisi; non è gate folding su hardware.

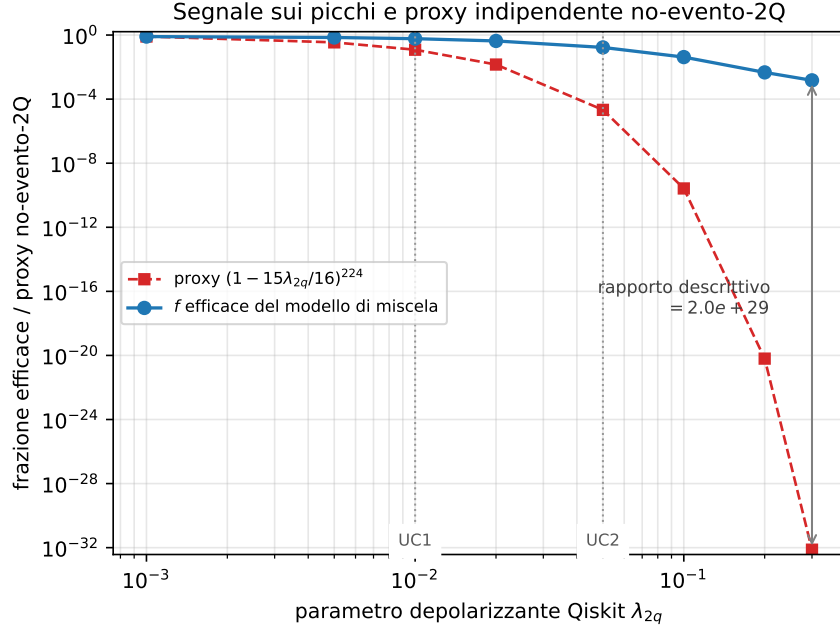


Figura D.2: Il proxy di nessun evento decade più rapidamente della massa strutturata osservata sui picchi.

Tabella D.10: I p sono corretti con Holm nella famiglia M1 contro ZNE-2, ZNE-3 e TOP-4.

strategia	successo	$\bar{M}$	shot/iter.	shot medi totali	p_{Holm}
M1	100%	1.37	1 024	1 399	—
ZNE-2	100%	1.50	2 048	3 072	1.000
ZNE-3	100%	1.57	3 072	4 813	1.000
TOP-4	100%	1.00	1 024	1 024	0.0024

Validazione fuori dalla campagna rumorosa. I moltiplicatori Beauregard per $N = 21/35$ sono validati idealmente e non entrano negli sweep. Per $N = 21$, $a = 2$ e otto controlli, la compilazione globale registra 21 036 CX e profondità 23 081; il proxy di nessun evento 2Q a $\lambda = 10^{-3}$ è 7.2379×10^{-10} , non una probabilità di successo.

D.2 Il Classificatore ML: Architettura, Addestramento e Metriche

Questa appendice documenta il classificatore valutato nella prima fase sperimentale (Capitolo 7) e, soprattutto, separa due domande che nella prima versione della campagna erano state confuse: *il candidato più frequente conduce ai fattori?* (TOP-1) e *esiste un candidato utile fra i primi sedici?* (TOP-16). Tutti i numeri riportati provengono dagli artefatti schema 2 rigenerati con il circuito $N = 15$, $a = 7$ corretto, il medesimo manifest del circuito e seed deterministici.

D.2.1 L’Ipotesi alla Prova: il Classificatore ML

Il Metodo 2 usa un classificatore come *gate di qualità*: riceve la distribuzione normalizzata sui $2^8 = 256$ esiti e, se approva l’istogramma, tenta l’estrazione dei fattori dai quattro candidati più frequenti. L’ablazione M_{TOP4} applica la stessa ricerca senza filtro e consente quindi di distinguere l’effetto del classificatore da quello della ricerca multi-candidato.

Per ciascuno dei due scenari sono stati generati 2 000 istogrammi da 1 024 shot. I cinque parametri del modello uniforme (λ_{1q} , λ_{2q} , T_1 , T_2 e readout) sono stati campionati indipendentemente in un intervallo relativo del $\pm 50\%$ attorno al preset. Il dataset è stato separato in train, validation e test con proporzioni 60/20/20: la scelta dell’architettura usa soltanto la validation; il test viene valutato una sola volta dopo il refit su train+validation.

La regola TOP-1 produce 1 262 positivi e 738 negativi in UC1 (63.1%/36.9%), e 1 453 positivi e 547 negativi in UC2 (72.7%/27.3%). Random Forest, SVM RBF calibrata e MLP (256, 128) sono state confrontate mediante F1 sulla validation. La Tabella D.11 riporta la valutazione indipendente del modello selezionato.

Tabella D.11: Metriche TOP-1 sul test set indipendente (400 istogrammi per scenario)

Scenario	Modello selezionato	F1	Accuratezza	AUC-ROC
UC1	SVM	0.919	0.895	0.965
UC2	SVM	0.923	0.888	0.958

Sulla validation l’SVM ottiene F1 pari a 0.908 in UC1 e 0.929 in UC2, contro 0.811/0.842 della Random Forest e 0.874/0.906 dell’MLP. Queste metriche mostrano che il modello riproduce bene la regola TOP-1; non dimostrano però che il filtro

riduca il costo operativo. Quel confronto richiede gli stessi istogrammi per M1, TOP-4 e M2 ed è riportato nella Sezione 7.2.

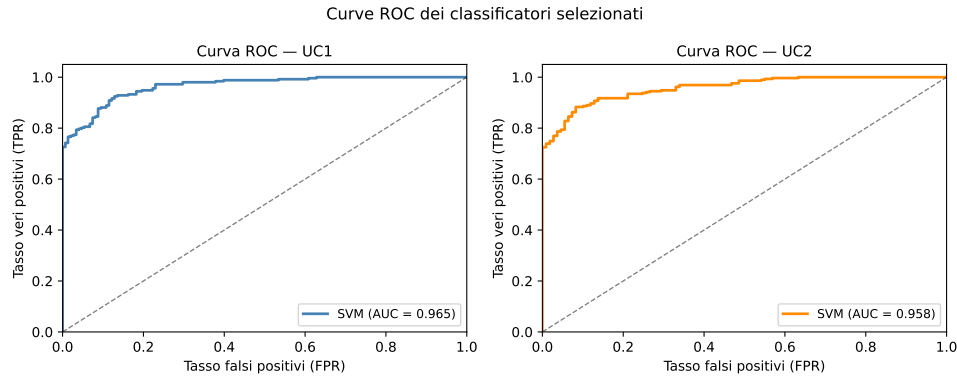


Figura D.3: Curve ROC sul test set indipendente dei due classificatori SVM selezionati mediante validation. Le aree sono 0.965 (UC1) e 0.958 (UC2); la diagonale rappresenta un classificatore casuale.

D.2.2 Audit delle Etichette: TOP-1 e TOP-16 Sono Problemi Diversi

La campagna rigenerata calcola entrambe le etichette sullo *stesso* istogramma, eliminando l'ambiguità del dataset storico. L'esito è riassunto nella Tabella D.12.

Tabella D.12: Bilanciamento delle due regole sul dataset completo. TOP-16 è a classe unica e non consente un addestramento discriminante.

Regola	UC1: positivi/negativi	UC2: positivi/negativi
TOP-1	1 262/738	1 453/547
TOP-16	2 000/0	2 000/0

Il runner non forza un modello sul dataset TOP-16: registra esplicitamente `single-class-dataset` e interrompe l'addestramento per quella regola. M2 usa perciò soltanto i modelli TOP-1 e conserva nel payload `label_top_k=1`. Questo controllo evita di attribuire capacità predittiva a un classificatore costante.

Una diagnostica separata su 60 esecuzioni nominali UC1 chiarisce la struttura della classe TOP-1. La moda è $y = 0$ in 30/60 casi e conduce ai fattori negli altri 30/60; i primi quattro candidati contengono invece almeno un esito utile in 60/60 casi. La Tabella D.13 riporta la distribuzione della moda.

Tabella D.13: Distribuzione della misura più frequente su 60 istogrammi nominali indipendenti.

Valore della moda	Occorrenze	Frazione
0	30	50.0%
64	5	8.3%
128	15	25.0%
192	10	16.7%

L'esito $y = 0$ rappresenta la fase nulla e non fornisce il periodo, ma non implica che l'istogramma sia privo di segnale. Nel campione rappresentativo della Figura D.4, la moda $y = 0$ raccoglie 176 conteggi, mentre $y = 64$, 192 e 128 ne raccolgono rispettivamente 160, 141 e 131: tutti e tre conducono ai fattori. La massa complessiva sui quattro picchi è 0.594.

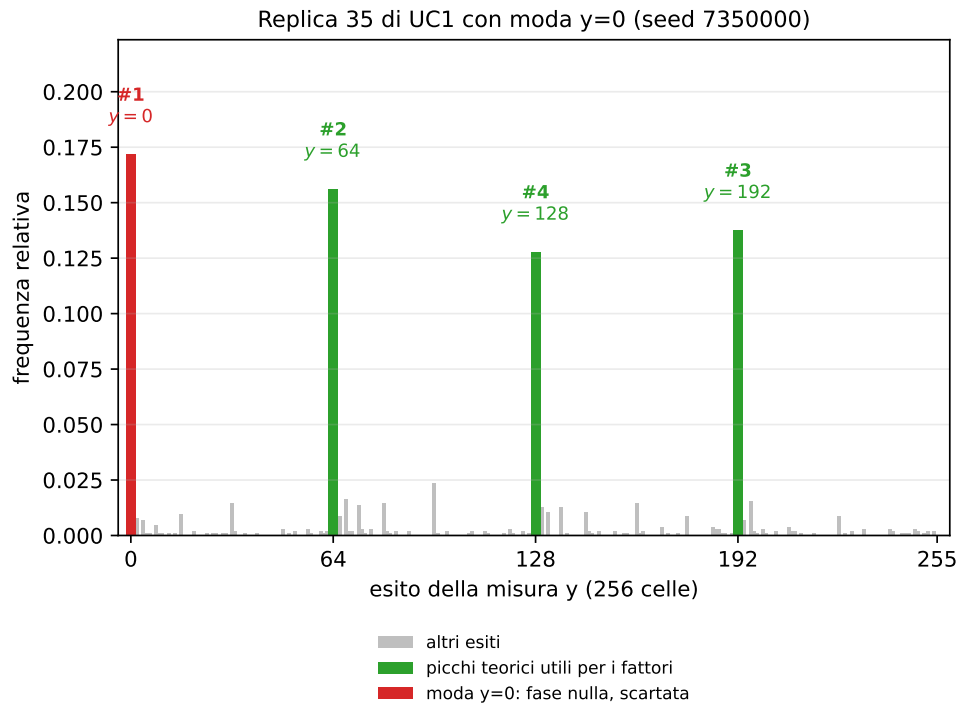


Figura D.4: Esempio nominale UC1 in cui TOP-1 fallisce perché la moda è $y = 0$, mentre i successivi tre picchi conducono ai fattori. Il classificatore TOP-1 apprende quindi una regola sul candidato dominante, non una misura generale della presenza di segnale quantistico.

Perché TOP-16 Produce una Classe Quasi Sempre Positiva

Per $N = 15$, $a = 7$ e otto bit di controllo, 63 dei 256 esiti conducono ai fattori tramite la procedura di frazioni continue adottata. Anche per un istogramma uniforme, la probabilità che nessuno dei primi K candidati sia utile è quindi

$$P_{\text{neg}}(K) = \frac{\binom{256-63}{K}}{\binom{256}{K}}. \quad (\text{D.2})$$

Per $K = 16$ essa vale soltanto 0.9275%: la regola assegna dunque un'etichetta positiva a oltre il 99% degli istogrammi uniformi per puro effetto combinatorio. Il risultato 2000/0 osservato nei due dataset è coerente con questo limite e non autorizza a concludere che il segnale sia perfetto.

Tabella D.14: Probabilità di non includere alcuno dei 63 esiti utili scegliendo K celle da 256 senza informazione sulla distribuzione.

Regola	$P_{\text{neg}}(K)$
TOP-1	75.39%
TOP-2	56.76%
TOP-4	32.06%
TOP-16	0.93%

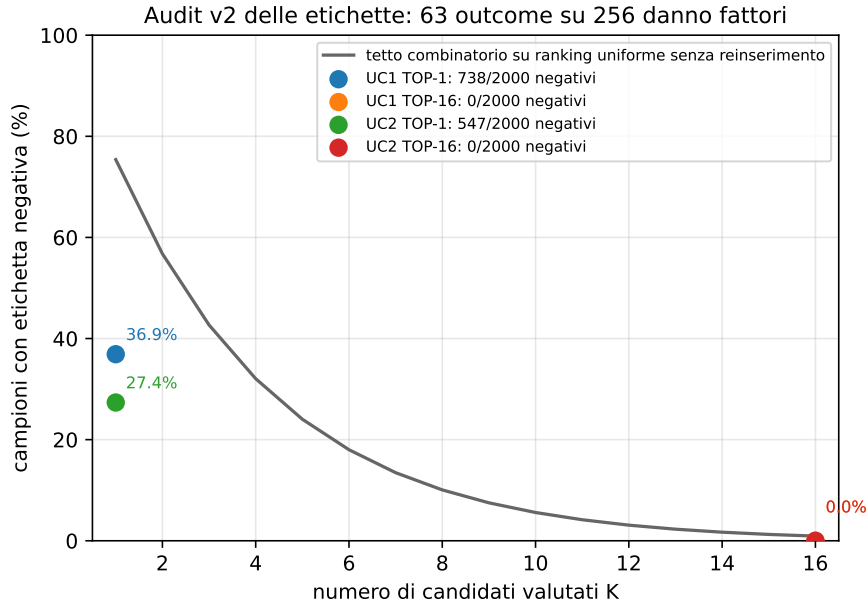


Figura D.5: Decadimento di $P_{\text{neg}}(K)$ al crescere del numero di candidati. La figura è rigenerata dai dati schema 2 e rende visibile perché TOP-16 non definisce un problema di classificazione bilanciato su questa istanza.

Il verdetto ha un perimetro preciso: non mostra che l'apprendimento automatico sia inutile in generale, ma che un filtro a valle addestrato su TOP-1 replica una decisione già superata da TOP-4, mentre la regola TOP-16 è quasi sempre positiva per costruzione. Questo motiva lo spostamento dell'apprendimento dalla distribuzione finale alla sindrome di errore nei capitoli dedicati alla QEC.

Appendice E

Fase 2: la Campagna QEC nel Dettaglio

E.1 La Decodifica Appresa: la Campagna Completa

Questa sezione riporta per intero la campagna sulla decodifica appresa, di cui il Capitolo [efchap:surface](#) riporta la sintesi.

La sezione precedente ha mostrato che il modello di rumore usato per *simulare* il codice è un'idealizzazione. Esiste però una seconda idealizzazione, indipendente dalla prima e altrettanto conseguente, che riguarda il modello di rumore usato per *decodificare*. Il MWPM non è un algoritmo generico: presuppone che gli errori si organizzino in un grafo, ossia che ogni meccanismo d'errore accenda al più due detection events e sia quindi rappresentabile come un arco. È questa proprietà — la **decomponibilità in archi** — a rendere il problema della decodifica un accoppiamento di peso minimo, ed è ciò che la libreria richiede esplicitamente quando si costruisce il modello d'errore con `decompose_errors=True`. Un errore correlato che colpisca simultaneamente due qubit dati adiacenti accende invece fino a quattro detector e non ammette alcun arco che lo rappresenti: non viola l'ipotesi di Pauli, che resta soddisfatta, ma viola quella di decomponibilità. Le due idealizzazioni sono distinte e vanno tenute separate.

Questa distinzione individua con precisione la domanda a cui la presente sezione risponde: *esiste un regime in cui un decoder **appreso dai dati** — che non riceve un grafo esplicito degli errori, ma stima la regola decisionale dai campioni — fa meglio del decoder analitico?* La domanda chiude anche il cerchio metodologico dell'intera tesi. Il Capitolo 7 ha stabilito che l'apprendimento automatico applicato *a valle* dell'esecuzione non aggiunge nulla; qui lo stesso strumento — un percettrone

multistrato di `scikit-learn`, la medesima tecnologia del classificatore giudicato superfluo — viene applicato alla **sindrome**, cioè a un dato che conserva relazioni spaziali e temporali non rappresentate dal solo istogramma finale. Cambia il punto di applicazione, non la tecnologia: è esattamente il confronto che permette di attribuire l'esito al *dove*, e non al *come*.

E.1.1 Disegno sperimentale

Il dato di addestramento non richiede alcuna infrastruttura aggiuntiva: è già ciò che Stim produce per stimare p_L . Il campionatore restituisce la matrice dei detection events e il corrispondente esito dell'osservabile logico, che sono rispettivamente le variabili indipendenti e l'etichetta del problema di apprendimento supervisionato:

```
det, obs = sampler.sample(shots, separate_observables=True)
# X = det (detection events)      -> sindrome misurata
# y = obs (logical observable)    -> flip logico da predire
```

Listing E.1: Il dataset del decoder appreso coincide con l'output del campionatore di Stim: `det` è la sindrome osservata, `obs` l'esito logico da predire. Il decoder analitico e quello appreso ricevono quindi esattamente la stessa informazione.

Il modello è un MLP con due strati nascosti (128, 64), attivazione ReLU, ottimizzatore Adam e arresto anticipato su una frazione di validazione interna; il decoder di riferimento è PyMatching sullo stesso insieme di test. Tutte le stime adottano il codice ruotato in base Z con $\text{rounds} = d$ e rumore *circuit-level*, come nella campagna della Sezione 9.4. La Tabella E.1 riassume i quattro esperimenti.

	Domanda	Condizione	Campioni
E1	la rete sostituisce il matching?	rumore nominale, modello esatto	4×10^5 per punto
E1b	quanto costa in dati il pareggio?	$d = 3$, $p = 3 \times 10^{-3}$	fino a 8×10^5
E2	e se il rumore esce dal modello?	crosstalk correlato	4×10^5 per punto
E4	e se la si affianca invece di sostituirla?	crosstalk correlato	8×10^5 per punto

Tabella E.1: I quattro esperimenti sulla decodifica appresa. E3, discusso nella Sezione E.1.4, riusa la configurazione di E2 senza nuovi campionamenti.

E.1.2 E1 — la sostituzione a parità di modello

Il primo esperimento pone il confronto nelle condizioni più favorevoli al decoder analitico: rumore uniforme e modello d'errore esatto, cioè esattamente ciò per cui il MWPM è costruito. Su quattordici configurazioni ($d = 3, 5$; sette valori di p) la rete **non supera mai** il matching con significatività statistica. A distanza 3 lo affianca, con un rapporto $p_L^{\text{MWPM}}/p_L^{\text{rete}}$ che sale da 0.83 a $p = 2 \times 10^{-3}$ fino a 1.01 a $p = 10^{-2}$; a distanza 5 resta indietro di un ordine di grandezza (da 0.05 a 0.29).

L'andamento monotono suggerisce la spiegazione, che il pannello (b) della Figura E.1 verifica direttamente: la rete è limitata dal **volume di dati**, non dalla capacità del modello. Fissando $d = 3$ e $p = 3 \times 10^{-3}$ e facendo variare il solo insieme di addestramento, il rapporto cresce in modo regolare da 0.50 (2.5×10^4 campioni) a 1.02 (8×10^5 campioni). L'ultimo punto misura $p_L = 3.98 \times 10^{-3}$ contro 4.04×10^{-3} del matching: un sorpasso nominale che, su un insieme di test di 10^5 campioni, vale 0.2 deviazioni standard e **non è statisticamente significativo**. Il risultato di E1 è dunque un pareggio, e il suo contenuto informativo è il prezzo di quel pareggio: dell'ordine di 10^5 – 10^6 campioni a distanza 3, con un fabbisogno che cresce rapidamente con d — coerente con il fatto che i decoder neurali allo stato dell'arte ricorrono ad architetture ricorrenti e a volumi di dati di diversi ordini di grandezza superiori [17, 53].

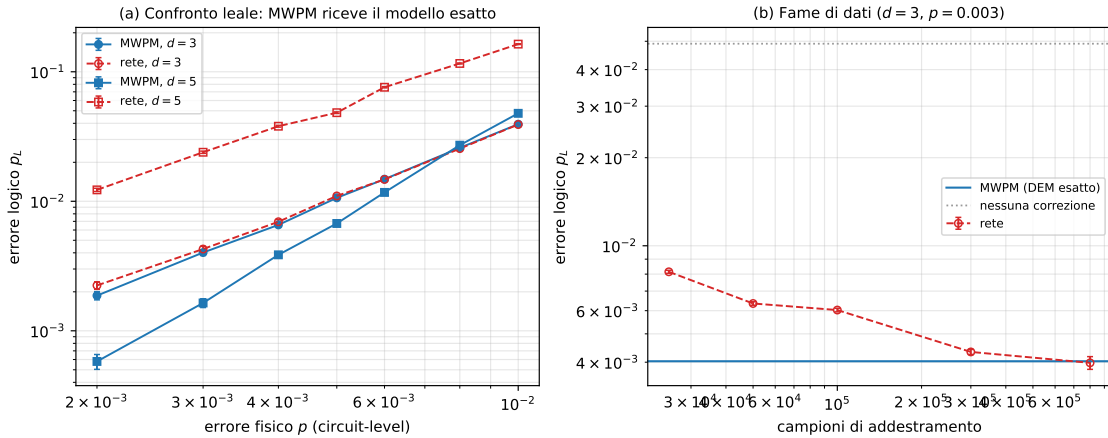


Figura E.1: (a) Errore logico del decoder appreso e del MWPM in funzione dell'errore fisico, per $d = 3$ e $d = 5$, con modello d'errore esatto fornito al matching. La rete affianca il matching a distanza 3 e resta al di sopra a distanza 5. (b) Errore logico della rete al crescere dell'insieme di addestramento ($d = 3$, $p = 3 \times 10^{-3}$): l'avvicinamento al matching è monotono e il pareggio si raggiunge fra 3×10^5 e 8×10^5 campioni. Rigenerabile da `figure_src/gen_qec_neural.py`.

E.1.3 E2 — quando il rumore esce dal modello del decoder

Il secondo esperimento rimuove l'ipotesi di decomponibilità. Al circuito viene aggiunto un canale correlato a due qubit fra i **qubit dati fisicamente adiacenti** sulla griglia, una volta per ciclo di sindrome e con intensità p_{ct} : è il modello elementare del **crosstalk**, la quarta fonte fisica di rumore discussa nel Capitolo 4, che l'accoppiamento capacitivo residuo produce fra qubit vicini. L'errore fisico resta fissato a $p = 3 \times 10^{-3}$, sotto la soglia misurata nella Sezione 9.4. Si confrontano tre bracci: il matching con il modello *nominale* (rumore uniforme — ciò che la calibrazione del dispositivo dichiara), il matching con il modello del circuito *reale* (che include il crosstalk), e la rete addestrata sui campioni effettivamente osservati.

Il crosstalk degrada la decodifica in modo severo: a $p_{ct} = 5 \times 10^{-3}$ l'errore logico a $d = 3$ passa da 3.8×10^{-3} a 3.1×10^{-2} , quasi un ordine di grandezza, pur restando l'errore fisico invariato. In questo regime la rete supera il matching su tutte e quattro le intensità testate a distanza 3, con guadagni fra 1.14 e 1.19 (Tabella E.2). A distanza 5 il vantaggio non si manifesta: la rete perde, per la stessa ragione di E1 — lo spazio delle sindromi cresce più rapidamente della sua capacità di generalizzazione.

d	p_{ct}	nessuna corr.	MWPM nominale	MWPM reale	rete
3	5×10^{-3}	0.0845	0.03097	0.05656	0.02707
	1×10^{-2}	0.1163	0.05966	0.08975	0.05012
	2×10^{-2}	0.1746	0.11184	0.15107	0.09547
	4×10^{-2}	0.2652	0.20806	0.25481	0.17568
5	5×10^{-3}	0.2007	0.03521	0.02796	0.08266
	1×10^{-2}	0.2648	0.08655	0.06526	0.13957
	2×10^{-2}	0.3565	0.19933	0.15662	0.28906
	4×10^{-2}	0.4459	0.37458	0.32515	0.44634

Tabella E.2: Errore logico p_L in presenza di crosstalk fra qubit dati adiacenti, a errore fisico costante $p = 3 \times 10^{-3}$. “Nessuna correzione” è la frequenza con cui l'osservabile logico risulta alterato in assenza di decodifica. A distanza 3 la rete domina entrambe le varianti del matching; a distanza 5 il quadro si inverte.

Un'osservazione inattesa: conoscere il rumore non basta. Il braccio con il modello d'errore *reale* — quello che conosce il crosstalk — si comporta in modo opposto alle due distanze, ed è il risultato più istruttivo dell'esperimento. A $d = 3$ è il **peggiore** dei tre (5.7×10^{-2} contro 3.1×10^{-2} del modello nominale a $p_{ct} = 5 \times 10^{-3}$): fornire al matching l'informazione corretta lo peggiora. La ragione è strutturale e non contraddittoria. Poiché l'errore correlato non è rappresentabile come arco, la

libreria è costretta a scomporlo in archi equivalenti, e su un grafo piccolo come quello di distanza 3 gli archi spurî introdotti dalla scomposizione sviano l'accoppiamento più di quanto l'informazione aggiuntiva lo guidi. A $d = 5$, dove il grafo è più ricco e la scomposizione pesa relativamente meno, il bilancio si inverte e il modello reale torna a essere il migliore.

Ne discende un enunciato che vale la pena isolare, perché delimita esattamente ciò che un decoder analitico può e non può fare: *quando la classe di errori non è esprimibile nella struttura del decoder, il limite non è la calibrazione ma la struttura stessa* — e migliorare la stima del rumore può non produrre alcun beneficio, o risultare controproducente.

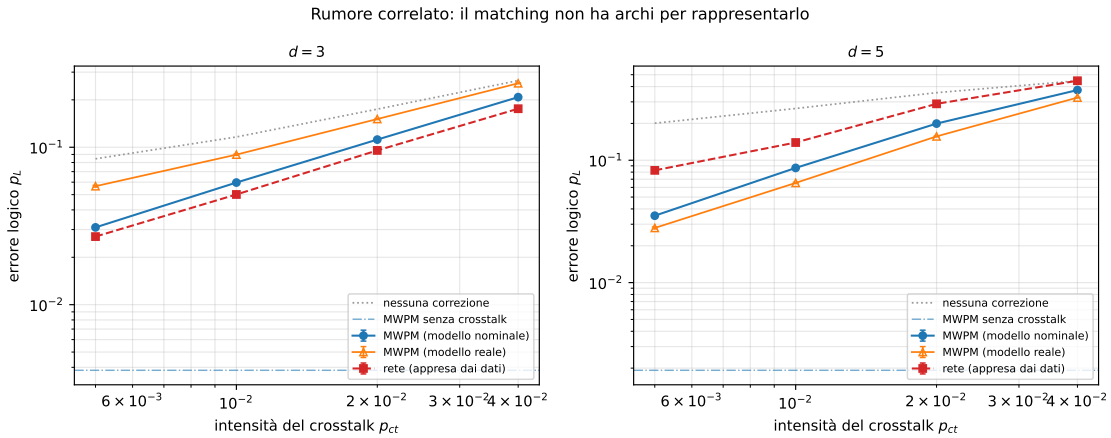


Figura E.2: Errore logico in funzione dell'intensità del crosstalk, a errore fisico costante $p = 3 \times 10^{-3}$. A distanza 3 la rete (rosso) domina il matching sia con modello nominale (blu) sia con modello reale (arancio), che risulta il peggiore; a distanza 5 l'ordine si inverte. La linea tratteggiata inferiore indica l'errore logico del matching in assenza di crosstalk, cioè il livello che la decodifica raggiungerebbe se il modello fosse corretto.

E.1.4 E3 — la rete come validatore della correzione

I due esperimenti precedenti valutano la rete come *sostituto* del decoder. Una domanda diversa, e operativamente più modesta, è se la rete sappia riconoscere gli shot in cui il matching ha sbagliato — cioè se possa **confermare o negare** la correzione proposta, senza doverla rimpiazzare. Si definisce allora la confidenza che la rete assegna alla risposta effettivamente data dal matching, e si verifica quanto una confidenza bassa predica un fallimento reale.

Nella configurazione $d = 3$, $p_{ct} = 10^{-2}$, in cui il matching sbaglia 5966 shot su 10^5 , il flag così costruito discrimina i fallimenti con $AUC = 0.941$. La Tabella E.3 ne

riporta il compromesso: abbassando la soglia si segnalano meno shot con precisione crescente, fino a individuare l'85.9% di errori veri segnalando appena lo 0.59% delle esecuzioni.

Soglia di confidenza	Shot segnalati	Precision	Recall
0.5	2.92%	0.664	0.324
0.3	1.54%	0.783	0.202
0.2	1.16%	0.825	0.160
0.1	0.59%	0.859	0.085

Tabella E.3: Capacità della rete di segnalare gli shot in cui il MWPM ha sbagliato ($d = 3$, $p_{ct} = 10^{-2}$, $AUC = 0.941$). La precisione cresce al restringersi della segnalazione: segnalando lo 0.59% degli shot, l'85.9% delle segnalazioni corrisponde a un fallimento reale.

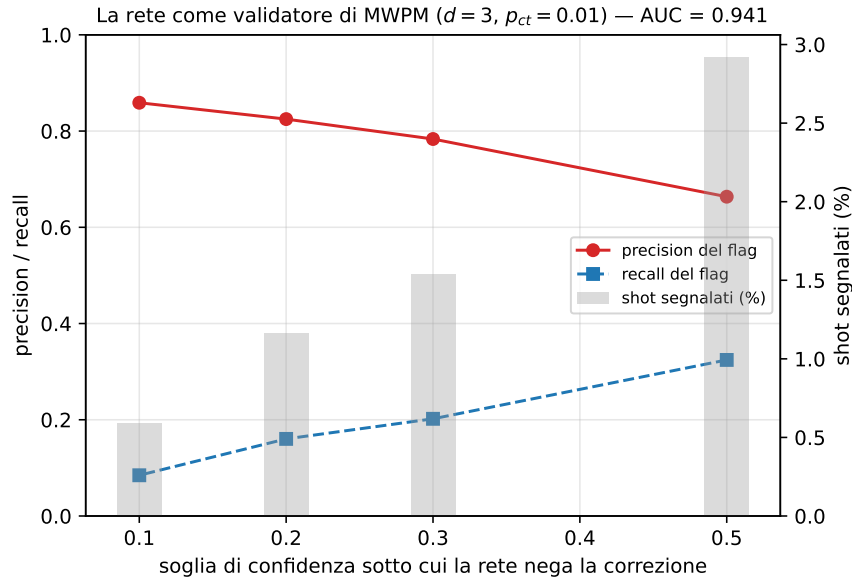


Figura E.3: Precision e recall del flag in funzione della soglia di confidenza, con la frazione di shot segnalati (barre, asse destro). Il flag non sostituisce la decodifica: la annota.

E.1.5 E4 — il decoder ibrido

L'esito di E3 suggerisce la costruzione che chiude la sezione. Se la rete riconosce i fallimenti del matching senza saper decodificare meglio di esso, allora il suo impiego corretto non è sostituirlo ma **correggerlo**. Si definisce quindi un decoder **ibrido** in

forma residuale:

$$\hat{y}_{\text{ibrido}} = \hat{y}_{\text{MWPM}} \oplus [\text{Pr}(\text{MWPM sbaglia} \mid \text{sindrome}) \geq \tau], \quad (\text{E.1})$$

dove $\oplus$ è la somma modulo 2 e la rete riceve in ingresso la sindrome e la decisione del matching. La soglia di ribaltamento τ è scelta su un insieme di **validazione separato** e poi applicata invariata all'insieme di test: sceglierla sul test costituirebbe una fuga di informazione, e su guadagni dell'ordine di pochi punti percentuali sarebbe un vizio sufficiente a invalidare il risultato. Fra le soglie candidate figura il valore che non ribalta mai, cosicché il criterio di validazione può sempre ricadere sul matching puro quando nessun ribaltamento conviene.

La formulazione ha una motivazione precisa, oltre alla convenienza pratica: alla rete non si chiede più di riprodurre da zero il calcolo di parità che il matching già esegue correttamente nella grande maggioranza dei casi, ma soltanto di modellarne il *residuo*, che è un evento raro e strutturato. È un compito più facile, ed è per questo che riesce dove la sostituzione fallisce.

d	p_{ct}	MWPM	rete sola	ibrido	ribalta	McNemar
3	0	0.00415	0.00409	0.00417	0.10%	$p = 0.67$
	5×10^{-3}	0.03089	0.02680	0.02607	1.19%	3×10^{-87}
	1×10^{-2}	0.05829	0.04917	0.04851	2.69%	1×10^{-156}
	2×10^{-2}	0.11144	0.09248	0.09183	5.38%	2×10^{-312}
	4×10^{-2}	0.20750	0.17615	0.17455	11.04%	$< 10^{-320}$
5	0	0.00165	0.01900	0.00165	0.00%	—
	5×10^{-3}	0.03561	0.06311	0.03521	0.32%	1×10^{-3}
	1×10^{-2}	0.08704	0.12874	0.08428	1.17%	3×10^{-30}
	2×10^{-2}	0.20231	0.25198	0.19662	2.88%	6×10^{-51}
	4×10^{-2}	0.37581	0.44182	0.37385	2.96%	4×10^{-7}
7	0	0.00055	0.06888	0.00055	0.00%	—
	5×10^{-3}	0.03707	0.24671	0.03707	0.00%	—
	1×10^{-2}	0.11319	0.36749	0.11318	0.00%	$p = 0.32$
	2×10^{-2}	0.29048	0.46219	0.29048	0.00%	—
	4×10^{-2}	0.46741	0.49980	0.46741	0.00%	—

Tabella E.4: Errore logico del decoder ibrido (E.1) confrontato con il MWPM e con la rete usata come sostituto. “Ribalta” è la frazione di shot in cui la rete inverte la decisione del matching. Il valore di significatività è quello del test di McNemar, appaiato sugli stessi campioni: poiché i due decoder differiscono *soltanto* sugli shot ribaltati, le coppie discordanti coincidono con essi e il test è esatto.

L’ibrido riduce l’errore logico in tutte e otto le configurazioni con crosstalk a

distanza 3 e 5, con guadagni del 18–21% nel primo caso e dell’1–3% nel secondo, tutti significativi al test di McNemar. Il dettaglio delle coppie discordanti rende l’effetto leggibile: a $d = 3$ e $p_{ct} = 4 \times 10^{-2}$ la rete rompe 7745 shot che il matching aveva risolto correttamente e ne aggiusta 14335, un rapporto di quasi due a uno; a $d = 5$ il bilancio si stringe (2769 contro 3160), da cui il guadagno più contenuto ma comunque netto.

Due proprietà meritano attenzione perché riguardano il comportamento del sistema fuori dal regime per cui è stato addestrato. La prima è che la frazione di ribaltamenti **scala con l’intensità del crosstalk** (dallo 0.10% all’11.04% a distanza 3): la rete non interviene a caso, ma in misura proporzionale a quanto il modello del matching è sbagliato. La seconda è il comportamento in assenza di crosstalk, dove il matching è già adeguato e non c’è nulla da correggere: a $d = 5$ la rete effettua **zero ribaltamenti** e l’ibrido coincide cifra per cifra con il MWPM, mentre a $d = 3$ ribalta lo 0.10% degli shot con effetto statisticamente nullo ($p = 0.67$). Il decoder ibrido *degrada con grazia*: riconosce di non avere nulla da aggiungere e si fa da parte. Va detto con esattezza che questa garanzia riguarda il criterio di validazione e non ogni singolo punto dell’insieme di test — il valore leggermente superiore a $d = 3$, $p_{ct} = 0$ ne è un esempio, entro il rumore statistico — sicché la formulazione corretta è che l’ibrido non risulta *mai peggiore del matching in modo statisticamente rilevabile*.

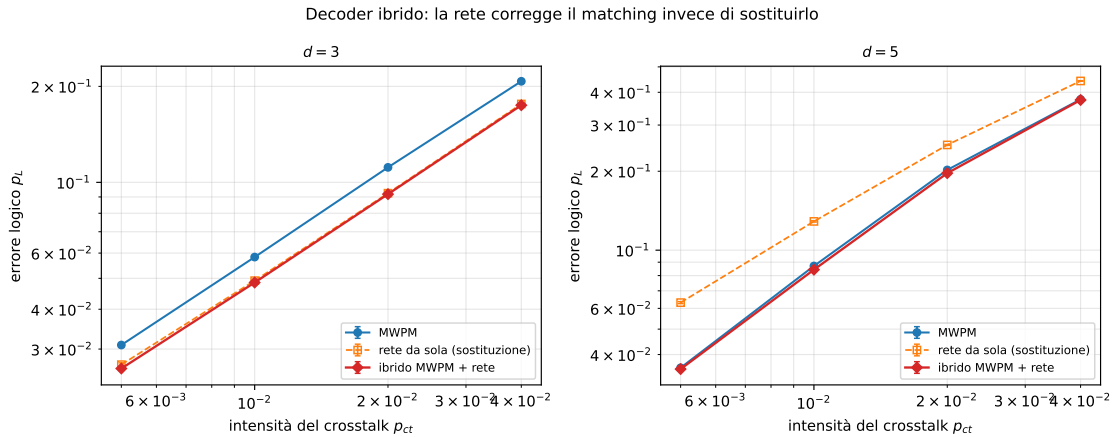


Figura E.4: Errore logico dei tre decoder in funzione dell’intensità del crosstalk. L’ibrido (rosso) è il migliore a entrambe le distanze; la rete usata come sostituto (arancio) domina il matching a $d = 3$ ma lo perde a $d = 5$, dove l’ibrido continua invece a guadagnare.

Il limite: il vantaggio non sopravvive alla distanza 7. Il dato più importante della Tabella E.4 è però l’ultimo blocco, e va enunciato senza attenuazioni. A distanza

7 l'ibrido **non ribalta alcuno shot** in nessuna delle cinque configurazioni: la soglia scelta in validazione è ogni volta quella corrispondente a “non ribaltare mai”, e il decoder ibrido coincide con il MWPM cifra per cifra. Il guadagno svanisce, e la progressione lungo le tre distanze è inequivocabile: 18–21% a $d = 3$, 1–3% a $d = 5$, esattamente zero a $d = 7$.

Due meccanismi possono spiegarlo, e vanno separati perché conducono a rimedi diversi. Il primo è la **scarsità di esempi**: l'evento da riconoscere è il fallimento del matching, la cui frequenza è p_L , e p_L decresce esponenzialmente con d . In assenza di crosstalk, a $d = 7$ si hanno $p_L = 5.5 \times 10^{-4}$ e 8×10^5 shot, ossia 264 esempi positivi nella partizione di addestramento, per un ingresso a 336 dimensioni. Il secondo è la **capacità rappresentativa**: la rete densa impiegata tratta i detector come dimensioni indipendenti, e ciò che distingue le due classi di errore su un reticolo grande è una correlazione a lungo raggio fra detector distanti.

Un primo indizio viene dal semplice conteggio. La spiegazione per scarsità è esatta nella configurazione senza crosstalk, dove i fallimenti del matching sono effettivamente poche centinaia; ma nelle altre quattro configurazioni a $d = 7$ la rete dispone da 1.8×10^4 a 2.2×10^5 esempi positivi — fino a duecentomila — e continua a non ribaltare alcuno shot. La spiegazione copre dunque una configurazione su cinque.

L'indizio non basta però a stabilire la causa, perché fra $d = 5$ (che funziona) e $d = 7$ (che non funziona) cambiano *due* cose insieme: la distanza del codice e il numero di detector. L'esperimento E9 le separa, sfruttando il fatto che il numero di detector è il prodotto fra gli stabilizzatori per ciclo e il numero di cicli, e può quindi essere variato a distanza fissa. Ne risulta un incrocio 2×2 le cui celle diagonali distinguono le due ipotesi, riportato nella Tabella E.5.

d	cicli	detector	$p_{ct} = 0.005$		$p_{ct} = 0.02$	
			es. pos.	residuo	es. pos.	residuo
7	3	144	5 933	si astiene	61 900	interviene
7	7	336	17 665	si astiene	139 275	si astiene
5	14	336	48 657	si astiene	188 582	si astiene
5	5	120	17 235	interviene	96 167	interviene

Tabella E.5: Esperimento E9: incrocio fra distanza del codice e numero di detector, ottenuto variando il numero di cicli di estrazione della sindrome. “Es. pos.” è il numero di fallimenti del MWPM disponibili all’addestramento su 4.8×10^5 shot; “residuo” indica se il criterio di validazione seleziona una soglia che ribalta almeno uno shot su diecimila. Le righe centrali sono le celle diagonali, quelle che distinguono le ipotesi: a distanza 5 con 336 detector e 1.9×10^5 esempi il residuo si astiene, a distanza 7 con 144 detector e tre volte meno esempi interviene. Il comportamento segue la colonna, non la riga.

L’esito è netto su entrambe le diagonali e va letto per colonne. A 336 detector il residuo si astiene sempre, a qualunque distanza e con qualunque numerosità — fino a 1.9×10^5 esempi positivi; a 120 e 144 detector interviene, anche a distanza 7 e anche con tre volte meno esempi. **L’ipotesi della distanza è confutata, quella della larghezza dell’ingresso è confermata.**

Una sfumatura completa l’enunciato e lo rende più preciso. A 144 detector con soli 5 933 esempi positivi il residuo si astiene: la numerosità è dunque un vincolo reale, al margine inferiore. Non è però il vincolo che opera nell’esperimento E4 a distanza 7, dove gli esempi erano 1.4×10^5 e la rete si asteneva comunque. La formulazione corretta è che disporre di abbastanza esempi è *necessario ma non sufficiente*, e che a parità di numerosità sufficiente è la larghezza dell’ingresso a decidere.

Che cosa significhi esattamente “si astiene” (E12). Il verbo va però disinnescato, perché astenersi è l’esito di una *selezione*: fra le soglie candidate figura quella corrispondente a “non ribaltare mai”, e che vinca in validazione significa che agire non conviene, non che non vi sia nulla da vedere. La distinzione non è oziosa, perché le due letture conducono a rimedi opposti — un’architettura diversa nella prima, una regola di decisione diversa nella seconda. L’esperimento E12 la risolve misurando l’AUC del residuo, che quantifica quanto la rete *ordini* i fallimenti del matching indipendentemente da qualunque soglia, e affiancando alla soglia scelta onestamente in validazione una soglia *oracolo*, la migliore sul test set stesso: inutilizzabile in

pratica, ma limite superiore assoluto di ciò che qualunque regola di decisione potrebbe ottenere.

detector	d	cicli	AUC	guadagno onesto	guadagno oracolo
24	3	3	0.912	1.2215	1.2233
120	5	5	0.769	1.0206	1.0215
144	7	3	0.788	1.0066	1.0066
336	7	7	0.621	1.0000	1.0000
336	5	14	0.573	1.0000	1.0003

Tabella E.6: Esperimento E12 a $p_{ct} = 2 \times 10^{-2}$. L’AUC è calcolata sul test set con bersaglio “il MWPM ha sbagliato”. Il guadagno oracolo usa la soglia migliore sul test set stesso e non è quindi realizzabile: serve unicamente a stabilire quanto del divario sia imputabile alla calibrazione. Ai livelli di crosstalk più bassi il quadro è il medesimo, con AUC sistematicamente più alte (0.958 a 24 detector, 0.771 a 336).

Il risultato smentisce entrambe le letture immediate. **La rete non è cieca:** a 336 detector l’AUC vale 0.57–0.77, ben sopra il caso, e la rete ordina dunque i fallimenti del matching assai meglio del lancio di una moneta. **Ma non si tratta neppure di calibrazione:** il guadagno oracolo è 1.0000, sicché nemmeno scegliendo la soglia sui dati di valutazione si otterrebbe alcunché.

La spiegazione sta nella forma stessa della correzione residuale, ed è quantitativa. Il ribaltamento è *simmetrico*: ribaltare uno shot che il matching aveva risolto costa esattamente quanto ribaltarne uno che aveva sbagliato rende. Perché l’operazione sia in attivo occorre dunque che fra gli shot ribaltati la **precisione superi il cinquanta per cento** — una soglia netta, non un ottimo graduale. L’AUC necessaria a raggiungerla dipende dal tasso di base: a 24 detector e crosstalk 5×10^{-3} il tasso di base è del 3.1% e un’AUC di 0.958 concentra i fallimenti abbastanza da superare la soglia, con un guadagno del 19%; a 336 detector, a parità di tasso di base, un’AUC di 0.771 richiederebbe un fattore di concentrazione quattordici volte superiore a quello che produce. Nessuna soglia porta la precisione sopra il cinquanta per cento, e il guadagno è allora *esattamente* 1.000 — non approssimativamente: è questa discontinuità a spiegare il salto brusco osservato lungo la Tabella E.5.

Ne discendono due conseguenze. La prima è che **l’astensione del criterio di validazione è corretta e non conservativa**: la soglia oracolo conferma che intervenire non converrebbe, e il metodo si comporta esattamente come dichiarato al momento della sua costruzione. La seconda riguarda il rimedio, e ne precisa il criterio di successo: un’architettura convoluzionale o a grafo è utile se e soltanto se innalza l’AUC quanto basta a riportare la precisione sopra la soglia del cinquanta per

cento, che è una condizione verificabile in anticipo su una rete addestrata — assai più informativa dell’osservazione che il modello “non regge”.

Questa è la delimitazione del risultato di M10, e conviene formularla in positivo. Il guadagno del decoder ibrido è supportato da un valore nominale estremamente piccolo del test di McNemar — a $d = 3$, $p < 10^{-320}$ — ma vive nel regime in cui la sindrome è stretta abbastanza da lasciare alla rete densa una discriminazione sufficiente a superare la soglia di precisione. Il valore p quantifica l’incompatibilità con l’ipotesi nulla nel modello del test, non la dimensione pratica dell’effetto, riportata separatamente tramite la riduzione di p_L . Poiché è proprio la crescita della distanza a costituire la strategia della correzione d’errore scalabile, il metodo così com’è non accompagna il codice nel regime che conta. Il rimedio indicato dalla diagnosi non è raccogliere più campioni — ne bastavano già — ma innalzare la discriminazione a parità di larghezza dell’ingresso, il che significa cambiare architettura: una rete convoluzionale o a grafo, che condivida i pesi fra regioni equivalenti del reticolo anziché trattare i 336 detector come dimensioni indipendenti, sfrutta la simmetria traslazionale del problema e rappresenta con pochi parametri le correlazioni locali che la rete densa deve stimare una per una. È l’estensione naturale, e la si indica fra gli sviluppi futuri (Capitolo 11).

E.1.6 E6 — e se il modello del decoder fosse semplicemente sbagliato?

Gli esperimenti fin qui condotti concedono al MWPM un vantaggio che l’hardware non offre: il detector error model che ne pesa gli archi è generato da Stim *dal circuito stesso*, ed è quindi una descrizione esatta del rumore. Un decoder in produzione lavora invece su un modello ricavato dalla calibrazione del dispositivo — misurata a intervalli, soggetta a deriva, inevitabilmente diversa dal rumore del momento. Se il vantaggio del modello appreso nascesse da questa discrepanza, sarebbe un vantaggio molto più generale di quello osservato con il crosstalk, perché la mis-calibrazione è la condizione ordinaria di ogni QPU.

L’esperimento la isola: il circuito viene campionato con il rumore reale, mentre il matching è costruito su un modello in cui l’errore di *misura* è sbagliato di un fattore $(1 + \delta)$ e quello di gate resta corretto.

Perché la perturbazione dev’essere sbilanciata. Un primo tentativo scalava tutte le probabilità di errore dello stesso fattore, e non produceva alcun effetto

misurabile: il costo rilevato era 1.000, 1.004 e 1.000 per δ pari a 0, +100% e -50%. La ragione è istruttiva e merita di essere esplicitata, perché delimita ciò che una calibrazione errata può e non può fare. Il MWPM minimizza il peso totale del matching, e il peso di un arco vale $\log \frac{1-p_e}{p_e} \approx -\log p_e$ per p_e piccolo: moltiplicare tutte le p_e per una costante trasla *tutti* i pesi della stessa quantità, e un matching di peso minimo è invariante rispetto a una traslazione uniforme. Ciò che rende il matching vulnerabile non è dunque sbagliare la *scala* del rumore, ma il suo *bilanciamento*: il rapporto fra archi temporali, che rappresentano errori di misura, e archi spaziali, che rappresentano errori sui dati. È questo rapporto che l'esperimento perturba.

d	δ	MWPM esatto	MWPM sbagliato	costo	rete	rete/MWPM
3	-80%	0.00407	0.00415	1.018	0.00525	0.79
	-50%	0.00435	0.00435	1.000	0.00493	0.88
	0	0.00427	0.00427	1.000	0.00558	0.77
	+100%	0.00387	0.00382	0.987	0.00443	0.86
	+300%	0.00388	0.00362	0.931	0.00429	0.84
	+900%	0.00385	0.00392	1.017	0.00463	0.85
5	-80%	0.00147	0.00153	1.034	0.02982	0.05
	-50%	0.00172	0.00173	1.010	0.02855	0.06
	0	0.00163	0.00163	1.000	0.02784	0.06
	+100%	0.00174	0.00175	1.005	0.02774	0.06
	+300%	0.00169	0.00167	0.985	0.02888	0.06
	+900%	0.00159	0.00191	1.199	0.03052	0.06

Tabella E.7: Errore logico del MWPM quando il modello che ne pesa gli archi attribuisce all'errore di misura il valore $p(1+\delta)$ anziché quello vero. La colonna “costo” è il rapporto fra l'errore logico con modello sbagliato e quello con modello esatto; “rete/MWPM” confronta il decoder appreso con il matching mis-calibrato — valori inferiori a 1 indicano che la rete è peggiore. Errore fisico reale $p = 3 \times 10^{-3}$, 4×10^5 campioni per configurazione.

L'esito è netto in entrambe le direzioni. Il MWPM si rivela **notevolmente robusto**: sbagliando l'errore di misura di un fattore cinque in difetto o dieci in eccesso, l'errore logico cambia tipicamente dell'1-3%, e in due configurazioni *migliora* leggermente. L'unico caso in cui la mis-calibrazione morde davvero è $d = 5$ con $\delta = +900\%$, dove il costo sale al 20%: sovrastimare di un ordine di grandezza l'errore di misura induce il matching a preferire spiegazioni temporali degli eventi rilevati anche quando la spiegazione corretta è spaziale. Ma occorre appunto un errore di calibrazione di un ordine di grandezza perché l'effetto sia visibile.

La rete, dal canto suo, **non batte il matching in nessuna delle dodici configurazioni**, neppure quelle in cui esso lavora sul modello sbagliato. A $d = 3$

resta indietro del 15–25%; a $d = 5$ è peggiore di un fattore venti, in coerenza con il crollo osservato nell’esperimento precedente.

Che cosa questo precisa del criterio. Il risultato è negativo, ma corregge un’imprecisione nel modo in cui il criterio della tesi era stato finora formulato. Si era detto che il modello appreso produce valore dove il metodo analitico è “cieco” rispetto a una parte dell’informazione; questo esperimento mostra che la cecità rilevante non è quella dei *parametri* ma quella della *struttura*. Un modello con i parametri sbagliati continua a rappresentare le stesse classi di errore, e il matching che ne deriva resta quasi ottimale perché la geometria del problema — quali eventi rilevati siano plausibilmente connessi — non cambia. Un modello che non contempla affatto una classe di errore, come il crosstalk fra qubit di dato adiacenti dell’esperimento E2, sbaglia invece la geometria stessa, e lì il margine esiste. La formulazione corretta del criterio è dunque: *l’apprendimento paga quando la classe di errore non è rappresentabile nella struttura del decoder, non quando è rappresentata con parametri imprecisi.*

Ne discende anche un’indicazione pratica di segno opposto a quella che ci si potrebbe attendere: investire in una calibrazione più accurata del modello di rumore, entro un ordine di grandezza, non produce guadagni apprezzabili sulla decodifica. È un’osservazione che vale la pena tenere presente perché la calibrazione periodica è un costo operativo reale nei dispositivi attuali.

E.1.7 E7 — e se si scegliesse semplicemente un decoder analitico migliore?

Resta l’obiezione più severa che si possa muovere a tutto l’impianto della sezione. Il MWPM è lo standard di fatto, ma non è il miglior decoder analitico disponibile: esso richiede che ogni meccanismo di errore sia decomponibile in archi di un grafo, e il crosstalk fra qubit di dato adiacenti viola precisamente questa ipotesi. Un decoder che non la richieda — **BP+OSD**, ossia *belief propagation* seguita da *ordered statistics decoding*, che opera direttamente sulla matrice di parità del detector error model [54, 55] — potrebbe catturare da solo il margine attribuito all’apprendimento. Se così fosse, il risultato di E4 non misurerebbe il valore di un modello appreso ma il costo di una scelta subottimale del decoder di riferimento.

Il confronto è condotto sugli stessi circuiti e con la stessa mis-informazione: entrambi i decoder analitici ricevono il DEM *nominale*, privo di crosstalk, che è

quanto un decoder reale avrebbe a disposizione. La significatività è valutata con il test di McNemar, appaiato sugli stessi shot.

d	p_{ct}	MWPM	BP+OSD	BP+OSD/MWPM	ibrido/MWPM	McNemar
3	0	0.00428	0.00374	1.146	0.993	1×10^{-9}
	5×10^{-3}	0.03193	0.03286	0.972	1.185	3×10^{-7}
	1×10^{-2}	0.05912	0.06034	0.980	1.202	5×10^{-7}
	2×10^{-2}	0.11176	0.11340	0.986	1.213	2×10^{-6}
	4×10^{-2}	0.20811	0.20826	0.999	1.189	$p = 0.77$
5	0	0.00192	0.00143	1.339	1.000	2×10^{-7}
	5×10^{-3}	0.03565	0.02807	1.270	1.011	2×10^{-113}
	1×10^{-2}	0.08638	0.07348	1.176	1.033	3×10^{-140}
	2×10^{-2}	0.19998	0.18288	1.094	1.029	6×10^{-108}
	4×10^{-2}	0.37561	0.36788	1.021	1.005	2×10^{-12}

Tabella E.8: Confronto fra il MWPM, il decoder BP+OSD e il decoder ibrido appreso, tutti sul medesimo modello di rumore nominale. Le due colonne di guadagno sono rapporti rispetto al MWPM: valori superiori a 1 indicano un errore logico minore. In grassetto, per ciascuna riga, il migliore fra i due approcci alternativi. Il valore di significatività si riferisce al confronto BP+OSD contro MWPM. Errore fisico $p = 3 \times 10^{-3}$, 2×10^5 campioni per configurazione.

Il primo dato da registrare è che **BP+OSD è effettivamente un decoder migliore**: sul rumore nominale, quello per cui il modello è corretto, riduce l'errore logico del 14.6% a distanza 3 e del 33.9% a distanza 5, con significatività schiacciante. Il vantaggio cresce con la distanza, come atteso: più il codice è grande, più numerose sono le configurazioni di errore degeneri che il matching, vincolato alla decomposizione in archi, non sa distinguere.

Il secondo dato è però quello che conta, ed è la struttura delle due colonne di guadagno letta lungo la colonna p_{ct} .

*Il guadagno di BP+OSD **decresce** al crescere del crosstalk; quello del decoder ibrido **cresce**.*

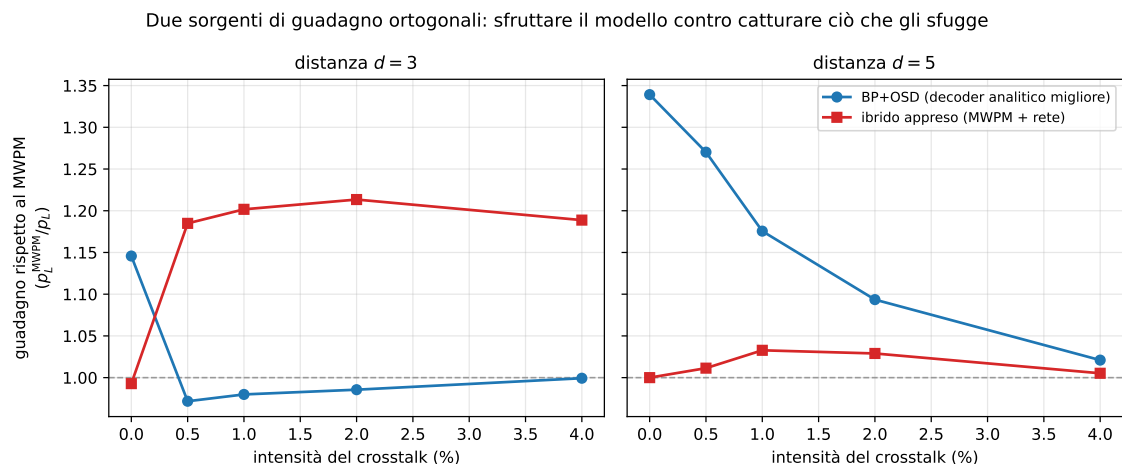


Figura E.5: Guadagno rispetto al MWPM dei due approcci alternativi, in funzione dell'intensità del crosstalk. La linea tratteggiata è il MWPM stesso. BP+OSD (blu) parte dal guadagno massimo, quando il modello di rumore che riceve è corretto, e lo perde man mano che il crosstalk lo rende incompleto; il decoder ibrido (rosso) parte da zero, perché a modello corretto non ha nulla da aggiungere, e guadagna in misura crescente. A distanza 3 le due curve si incrociano immediatamente; a distanza 5 il vantaggio iniziale di BP+OSD è così ampio da mantenerlo davanti anche eroso. Figura rigenerabile da `figure_src/gen_confronto_decoder.py`.

A distanza 5 il vantaggio di BP+OSD scende monotonamente da 1.339 a 1.021 mentre il crosstalk aumenta; a distanza 3 esso si annulla immediatamente, e il decoder diventa lievemente *peggiore* del matching. Specularmente, il guadagno dell'ibrido è nullo in assenza di crosstalk e sale fino a 1.213. I due metodi non competono sullo stesso asse: **BP+OSD sfrutta meglio il modello che riceve, il decoder appreso cattura ciò che al modello sfugge**. Il primo rende al massimo quando il modello è corretto e si degrada man mano che diventa incompleto — perché fidarsi più a fondo di una descrizione sbagliata è un danno, non un vantaggio; il secondo rende zero quando il modello è corretto, perché non c'è nulla da aggiungere, e cresce esattamente nella misura in cui il modello si allontana dal dispositivo.

Ne discende una conclusione più articolata di quella che l'esperimento si proponeva di verificare, e va enunciata riconoscendo entrambi i suoi lati. Da un lato l'obiezione è **in parte fondata**: a distanza 5, dove il vantaggio di partenza di BP+OSD è ampio, esso resta superiore all'ibrido in tutte le configurazioni, e adottare un decoder analitico migliore rende più che addestrare una rete. Chi disponga di un budget di ingegneria limitato dovrebbe spenderlo lì per primo, ed è una raccomandazione pratica che questa tesi non avrebbe potuto formulare senza l'esperimento. Dall'altro lato l'obiezione **non elimina il risultato**: a distanza 3 l'ibrido guadagna 18–21%

proprio dove BP+OSD perde.

Resta da chiarire che cosa significhi “agire su assi ortogonali”. L’inferenza immediata è che i due guadagni siano *sommabili*, e che costruire il residuo appreso sopra BP+OSD anziché sopra il matching debba produrre il prodotto dei due — ossia, a distanza 5 e crosstalk 5×10^{-3} , un guadagno di $1.289 \times 1.010 = 1.302$. È una previsione quantitativa, e l’esperimento successivo la mette alla prova anziché darla per buona.

E.1.8 E8 — le due sorgenti si sommano davvero? E10 — il pavimento comune

La previsione e il suo esito. L’esperimento E8 sostituisce il decoder di base della (E.1): il residuo appreso viene costruito sopra BP+OSD anziché sopra il matching, e i quattro bracci — MWPM, BP+OSD, ciascuno con e senza residuo — sono misurati *sui medesimi campioni*, il che rende ogni confronto appaiato ed elimina il disallineamento fra le campagne precedenti. Le due ipotesi in gioco erano dichiarate prima dell’esecuzione: quella *moltiplicativa*, per cui il combinato approssima il prodotto; e quella di *saturazione*, per cui il residuo trova sopra BP+OSD assai meno di quanto trovasse sopra il matching, perché una parte di ciò che recuperava era semplicemente il costo di un decoder analitico subottimale. La seconda è l’ipotesi sfavorevole al risultato di questa sezione, ed è quella che i dati sostengono.

d	p_{ct}	BP+OSD	MWPM+res	combinato	prodotto previsto
3	0	1.181	1.079	1.190	1.274
3	0.005	0.985	1.192	1.183	1.173
3	0.01	0.982	1.205	1.203	1.183
3	0.02	0.985	1.211	1.200	1.193
3	0.04	0.997	1.184	1.177	1.180
5	0	1.180	1.000	1.180	1.180
5	0.005	1.289	1.010	1.289	1.302
5	0.01	1.196	1.026	1.200	1.228
5	0.02	1.093	1.028	1.104	1.123
5	0.04	1.024	1.007	1.024	1.032

Tabella E.9: Esperimento E8. Tutti i guadagni sono rapporti $p_L^{\text{MWPM}}/p_L^{\text{metodo}}$ misurati sui medesimi 8×10^5 campioni per configurazione, 2×10^5 dei quali di test. “Combinato” è il residuo appreso costruito sopra BP+OSD; “prodotto previsto” è il valore che l’ipotesi moltiplicativa richiede. Il combinato non lo raggiunge mai quando entrambi i componenti hanno un guadagno reale, e coincide invece con il migliore dei due singoli.

Il confronto decisivo è nelle righe in cui *entrambi* i componenti guadagnano. A distanza 3 ve n’è una sola, quella senza crosstalk: il prodotto richiede 1.274, il combinato si ferma a 1.190, ossia esattamente il guadagno di BP+OSD da solo — e il residuo, sopra BP+OSD e senza crosstalk, ribalta lo 0.04% delle predizioni con $p = 0.58$, cioè non trova nulla. Nelle altre righe di quel blocco il “prodotto” è soddisfatto soltanto perché degenerare: con BP+OSD a 0.98 esso coincide per costruzione con il guadagno del residuo. A distanza 5, dove BP+OSD conserva un vantaggio ampio anche sotto crosstalk, tutte e cinque le righe sono informative e tutte e cinque smentiscono la previsione: lo scarto dal prodotto è negativo ovunque, da -0.007 a -0.027 , e il residuo migliora BP+OSD in modo statisticamente significativo in *una sola* configurazione su cinque, e ivi dell’uno per cento.

Il decoder combinato eguaglia sempre il migliore dei due singoli, mai il loro prodotto. Le due sorgenti di guadagno non sono sommabili: sono alternative.

Perché: esiste un pavimento comune. L’esperimento E10 individua la ragione. Sei bracci sono misurati sui medesimi campioni a distanza 3, in ordine crescente di informazione utilizzata: il decoder *banale* che predice sempre l’assenza di flip logico, la rete usata *da sola* senza alcun modello analitico, il MWPM, BP+OSD, e i due residui costruiti sui due decoder analitici.

p_{ct}	banale	MWPM	BP+OSD	rete sola	MWPM+res	BP+OSD+res	disp.
0	0.04863	0.00432	0.00367	0.00431	0.00405	0.00354	19.4%
0.005	0.08369	0.03106	0.03195	0.02704	0.02676	0.02677	1.0%
0.01	0.11611	0.05928	0.06035	0.04965	0.04885	0.04982	2.0%
0.02	0.17464	0.11169	0.11292	0.09222	0.09108	0.09156	1.2%
0.04	0.26492	0.20753	0.20819	0.17565	0.17413	0.17536	0.9%

Tabella E.10: Esperimento E10: errore logico dei sei bracci a distanza 3, tutti sugli stessi 8×10^5 campioni. La colonna “disp.” è la dispersione relativa fra i tre bracci di fondo — rete sola, MWPM+residuo, BP+OSD+residuo. In presenza di crosstalk essi convergono entro il 2% pur partendo da metodi radicalmente diversi; in sua assenza divergono del 19% e si ordinano per qualità intrinseca del decoder.

Il quadro si legge in due regimi, ed è la struttura stessa del risultato.

A *modello corretto* (prima riga) il pavimento non esiste: i metodi si ordinano per qualità, BP+OSD davanti a tutti con 3.54×10^{-3} e la rete sola ultima con 4.31×10^{-3} , e la dispersione fra i bracci di fondo è del 19%. Qui l’unica cosa che conta è quanto bene si sfrutti il modello, e il residuo appreso non ha nulla da aggiungere.

A *modello strutturalmente incompleto* (le altre quattro righe) i tre bracci di fondo collassano entro lo 0.9–2.0% l’uno dall’altro, mentre i due decoder analitici da cui provengono restano il 16–18% sopra. Vi arriva anche la rete usata da sola, che non dispone di alcun decoder analitico: il pavimento non è dunque una proprietà del metodo di decodifica, ma **dell’informazione contenuta nella sindrome**. Sotto crosstalk ciascuno di questi metodi la estrae per intero, e incontra lo stesso limite.

L’onestà richiede una precisazione sulla parola “coincidono”. Il test di McNemar fra la rete sola e il MWPM+residuo dà $p = 7.8 \times 10^{-4}$, 8.2×10^{-4} e 2.6×10^{-3} ai tre livelli superiori di crosstalk: con 2×10^5 campioni di test si risolve anche uno scarto dell’uno per cento, e quelle differenze residue sono *reali*. Ciò che si afferma è più debole e più preciso: i tre metodi atterrano entro l’1–2% l’uno dall’altro, e tale scarto è un ordine di grandezza inferiore al divario che separa tutti e tre dal decoder analitico da cui partono.

Il fallimento di E8 discende allora da E10 senza bisogno di ulteriori ipotesi: impilare i due metodi non porta sotto il pavimento, perché il pavimento non appartiene a nessuno dei due.

Il pavimento è del problema o del modello? (E11) Alla lettura appena data si può muovere un’obiezione che va presa sul serio, perché è la stessa che questa sezione muove agli altri risultati. I tre bracci che convergono contengono *tutti e tre la medesima rete*, un percettrone (128, 64) addestrato sugli stessi 4.8×10^5 campioni:

che convergano potrebbe significare soltanto “questo è quanto estrae questa rete”, e il pavimento sarebbe una proprietà del modello, non del problema. Per distinguere le due letture occorrono metodi più potenti di quel percettrone. L’esperimento E11 ne mette due alla prova, a distanza 3 e sui medesimi campioni.

Il primo è una rete con la stessa ricetta ma architettura (512, 256, 128), ossia circa quaranta volte i parametri. Il secondo è il **decoder empirico a massima verosimiglianza**: per ogni sindrome osservata in addestramento si predice l’osservabile di maggioranza, con ricaduta sul MWPM per le sindromi mai viste. Quest’ultimo non stima parametri — memorizza — e non ha dunque alcun limite di capacità.

p_{ct}	pavimento	rete 40×	residuo 40×	tabella empirica	copertura
0.005	0.02688	0.02687	0.02617	0.02765	98.1%
0.01	0.04935	0.04920	0.04850	0.05032	97.4%
0.02	0.09251	0.09222	0.09169	0.09499	95.6%
0.04	0.17602	0.17463	0.17377	0.18324	91.3%

Tabella E.11: Esperimento E11, distanza 3. “Pavimento” è il valore di E10 ricalcolato sugli stessi campioni. “Copertura” è la frazione di shot di test la cui sindrome era già stata osservata in addestramento, e misura quanto la tabella empirica sia effettivamente una tabella anziché un MWPM travestito.

L’obiezione è **respinta**: la rete con quaranta volte i parametri atterra sul pavimento entro lo 0.8%, e la tabella empirica — che di capacità ne ha infinita — risulta addirittura dal 2 al 4% *peggiore*. Il pavimento non è dunque il tetto dell’architettura scelta.

Sarebbe però scorretto concluderne che esso coincida con il limite informativo della sindrome, e la ragione sta nella tabella stessa. La sua copertura è alta (91–98%) ma la sua *profondità* non lo è: a crosstalk 4×10^{-2} le sindromi distinte osservate sono 63 180 su 4.8×10^5 campioni di addestramento, ossia circa sette osservazioni per sindrome, e un voto di maggioranza su sette campioni è in larga parte rumore. La tabella non fallisce perché l’informazione manchi, ma perché a questa numerosità non riesce a stimarla; essa mette dunque alla prova l’ipotesi della capacità — e la scarta — senza stabilire alcun limite superiore informativo. Nella stessa direzione va notato che il residuo costruito sulla rete grande scende dell’1–3% sotto il pavimento in tutti e quattro i punti, in modo piccolo ma sistematico: il pavimento non è perfettamente duro.

L’enunciato che i dati sostengono è pertanto il seguente, e va tenuto entro questi limiti: *il pavimento è robusto a un aumento di quaranta volte della capacità del modello*

e a un'alternativa non parametrica, e non è quindi un artefatto dell'architettura; se esso coincida con il limite informativo della sindrome resta una questione aperta, che solo una decodifica esatta a massima verosimiglianza potrebbe chiudere.

E.1.9 E5 — il decoder è trasferibile fra dispositivi?

Gli esperimenti precedenti addestrano e valutano il decoder sul *medesimo* profilo di rumore. È una scelta che lascia aperta l'obiezione più concreta che si possa muovere a un decoder appreso: se il modello funzionasse soltanto sul profilo su cui è stato addestrato, andrebbe riaddestrato per ogni esemplare di QPU e a ogni deriva di calibrazione, e il costo pratico ne annullerebbe il vantaggio. La domanda decisiva è dunque se la rete abbia appreso una *regola* o le idiosincrasie di un profilo.

La verifica consiste nell'addestrare su un'intensità di rumore correlato e valutare su un'altra, senza alcun riaddestramento. Il confronto ha due termini di riferimento: il MWPM, che non si addestra e resta quindi invariante, e la rete addestrata sullo stesso profilo di test, che rappresenta il limite superiore ottenibile conoscendo perfettamente il dominio.

Addestrata su	Valutata su p_{ct}			
	5×10^{-3}	10^{-2}	2×10^{-2}	4×10^{-2}
5×10^{-3}	<i>0.02804</i>	0.05000	0.09684	0.18749
10^{-2}	0.02800	<i>0.04912</i>	0.09492	0.18140
2×10^{-2}	0.02814	0.04915	<i>0.09286</i>	0.17800
4×10^{-2}	0.02841	0.04878	0.09312	<i>0.17643</i>
MWPM (nessun addestramento)	0.03232	0.05742	0.11172	0.20853

Tabella E.12: Errore logico dell'ibrido al variare del profilo di addestramento e di quello di valutazione. In corsivo la diagonale (addestramento e test sullo stesso profilo), in grassetto i due casi in cui una rete *trasferita* supera quella addestrata in casa. Tutte e dodici le configurazioni fuori diagonale battono comunque il MWPM.

L'esito è netto: **tutte e dodici** le reti trasferite mantengono il vantaggio sul decoder analitico, e il costo del trasferimento — il peggioramento medio rispetto alla rete addestrata sul profilo di test — è dell'1.6%. In due casi la rete trasferita risulta addirittura migliore di quella addestrata in casa, differenza che rientra nella variabilità statistica ma che conferma l'assenza di una dipendenza sistematica dal profilo di addestramento.

La regola appresa non è dunque il profilo di rumore, ma qualcosa di più generale: verosimilmente la struttura delle configurazioni di sindrome che il matching interpreta

male, la quale dipende dalla geometria del codice e dalla natura correlata dell'errore più che dalla sua intensità. Ne discende che il modello non richiede riaddestramento a ogni ricalibrazione del dispositivo — il che rimuove l'ostacolo pratico principale all'impiego di un decoder appreso, e distingue nettamente questo risultato da quello della prima fase, dove il classificatore richiedeva una fase di addestramento dedicata proprio per essere poi giudicato inutile.

E.1.10 Bilancio

I sette esperimenti convergono su una conclusione che è più informativa di un semplice confronto fra decoder, e che i due controlli finali hanno reso più precisa anziché più debole. **Sostituire** il decoder analitico con una rete non conviene: a parità di modello di rumore la rete pareggia a distanza 3 — al prezzo di 10^5 – 10^6 campioni — e perde al crescere della distanza. **Affiancarlo** sì: un decoder ibrido in cui la rete impara soltanto *quando ribaltare* la decisione del matching riduce l'errore logico del 18–21% a distanza 3 e dell'1–3% a distanza 5 in presenza di rumore correlato, e ricade sul matching quando il rumore è quello previsto.

Il guadagno compare dunque a due condizioni congiunte, e gli esperimenti E6 ed E7 permettono di enunciarle con precisione. La prima è che il rumore *esca dalla struttura* del modello del decoder — non che il modello sia mal calibrato: perturbandone i parametri fino a un ordine di grandezza l'errore logico varia dell'1–3%, e la rete non vince in nessuna delle dodici configurazioni. La seconda è che il reticolo resti *rappresentabile* dal modello appreso: a distanza 7, su 336 detector trattati come dimensioni indipendenti, l'ibrido si astiene dall'intervenire in tutte e cinque le configurazioni — anche là dove dispone di oltre duecentomila esempi positivi, sicché il limite è di architettura e non di numerosità (Tabella E.5).

A queste due condizioni se ne aggiunge una terza, emersa dal confronto con BP+OSD, che riguarda la scelta del termine di paragone. Il guadagno dell'ibrido va misurato contro il *miglior* decoder analitico disponibile, non contro quello standard: a distanza 5 BP+OSD recupera assai più margine dell'ibrido, e chi disponga di un budget di ingegneria limitato dovrebbe spenderlo lì per primo. Le due sorgenti di guadagno restano ortogonali — quella analitica decresce al crescere del rumore correlato, quella appresa cresce — ma non per questo sommabili: E8 ed E10 mostrano che entrambe conducono al medesimo pavimento, e che impilarle non porta al di sotto di esso. La costruzione corretta è dunque *sceglierne una in base al regime*, non cumularle: il miglior decoder analitico dove il modello è corretto, il residuo appreso

dove non lo è — e in quest’ultimo caso quale sia il decoder analitico sottostante cessa di contare.

Il perimetro va dichiarato. L’architettura impiegata è una rete densa di capacità modesta, deliberatamente scelta per continuità con lo strumento della prima fase sperimentale: i decoder allo stato dell’arte adottano architetture ricorrenti e convoluzionali che sfruttano la località del codice e la struttura temporale dei cicli di sindrome [17], e i risultati qui riportati non vanno letti come una stima del potenziale della decodifica appresa, ma come una verifica del *principio* su un’istanza trattabile. Le distanze esplorate arrivano a $d = 7$ — dove, come mostrato, il vantaggio si annulla — l’errore fisico è fissato a un solo valore sotto soglia, e il rumore correlato modellato resta di tipo Pauli: gli errori coerenti e il leakage discussi nella Sezione 9.5 costituiscono un asse ulteriore, sul quale ci si attende che il vantaggio del decoder appreso sia maggiore, non minore, poiché è lì che il modello analitico è più lontano dalla fisica.

Resta il fatto metodologico che collega questa sezione al Capitolo 7 e che il Capitolo 10 riprenderà: in entrambe le fasi sperimentali di questo lavoro il **complemento a basso costo ha battuto la sostituzione ambiziosa**. Nella prima, la ricerca multi-candidato TOP-4 ha battuto il classificatore che avrebbe dovuto filtrare l’output; nella seconda, una rete che si limita a segnalare quando il decoder sbaglia batte la rete che pretende di sostituirlo. L’apprendimento automatico non è né inutile né risolutivo: produce valore esattamente dove dispone di informazione che nessun metodo più semplice è in grado di sfruttare.

E.2 Compilazione Consapevole della Struttura: un’Esplorazione

Questa appendice esamina due verifiche esterne al confronto principale M1–M2: la sensibilità alla mappatura su una topologia eterogenea (M11/M11b) e il comportamento della ricerca TOP-4 nel proxy per gate (M13). Entrambe sono analisi esplorative. Il loro scopo è verificare se una metrica disponibile prima dell’esecuzione o una regola disponibile dopo la misura conservino potere informativo; non dimostrare prestazioni su una QPU reale. Il Capitolo [efchap:integrazione](#) ne riporta la sintesi.

E.3 Previsione e selezione non sono la stessa domanda

M11 usa la snapshot offline **FakeSherbrooke** distribuita con Qiskit IBM Runtime. Le proprietà `target.error` sono interpretate come *average gate infidelity* (AGI) complessiva e convertite in un canale depolarizzante con la stessa AGI. Il readout, esposto dalla snapshot come un solo errore medio, viene modellato simmetricamente. Le direzioni native ECR sono conservate e ogni ECR compilato deve avere una calibrazione esplicita; `rz` è virtuale.

Non viene aggiunto un secondo canale T_1/T_2 . L'AGI del gate è già una misura complessiva della sua non idealità e sommarvi il rilassamento ricavato dalla stessa snapshot produrrebbe un doppio conteggio non quantificato. Di conseguenza M11 non separa il contributo della decoerenza: studia l'eterogeneità della snapshot attraverso AGI e readout.

Il punteggio di calibrazione di un circuito compilato è

$$s = 1 - \prod_{g \in C} (1 - e_g), \quad (\text{E.2})$$

dove e_g è l'errore target dell'istruzione fisica o della misura. Un punteggio minore indica una fedeltà nominale maggiore. La correlazione fra s e successo risponde alla domanda *predittiva*; scegliere il layout con s minimo e valutarlo su dati non usati per la scelta risponde alla domanda *decisionale*. Le due non vanno confuse.

E.4 M11: pilota sui layout

Si campionano 60 sottografi connessi di 12 qubit mediante crescita casuale. Il campionamento non è uniforme sullo spazio dei sottografi: l'esperimento resta quindi esplorativo e le incertezze non descrivono l'intera topologia Sherbrooke. Su ciascun sottografo il circuito $N = 15$, $a = 7$ viene compilato in `sx/rz/x/ecr` con direzioni ECR native, livello di ottimizzazione 3 e seed 42.

Gli 8192 shot per layout sono divisi in otto batch con seed comuni. Quattro batch costituiscono la partizione di selezione e quattro l'holdout. Si confrontano due regole: layout con punteggio s minimo e layout con successo massimo sulla partizione di selezione; entrambe sono valutate soltanto sull'holdout. Questo evita di chiamare “migliore” il massimo osservato sugli stessi dati usati per stimarne la prestazione.

Voce	Contratto M11
backend	snapshot offline FakeSherbrooke, 127 qubit
layout richiesti	60 sottografi connessi, random-growth non uniforme
circuito	$N = 15$, $a = 7, 8$ qubit di conteggio
base / ottimizzazione	sx/rz/x/ecr , livello 3, seed 42
campionamento	8192 shot, 8 batch, split 4/4
inferenza	Spearman + bootstrap 2000 sui layout

Tabella E.13: Contratto preregistrato di M11. La snapshot è riproducibile tramite hash di calibrazione; non è una misura live di un dispositivo al momento dell'esecuzione.

Stima	Interpretazione ammessa
Spearman $s-P_{\text{holdout}}$	associazione di rango sul campione random-growth
P_{holdout} , minimo s	resa della regola basata solo sulla calibrazione
P_{holdout} , massimo train	resa di una selezione empirica su dati separati
differenza holdout	confronto esplorativo tra le due regole, non oracolo globale

Tabella E.14: La correlazione valuta capacità predittiva; il confronto holdout valuta una decisione. Nessuna riga autorizza un claim causale sull'hardware.

Dei 60 layout richiesti, 40 producono un modello di rumore valido. I 20 fallimenti non sono simulazioni con successo nullo: la snapshot assegna ad almeno un gate usato un'AGI pari a 1, valore fuori dal dominio della conversione depolarizzante adottata, e quei layout vengono pertanto esclusi. Fra i layout validi, P_{holdout} varia da 0,297 a 0,516, con mediana 0,431.

La correlazione di rango tra punteggio di calibrazione e successo holdout è $\rho_s = -0,184$, con IC bootstrap al 95% $[-0,456; 0,124]$ (2000 ricampionamenti). L'intervallo include ampiamente lo zero: sul campione random-growth il punteggio non mostra una capacità predittiva conclusiva. Il layout selezionato dal minimo punteggio (ID 35) ottiene $P_{\text{holdout}} = 0,451$; quello selezionato dal massimo successo sulla partizione train (ID 48) ottiene 0,516 sull'holdout. La differenza descrittiva è quindi +0,065, ma non è accompagnata da un intervallo dedicato e non va generalizzata oltre questo pilota.

La snapshot `fake_sherbrooke` è identificata nel JSON dal proprio hash SHA-256, il cui prefisso è 04b6a48f. Il risultato completo è nel file `results_M11_pilota_v2_20260826_222137.json`.

E.5 M11b: readout effettivo dopo il routing

Una seconda verifica genera variazione entro 12 sottografi. Per ciascuno vengono proposti cinque *initial layout*: registri di conteggio inizialmente assegnati ai qubit con readout nominale basso o alto, layout scelto dal transpiler e due permutazioni casuali. Queste etichette non sono trattamenti. Il routing può spostare i qubit virtuali, perciò l'esposizione analizzata è l'errore medio di readout dei qubit **effettivamente misurati nel circuito finale**.

Voce	Contratto M11b
sottografi	12, ciascuno con cinque strategie descrittive
base / ottimizzazione	sx/rz/x/ecr , livello 2, seed 42
shot	8192 per strategia, otto batch, holdout 50%
esposizione	readout medio sui qubit misurati dopo routing
controlli	effetti fissi di sottografo, numero ECR, profondità
stima primaria	Spearman parziale readout–successo
incertezza	cluster bootstrap sui sottografi analizzabili, 2000 ricampionamenti

Tabella E.15: L'unità indipendente del bootstrap è il sottografo, non la singola strategia annidata.

Etichetta iniziale	Uso nell'analisi
init-readout-basso	genera una permutazione iniziale; non garantisce il readout finale
init-readout-alto	genera variazione nella direzione opposta
transpiler	lascia la scelta iniziale al transpiler
casuale-1/2	due permutazioni riproducibili di confronto

Tabella E.16: Le etichette sono strumenti per produrre variazione entro sottografo. Il predittore usato nel modello è sempre la proprietà osservata dopo il routing.

Dei 12 sottografi richiesti, nove risultano analizzabili per tutte le cinque strategie, per un totale di 45 osservazioni holdout. Nei tre sottografi esclusi la conversione del modello di calibrazione incontra un'AGI pari a 1, fuori dal dominio della parametrizzazione depolarizzante: anche in questo caso le righe escluse non rappresentano esecuzioni con successo nullo.

Dopo la rimozione degli effetti fissi di sottografo e il controllo per numero di ECR e profondità, la correlazione parziale di Spearman fra errore medio di readout effettivo e successo è $\rho_s = -0,605$. L'IC al 95% ottenuto con 2000 ricampionamenti

cluster a livello di sottografo è $[-0, 826; -0, 190]$. Nel campione osservato, a maggiore errore di lettura corrisponde quindi un successo inferiore anche entro sottografo; l'intervallo non include lo zero. Il risultato resta associativo, non causale, sia per il numero ridotto di cluster sia per la possibile presenza di proprietà topologiche non misurate correlate con readout, conteggio ECR e profondità.

Il risultato canonico è `results_M11b_ruoli_v2_20260827_115109.json`; l'analisi indipendente è `analysis_M11b_v2_20260827_115119.json` e vincola il sorgente all'hash SHA-256 `1c4b5b4fad1e2289...f41c196b42152215`.

E.6 M13: TOP-4 sotto il proxy per gate

M13 usa lo stesso circuito, lo stesso manifest e la stessa definizione di p_L di M8. Per ciascun punto esegue 200 repliche da 1024 shot e riusa ogni istogramma per tre endpoint: massa di successo per singola misura, TOP-1 e TOP-4. Il successo TOP-K è binario per replica: almeno uno dei candidati selezionati conduce ai fattori.

Un pareggio al confine del quarto posto rende la selezione non identificata. Il runner salva quindi: (i) una scelta pseudo-casuale riproducibile; (ii) un limite inferiore, vero solo quando ogni scelta ammissibile contiene un esito utile; (iii) un limite superiore, vero quando almeno una scelta ammissibile può contenerlo. Il contrasto primario fra $p_L = 0$ e il massimo preregistrato usa un intervallo Newcombe; l'equivalenza richiede che l'intero intervallo cada entro $\pm 0, 10$. Una seconda verifica usa un involuppo Wilson simultaneo su tutte le risoluzioni dei pareggi.

p_L	per misura	TOP-1	TOP-4 seeded	identificato
0	0,7512	0,805	1,000	[1,000; 1,000]
10^{-4}	0,7459	0,730	1,000	[1,000; 1,000]
5×10^{-4}	0,7377	0,690	1,000	[1,000; 1,000]
10^{-3}	0,7229	0,620	1,000	[1,000; 1,000]
$1,7 \times 10^{-3}$	0,7080	0,620	1,000	[1,000; 1,000]
2×10^{-3}	0,7003	0,650	1,000	[1,000; 1,000]
5×10^{-3}	0,6396	0,505	1,000	[1,000; 1,000]
10^{-2}	0,5652	0,645	1,000	[1,000; 1,000]
2×10^{-2}	0,4528	0,695	1,000	[1,000; 1,000]
5×10^{-2}	0,2979	0,760	1,000	[1,000; 1,000]
10^{-1}	0,2506	0,535	0,915	[0,855; 0,940]
2×10^{-1}	0,2469	0,245	0,670	[0,535; 0,860]

Tabella E.17: Endpoint estratti dall'artefatto M13 schema 2 del 26 agosto 2026. La colonna "identificato" riporta i limiti inferiore e superiore generati dai pareggi al confine del TOP-4.

Il contrasto primario preregistrato confronta il TOP-4 seeded a $p_L = 0$ con quello al massimo valore della griglia, $p_L = 0, 2$. La differenza stimata è 0,330 e

l'IC Newcombe 95% è $[0, 266; 0, 398]$. La regola di equivalenza richiedeva l'intero intervallo entro $\pm 0, 10$; la regola di discriminazione richiedeva invece che il limite inferiore della diminuzione superasse $0, 10$. L'esito preregistrato è una diminuzione discriminante: il TOP-4 non è equivalente sul dominio completo della griglia.

Le analisi di sensibilità confermano che la gestione dei pareggi è materiale. Se tutti i pareggi sono risolti nel senso più sfavorevole, la diminuzione stimata è $0, 465$ con IC $[0, 395; 0, 534]$; nel senso più favorevole è $0, 140$ con IC $[0, 095; 0, 195]$, non sufficiente per superare il margine con la stessa regola di discriminazione. L'inviluppo simultaneo Bonferroni-Wilson su tutte le risoluzioni ammissibili è $[0, 059; 0, 553]$ e non supporta equivalenza per tutte le risoluzioni dei pareggi. Il risultato va quindi letto così: TOP-4 è estremamente efficace fino a $p_L = 0, 05$ su $N = 15$, ma degrada in modo statisticamente visibile agli errori più alti della griglia.

E.7 Conclusione e limiti

M11 e M11b non sono un benchmark su hardware. Il simulatore e il punteggio derivano dalla stessa snapshot; questa coerenza rende il confronto controllato, ma può favorire la metrica nominale e non include deriva, crosstalk, leakage o errori coerenti. Il campionamento random-growth non è uniforme. M11b aumenta il controllo confrontando strategie entro lo stesso sottografo, ma resta osservazionale: readout, numero ECR e profondità possono essere correlati con variabili topologiche non misurate.

M13 risponde a una domanda diversa. Una strategia TOP-4 può restare efficace anche quando la massa utile per misura è fortemente degradata, soprattutto nell'istanza $N = 15$ con pochi picchi strutturali. Perciò TOP-4 deve essere riportato insieme alla metrica per misura e all'incertezza dei pareggi; da solo non misura la fedeltà dello stato quantistico e non può essere estrapolato a moduli maggiori.

Bibliografia

- [1] Peter W. Shor. “Algorithms for Quantum Computation: Discrete Logarithms and Factoring”. In: *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1994, pp. 124–134. DOI: 10.1109/SFCS.1994.365700.
- [2] Arjen K. Lenstra e Hendrik W. Lenstra Jr. “The Development of the Number Field Sieve”. In: *Lecture Notes in Mathematics*. Vol. 1554. Springer-Verlag, 1993. DOI: 10.1007/BFb0091537.
- [3] Ronald L. Rivest, Adi Shamir e Leonard Adleman. “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems”. In: *Communications of the ACM* 21.2 (1978), pp. 120–126. DOI: 10.1145/359340.359342.
- [4] John Preskill. “Quantum Computing in the NISQ Era and Beyond”. In: *Quantum* 2 (2018), p. 79. DOI: 10.22331/q-2018-08-06-79.
- [5] Unathi Skosana e Mark Tame. “Demonstration of Shor’s Factoring Algorithm for $N = 21$ on IBM Quantum Processors”. In: *Scientific Reports* 11 (2021), p. 16599. DOI: 10.1038/s41598-021-95973-w.
- [6] Craig Gidney e Martin Ekerå. “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits”. In: *Quantum* 5 (2021), p. 433. DOI: 10.22331/q-2021-04-15-433.
- [7] Michael A. Nielsen e Isaac L. Chuang. *Quantum Computation and Quantum Information*. 10th Anniversary Edition. Cambridge, UK: Cambridge University Press, 2000. ISBN: 978-1107002173.
- [8] Kristan Temme, Sergey Bravyi e Jay M. Gambetta. “Error Mitigation for Short-Depth Quantum Circuits”. In: *Physical Review Letters* 119.18 (2017). Articolo fondativo della Zero-Noise Extrapolation (ZNE): propone l’amplificazione controllata del rumore seguita da estrapolazione a zero rumore per stimare

l'output ideale di un circuito NISQ, p. 180509. DOI: 10.1103/PhysRevLett.119.180509.

- [9] Andrew M. Steane. “Error Correcting Codes in Quantum Theory”. In: *Physical Review Letters* 77.5 (1996), pp. 793–797. DOI: 10.1103/PhysRevLett.77.793.
- [10] Austin G. Fowler et al. “Surface Codes: Towards Practical Large-Scale Quantum Computation”. In: *Physical Review A* 86.3 (2012), p. 032324. DOI: 10.1103/PhysRevA.86.032324.
- [11] Qiskit contributors. *Qiskit: An Open-source Framework for Quantum Computing*. <https://qiskit.org>. 2023. DOI: 10.5281/zenodo.2573505.
- [12] Craig Gidney. “Stim: a fast stabilizer circuit simulator”. In: *Quantum* 5 (2021), p. 497. DOI: 10.22331/q-2021-07-06-497.
- [13] Oscar Higgott. *PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching*. arXiv:2105.13082. 2021. arXiv: 2105.13082.
- [14] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [15] Stéphane Beauregard. “Circuit for Shor’s Algorithm Using $2n + 3$ Qubits”. In: *Quantum Information and Computation* 3.2 (2003). arXiv:quant-ph/0205095. Ottimizzazione del circuito originale di Shor: riduce il numero di qubit da $O(n)$ a $2n + 3$ tramite addizione modulare in-place, pp. 175–185.
- [16] Raul Conchello Vendrell et al. *Plaquette: A hardware-aware design platform for fault-tolerant quantum computers*. arXiv:2607.08767. 2026. arXiv: 2607.08767.
- [17] Johannes Bausch et al. “Learning High-Accuracy Error Decoding for Quantum Processors”. In: *Nature* 635 (2024). AlphaQubit: decoder neurale sviluppato da Google DeepMind e Google Quantum AI. Supera i decoder analitici classici su hardware Sycamore a 241 qubit, pp. 834–840. DOI: 10.1038/s41586-024-08148-8.
- [18] Richard P. Feynman. “Simulating Physics with Computers”. In: *International Journal of Theoretical Physics* 21.6–7 (1982), pp. 467–488. DOI: 10.1007/BF02650179.
- [19] David Deutsch. “Quantum Theory, the Church-Turing Principle and the Universal Quantum Computer”. In: *Proceedings of the Royal Society of London. Series A* 400.1818 (1985), pp. 97–117. DOI: 10.1098/rspa.1985.0070.

- [20] William K. Wootters e Wojciech H. Zurek. “A Single Quantum Cannot be Cloned”. In: *Nature* 299 (1982), pp. 802–803. DOI: 10.1038/299802a0.
- [21] Mohan Sarovar et al. “Detecting crosstalk errors in quantum information processors”. In: *Quantum* 4 (2020), p. 321. DOI: 10.22331/q-2020-09-11-321.
- [22] Dorit Aharonov e Michael Ben-Or. “Fault-Tolerant Quantum Computation with Constant Error Rate”. In: *SIAM Journal on Computing* 38.4 (2008). Versione preliminare presentata a STOC 1997; preprint arXiv:quant-ph/9906129, pp. 1207–1282. DOI: 10.1137/S0097539799359385.
- [23] Peter W. Shor. “Scheme for Reducing Decoherence in Quantum Computer Memory”. In: *Physical Review A* 52.4 (1995), R2493–R2496. DOI: 10.1103/PhysRevA.52.R2493.
- [24] Giacomo Torlai e Roger G. Melko. “Neural Decoder for Topological Codes”. In: *Physical Review Letters* 119.3 (2017), p. 030501. DOI: 10.1103/PhysRevLett.119.030501.
- [25] Joschka Roffe. “Quantum error correction: an introductory guide”. In: *Contemporary Physics* 60.3 (2019), pp. 226–245. DOI: 10.1080/00107514.2019.1667078.
- [26] Daniel Gottesman. “Stabilizer Codes and Quantum Error Correction”. Tesi di dott. California Institute of Technology, 1997. arXiv: quant-ph/9705052.
- [27] A. Yu. Kitaev. “Fault-tolerant quantum computation by anyons”. In: *Annals of Physics* 303.1 (2003), pp. 2–30. DOI: 10.1016/S0003-4916(02)00018-0.
- [28] Wojciech Hubert Zurek. “Decoherence, einselection, and the quantum origins of the classical”. In: *Reviews of Modern Physics* 75.3 (2003), pp. 715–775. DOI: 10.1103/RevModPhys.75.715.
- [29] Lieven M. K. Vandersypen et al. “Experimental Realization of Shor’s Quantum Factoring Algorithm Using Nuclear Magnetic Resonance”. In: *Nature* 414 (2001), pp. 883–887. DOI: 10.1038/414883a.
- [30] Thomas Monz et al. “Realization of a Scalable Shor Algorithm”. In: *Science* 351.6277 (2016), pp. 1068–1070. DOI: 10.1126/science.aad9480.
- [31] Aditya Singh, Aakash Khochare e Subhadeep Palit. *Practical Challenges in Executing Shor’s Algorithm on Existing Quantum Platforms*. 2025. arXiv: 2512.15330.

- [32] Oded Regev. *An Efficient Quantum Factoring Algorithm*. 2023. arXiv: 2308.06572.
- [33] IBM Quantum. *IBM Quantum Roadmap*. <https://www.ibm.com/roadmaps/quantum/>. Roadmap aggiornata a marzo 2026; obiettivi dichiarati soggetti a modifica. 2026.
- [34] Lov K. Grover. “A Fast Quantum Mechanical Algorithm for Database Search”. In: *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 1996, pp. 212–219. DOI: 10.1145/237814.237866.
- [35] Charles H. Bennett et al. “Strengths and Weaknesses of Quantum Computing”. In: *SIAM Journal on Computing* 26.5 (1997), pp. 1510–1523. DOI: 10.1137/S0097539796300933.
- [36] Christof Zalka. “Grover’s Quantum Searching Algorithm is Optimal”. In: *Physical Review A* 60.4 (1999), pp. 2746–2751. DOI: 10.1103/PhysRevA.60.2746.
- [37] Dewang Sun, Naifeng Zhang e Franz Franchetti. “Optimization and Performance Analysis of Shor’s Algorithm in Qiskit”. In: *IEEE High Performance Extreme Computing Conference (HPEC)*. Available at https://users.ece.cmu.edu/~franzf/papers/hpec_2023_Quantum_final.pdf. IEEE, 2023.
- [38] IBM Quantum. *Shor’s Algorithm – IBM Quantum Documentation*. <https://quantum.cloud.ibm.com/docs/en/tutorials/shors-algorithm>. Tutorial ufficiale IBM per l’implementazione dell’algoritmo di Shor con Qiskit Runtime su hardware reale (`ibm_marrakesh`). Include Dynamical Decoupling e Gate Twirling come tecniche di soppressione del rumore. 2024.
- [39] Google Quantum AI. *Cirq: A Python Framework for Writing, Manipulating, and Running Quantum Circuits*. <https://quantumai.google/cirq>. Framework open-source Google per processori superconduttori Sycamore. 2024.
- [40] Ville Bergholm et al. *PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations*. arXiv:1811.04968. Framework Xanadu orientato a circuiti variazionali e quantum machine learning. 2022.
- [41] Graychii. *Shor Algorithm Implementation*. <https://github.com/Graychii/Shor-Algorithm-Implementation>. Implementazione Python dell’algoritmo di Shor su Qiskit. 2023.
- [42] Haoran Liao et al. *Machine Learning for Practical Quantum Error Mitigation*. 2023. arXiv: 2309.17368.

- [43] John S. Bell. “On the Einstein Podolsky Rosen Paradox”. In: *Physics* 1.3 (1964), pp. 195–200. DOI: 10.1103/PhysicsPhysiqueFizika.1.195.
- [44] Alain Aspect, Philippe Grangier e Gérard Roger. “Experimental Realization of Einstein-Podolsky-Rosen-Bohm Gedankenexperiment: A New Violation of Bell’s Inequalities”. In: *Physical Review Letters* 49.2 (1982), pp. 91–94. DOI: 10.1103/PhysRevLett.49.91.
- [45] Brian D. Josephson. “Possible New Effects in Superconductive Tunnelling”. In: *Physics Letters* 1.7 (1962), pp. 251–253. DOI: 10.1016/0031-9163(62)91369-0.
- [46] Jens Koch et al. “Charge-insensitive Qubit Design Derived from the Cooper Pair Box”. In: *Physical Review A* 76.4 (2007), p. 042319. DOI: 10.1103/PhysRevA.76.042319.
- [47] Fabrice Boudot et al. “Factoring RSA-250”. In: *Cryptology ePrint Archive* (2020). Report 2020/697, <https://eprint.iacr.org/2020/697>.
- [48] National Institute of Standards and Technology. *What Is Post-Quantum Cryptography?* <https://www.nist.gov/cybersecurity-and-privacy/what-post-quantum-cryptography>. Aggiornato il 27 febbraio 2026; consultato il 26 agosto 2026. 2026.
- [49] National Security Agency. *Commercial National Security Algorithm Suite 2.0 (CNSA 2.0)*. https://media.defense.gov/2022/Sep/07/2003071834/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS_.PDF. Cybersecurity Advisory, settembre 2022. Fissa le scadenze di migrazione verso algoritmi resistenti ai computer quantistici entro il 2030–2035. 2022.
- [50] National Institute of Standards and Technology. *Post-Quantum Cryptography Standards: FIPS 203, 204, 205*. <https://csrc.nist.gov/pubs/fips/203/final>. FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA). Pubblicazione formale agosto 2024. 2024.
- [51] Apple Security Engineering and Architecture. *iMessage with PQ3: The new state of the art in quantum-secure messaging*. <https://security.apple.com/blog/imessage-pq3/>. Febbraio 2024. Primo protocollo di messaggistica con crittografia post-quantistica di livello 3, basato su ML-KEM per lo scambio di chiavi ibride. 2024.

- [52] Frank Arute et al. “Quantum Supremacy Using a Programmable Superconducting Processor”. In: *Nature* 574 (2019), pp. 505–510. DOI: 10.1038/s41586-019-1666-5.
- [53] Johannes Bausch, Andrew W. Senior et al. *Learning to Decode the Surface Code with a Recurrent, Transformer-Based Neural Network*. Decoder transformer per surface code; supera i decoder state-of-the-art su hardware Google Sycamore per distanze 3–11. 2023. arXiv: 2310.05900.
- [54] Pavel Panteleev e Gleb Kalachev. “Degenerate Quantum LDPC Codes With Good Finite Length Performance”. In: *Quantum* 5 (2021), p. 585. DOI: 10.22331/q-2021-11-22-585.
- [55] Joschka Roffe et al. “Decoding Across the Quantum Low-Density Parity-Check Code Landscape”. In: *Physical Review Research* 2.4 (2020), p. 043423. DOI: 10.1103/PhysRevResearch.2.043423.

Ringraziamenti

Desidero ringraziare in primo luogo il mio relatore, **Ing. Floriano Caprio**, per la disponibilità, la pazienza e la guida preziosa fornita durante l'intero percorso di sviluppo di questo lavoro. I suoi consigli e la sua competenza sono stati fondamentali per orientarmi in un campo così vasto e in continua evoluzione.

Ringrazio l'**Università Campus Bio-Medico di Roma** per il percorso formativo offerto, che mi ha fornito gli strumenti teorici e pratici per affrontare questa tesi.

Un ringraziamento speciale va alla mia **famiglia**, che ha rappresentato un punto fermo e una fonte inesauribile di sostegno morale durante gli anni di studio.

Ringrazio infine tutti i **colleghi e amici** che hanno condiviso con me questo percorso, rendendo più leggere le giornate di studio e ricerca.

Claudio Dragotta

Roma, 2026